Reduced-Order Planning and Control for Legged Robot Navigation in Cluttered Environments

BY

Daniel Torres
BS, University of Illinois Chicago, 2020

DISSERTATION

Submitted as partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Mechanical Engineering
in the Graduate College of the
University of Illinois Chicago, 2026

Chicago, Illinois

Defense Committee:
Pranav Bhounsule, Chair and Advisor
Michael J. Scott
Daniel S. Seara
Abriana Stewart-Height
Ernesto Hernandez Hinojosa, University of Michigan

Copyright

© 2026 Daniel Torres

Accessibility Statement

This dissertation is available in an accessible EPUB format in addition to this PDF. Both versions are submitted to UIC's electronic thesis and dissertation system.

Dedication

To my wife,
the one person who inspires and motivates me every day and every single step of the way.

Acknowledgements

I am grateful to every person who has supported me on the way to and through this PhD.

To Mr. Kim, my fifth- and sixth-grade math teacher, who helped me discover my passion for learning and problem-solving.

To Dr. Carmen Lilley, for setting me on this journey and for her kindness and support throughout. To the UIC faculty and staff who helped along the way, including April, Crystal, Lauren, Lieke, and Benn Williams.

To all the UIC MIE staff who help run the whole show and make it possible for us students to focus on our research and studies: Alan, Runyu, Urszula, Sonya, and Wanda, most of whom have witnessed my journey from undergrad to PhD.

To my lab mates, past and present, who have been a source of support and friendship: Abhishek, Venkata, Jeremy, Francesco, Paul, Josh, Simone, Ragib, Safwan, Subramanian, Chun-Ming, Luca, Sal, Ernesto, and Jim.

To my advisor, Dr. Pranav Bhounsule, for his wisdom, guidance, and kindness. I am grateful for the opportunity to learn from him, not just academically but also personally.

To my family and friends, for their constant support along the way even if they always asked me when I was going to finish my PhD. I am grateful for their love and encouragement, and for helping me keep perspective on what is important in life.

And most of all, to my wife, who has helped me grow both personally and professionally. This journey would not have been possible without her love, support, and patience.

DT

Preface

This material is based upon work supported by the National Science Foundation Graduate Research Fellowship Program under Grant No. 2236870. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the National Science Foundation.

Daniel Torres
July 27, 2026

Author's Contribution

This dissertation includes material from three co-authored manuscripts, reproduced in Chapters 2, 3, and 4, and one co-authored study written for this dissertation, presented in Chapter 5. The contributions of each author to these works are described below.

Chapter 2: A significant portion of this chapter is adapted from a published paper with the following citation: [Torres, D., Hernandez Hinojosa, E., & Bhounsule, P. A. (2023). Control of a Bipedal Walking Using Partial Feedback Linearization and Gaussian Process Regression-Based of the Step-to-Step Map. In *Proceedings of the ASME 2023 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Volume 8: 47th Mechanisms and Robotics Conference (MR) (p. V008T08A063). Boston, MA, August 20–23, 2023. `https://doi.org/10.1115/DETC2023-116945`] in which Daniel Torres and Pranav A. Bhounsule contributed to conceiving the idea; Daniel Torres carried out the simulations, data collection, and analysis and wrote the manuscript; and Ernesto Hernandez Hinojosa and Pranav A. Bhounsule advised and edited it, with Ernesto Hernandez Hinojosa reviewing the Gaussian process models to ensure correct results. The partial feedback linearization and step-to-step Poincaré map formulation used as the basis for this work builds on prior publications by Pranav A. Bhounsule and Ernesto Hernandez Hinojosa [**14, 41**].

Chapter 3: A significant portion of this chapter is adapted from an accepted manuscript with the following citation: [Torres, D., & Bhounsule, P. A. (2026). Comparative Study of Local Planners for Quadrupeds Using BARN Environments. Accepted to the 6th Modeling, Estimation and Control Conference (MECC 2026), Phoenix, AZ, October 25–28, 2026.] in which Daniel Torres and Pranav A. Bhounsule contributed to conceiving the idea; Daniel Torres implemented and tuned the Potential Field, Dynamic Window Approach, and Model Predictive Control planners, carried out the simulation and hardware experiments on the Unitree A1, and wrote the manuscript; and Pranav A. Bhounsule provided the underlying motion control framework and hardware infrastructure and advised and edited it. The hierarchical control and planning framework used as the basis for this work was developed in prior publications by Pranav A. Bhounsule [**2, 15**].

Chapter 4: A significant portion of this chapter is adapted from an accepted manuscript with the following citation: [Torres, D., & Bhounsule, P. A. (2026). Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments. Accepted to the 6th Modeling, Estimation and Control Conference (MECC 2026), Phoenix, AZ, October 25–28, 2026.] in which Daniel Torres and Pranav A. Bhounsule contributed to conceiving the idea; Daniel Torres formulated the BiKE-Legs and DRiVE-Legs kinematic models and their integration into the Dynamic Window Approach, carried out the simulation studies and the Digit/MuJoCo validation, and wrote the manuscript; and Pranav A. Bhounsule developed the constrained inverse dynamics controller with task-space feedback and advised and edited it.

Chapter 5: This chapter was written for this dissertation and is not reproduced from a published or submitted manuscript. Daniel Torres, Safwan Mondal, and Pranav A. Bhounsule contributed to conceiving the idea; Daniel Torres implemented the simulation environments and the DWA weight-profile framework, trained the reinforcement learning policy and distilled it into the decision tree, carried out the 2D and MuJoCo quadruped experiments and the statistical analysis, and wrote the chapter; Safwan Mondal assisted with the reinforcement learning formulation; and Safwan Mondal and Pranav A. Bhounsule advised and edited it.

Contents

Copyright ii

Accessibility Statement iii

Dedication iv

Acknowledgements v

Preface vi

Author's Contribution vii

List of Tables xii

List of Figures xiii

List of Abbreviations xvii

Notations xviii
Chapter 2: Stepping Control for Bipedal Walking via Partial Feedback Linearization and Gaussian Process Regression. xviii
Chapter 3: Comparative Study of Local Planners for Quadrupeds Using BARN Environments. xix
Chapter 4: Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments. xix
Chapter 5: Interpretable Adaptive Dynamic Window Approach for Legged Robot Local Navigation. xx

Summary xxii

Chapter 1. Introduction 1
1.1. Motivation 1
1.2. Legged Robot Navigation as a Coupled Planning and Control Problem 2
1.3. Limitations of Conventional Local Planning for Legged Robots 3
1.4. Reduced-Order Models for Real-Time Legged Navigation 5
1.5. Planner Parameterization and Interpretability 6
1.6. Thesis Statement and Research Questions 7
1.7. Contributions 8
1.8. Dissertation Organization 9

Chapter 2. Stepping Control for Bipedal Walking via Partial Feedback Linearization and Gaussian Process Regression 11

2.1. Introduction 11
2.2. Background and Related Work 12
2.3. Model 15
2.4. Methods 18
2.5. Results 22
2.6. Discussion 26
2.7. Conclusions and Future Work 30

Chapter 3. Comparative Study of Local Planners for Quadrupeds Using BARN Environments 31
3.1. Introduction 31
3.2. Methods 34
3.3. Results and Discussion 39
3.4. Conclusion 45

Chapter 4. Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments 47
4.1. Introduction 47
4.2. Methods 50
4.3. Results 57
4.4. Conclusion and Future Work 60

Chapter 5. Interpretable Adaptive Dynamic Window Approach for Legged Robot Local Navigation 62
5.1. Introduction 62
5.2. Local Navigation Setup 66
5.3. Interpretable RL-DWA Framework 71
5.4. Evaluation and Explainability 80
5.5. Quadruped Transfer Study in MuJoCo 87
5.6. Conclusion 99

Chapter 6. Conclusion 102
6.1. Main Findings 103
6.2. Answers to the Research Questions 105
6.3. Limitations 106
6.4. Future Work 107
6.5. Closing Remarks 109

Appendix A. Copyright Permissions 111
Chapter 2: Stepping Control for Bipedal Walking via Partial Feedback Linearization and Gaussian Process Regression. 111
Chapter 3: Comparative Study of Local Planners for Quadrupeds Using BARN Environments. 111
Chapter 4: Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments. 112

Bibliography 113

Vita 119
Education 119
Research Interests 119
Academic Experience 119
Teaching Experience 119
Publications 120
Selected Presentations 120
Honors and Awards 120
Selected Service 121

List of Tables

3.1 **Simulation:** Planner performance and compute-time statistics across all environments. Success metrics are computed over all runs, while traversal time, path length, and minimum clearance are reported over successful runs only (mean ± standard deviation). 40

3.2 **Hardware:** Performance and compute-time statistics for BARN Env. 000, averaged over five runs. 44

4.1 Performance of BiKE-Legs (BL) and DRiVE-Legs (DL) across various obstacle size/fraction and DWA planning horizons. 58

5.1 Quantitative navigation performance across validation and test environments. Path length and time to goal are averaged over successful episodes only. 81

5.2 Generalization performance on harder held-out test environments. Path length and time to goal are averaged over successful episodes only. 81

5.3 PCA of the five BARN path-difficulty metrics: variance explained and component loadings. Loadings are shown for PC1–PC3 only. PC4 and PC5 are omitted because together they account for 2.8% of the variance. 89

5.4 Navigation performance over all 300 BARN environments in MuJoCo. Path length and time to goal are averaged over successful episodes only. 90

5.5 Paired statistical comparison over all 300 environments. All differences are computed as DST-DWA − Baseline DWA. 91

5.6 Density-stratified paired comparison (all 300 environments). 92

5.7 PC1-quartile-stratified paired comparison (all 300 environments). 92

5.8 Intensive margin: paired differences (DST-DWA − Baseline DWA) restricted to the 72 jointly successful environments. 93

5.9 Outcome archetypes over all 300 environments. 94

List of Figures

2.1 **Overview of the approach:** (a) PFL reduces the stance phase dynamics from $\Theta = [\Theta_u, \Theta_c]$ (10 dimensions) to $\Theta = \Theta_u$ (2 dimensions). (b) A Poincaré section is chosen at mid-stance. We generate random input state at the Poincaré section and controls at the step and simulate till the next Poincaré section to generate data for the Poincaré map given by $\mathbf{F}$, $\Theta_u^{i+1} = \mathbf{F}(\Theta_u^i, \mathbf{U}^i)$. (c) The Poincaré map is curve fitted $\Theta_u^{i+1} = \overline{\mathbf{F}}(\Theta_u^i, \mathbf{U}^i)$ where $\overline{F}$ is a Gaussian process regression model and Gaussian process classification is used to identify the boundary of the model. (d) Nonlinear programming is used to solve a suitably formulated quadratic program. For a video see [**16**]. 15

2.2 **Humanoid model:** (a) configuration variables describing the degrees of freedom, (b) mass, center of mass, inertia about center of mass, and length parameters. 16

2.3 A representation of the walking cycle. 18

2.4 Feature space of the test set. 23

2.5 Confusion matrix for the test set. 24

2.6 GPR performance for the test set. 25

2.7 Sinusoidal velocity tracking performance. 26

2.8 Velocity tracking percentage error. 27

2.9 Optimization and simulation velocity at every step in the stepping stones simulation. 28

2.10 Nominal control percent deviation at every step in the stepping stones optimization simulation. 29

3.1 Still-motion path of the robot in BARN Env. 000. The hardware trajectory is shown on top (obstacles are virtually enforced using a virtual camera), while the corresponding MuJoCo reconstruction from recorded hardware data with obstacles (white) is shown below. 33

3.2 (a) Block diagram of the robot controller. A state machine is used to program the robot controller logic. The inputs to the state machine are shown in the box with gray highlight, but only two inputs, the linear speed in fore-aft direction $\dot{x}_{\text{ref}} = v_x = v$ and yaw speed $\dot{\phi}_{\text{ref}} = \omega$ are used for control in this paper. (b) The single rigid body model is used for regulating force/torque on the body from the stance leg. Details are provided in [**2**]. 33

3.3 **Simulation:** Performance outcomes across BARN environments for all planners. Each point is one run; vertical position shows the outcome (collision, timeout, goal), and color indicates BARN score for successful runs. 40

3.4 **Simulation:** Representative BARN environments: (a) sparse with wide passages and (b) dense with narrow corridors. Dashed lines indicate collision. 41

3.5 **Hardware:** Velocity tracking error in BARN Env. 042 for (a) DWA, (b) MPC, and (c) PF. Each panel shows the difference between estimated and commanded linear and angular velocity, with mean absolute error (MAE) annotated for reference. 43

3.6 **Hardware:** Trajectory reproducibility across 5 runs on BARN Env. 000 for (a) DWA, (b) MPC, and (c) PF. All runs successfully reached the goal, with PF producing the most consistent paths and MPC exhibiting the highest path-length variability. 45

4.1 **Overview:** The BiKE-Legs model (bicycle kinematics enhanced with legs) is used along with the Dynamic Window Approach, a sampling-based approach for motion planning on the humanoid Digit in the MuJoCo simulator. 49

4.2 **Digit model:** The reduced lower-body model has 22 generalized coordinates: six floating-base coordinates and eight coordinates in each leg. Each leg has six actuated DoFs and two passive coordinates associated with the closed-chain leg mechanism. Figure adapted from **[20]**. 52

4.3 **Two kinematic models for planning (top-down view):** The center of mass is (x, y) and foot position is (x_f, y_f). The body frame shown using dashed lines is at an angle θ to the global x-axis (not shown). w is the half body width, the lateral offset from the center of mass to the body edge. In the BiKE-Legs, the **Bi**cycle **K**inematics **E**nhanced with **Legs** $\psi = \phi$ and the DRiVE-Legs, the **D**ifferential d**RiV**e **E**nhanced with **Legs** $\psi = \omega T$ with step time T. 55

4.4 **Motion Control:** Tracking velocity of the center of mass using constrained inverse dynamics with task-space feedback control. 60

4.5 Results for 60 obstacles, 0.2 density, 3-step horizon, with a maneuver cost, BiKE-Legs model in MuJoCo. 61

5.1 Overall proposed pipeline for interpretable RL-based DWA navigation. Local LiDAR-like observations, goal information, and robot motion states are converted into compact interpretable features. During RL-DWA execution, a PPO policy selects a discrete DWA weight profile, and the DWA planner computes the final feasible velocity command. After training, rollouts from the frozen RL-DWA policy are used to train a shallow decision tree that maps the same interpretable features to the RL-selected DWA profiles. The resulting DST-DWA policy replaces the neural policy during evaluation, enabling rule-based interpretation while retaining adaptive DWA weight selection. 69

5.2 Interpretable local feature representation used for RL-DWA policy learning and decision-tree distillation. The robot-centric LiDAR observations are summarized into sector-based clearance features in the front, front-left, front-right, left, and right regions, together with the minimum observed clearance. Additional compact features describe left–right and front-left–front-right clearance imbalance, goal-relative quantities such as distance-to-goal and heading error, and motion-state quantities such as linear and angular velocity. These human-readable local

features form the state input for both the PPO-based DWA weight-profile selector and the distilled decision-tree policy. 72

5.3 Training convergence curve showing success-rate progression across PPO training iterations. 78

5.4 Action-colored trajectory comparison in BARN Environment 12. The three panels show the closed-loop behavior of Baseline DWA, RL-DWA, and DST-DWA under the same start-goal configuration. Trajectory segments are colored according to the DWA weight profile selected at each control step: balanced, obstacle-safe, speed-focused, goal-focused, and cautious-balanced. Baseline DWA uses a constant balanced profile throughout execution, whereas RL-DWA and DST-DWA adaptively switch profiles based on the local navigation state. In this example, the fixed-weight planner collides, while both adaptive methods reach the goal. Each panel also prints a bold status label above it ("collision" or "goal reached"); that label's color may not meet minimum text-contrast requirements on its own, so the outcome is restated here in this caption. 83

5.5 Example decision tree for distilled DWA profile selection. Internal nodes represent threshold conditions on interpretable local features such as current speed, heading error, distance to goal, and local obstacle clearance. Leaf nodes correspond to selected DWA weight profiles, allowing each navigation mode decision to be traced through explicit feature-threshold rules. 84

5.6 PC1–PC3 loadings across the five BARN difficulty metrics. 90

5.7 All 300 environments in PC1/PC2 space colored by paired outcome (both succeed, both fail, or discordant), with quartile boundaries overlaid. Discordant pairs of both colors scatter across the full PC1 range. The "both succeed" marker (light green) is low-contrast against the plot background; that category corresponds to the 72 environments, discussed below, where both planners reached the goal. 93

5.8 Failure archetype counts per planner. Exact counts are given in Table 5.9 above. 95

5.9 DST-DWA's dominant mode in successes vs. stuck timeouts. 96

5.10 DST-DWA rescue (BARN Environment 12). Baseline DWA timed out stuck. DST-DWA reached the goal in 18.8 s, passing within 0.13 m of the baseline planner's failure point. Same corridor, different timing and mode. As with Figure 5.4, the bold status label above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here. 97

5.11 DST-DWA loss (BARN Environment 3). The canonical over-caution failure: DST-DWA committed to the obstacle-safe profile near the start and froze into a stuck timeout, while the fixed weights pushed straight through. As with Figure 5.4, the bold status label above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here. 98

5.12 Joint success, efficiency tail (BARN Environment 88). Both planners reached the goal, but DST-DWA wandered: 38.8 s and 14.6 m against Baseline DWA's 13.2 s and 4.4 m. The typical joint-success case shows near-identical performance (the medians in Table 5.8 are near zero). This example illustrates the cautious tail that drives the intensive-margin means. As with Figure 5.4, the bold status label

above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here. 99

5.13 Joint failure, different mechanisms (BARN Environment 293). Both planners were trapped by the same bottleneck but failed differently: Baseline DWA stalled and clipped an obstacle at near-zero speed (collision), while DST-DWA circled in obstacle-safe mode until timeout. As with Figure 5.4, the bold status label above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here. 100

List of Abbreviations

ARD: Automatic Relevance Determination
AT: Actual Traversal time
BARN: Benchmark for Autonomous Robot Navigation
BiKE-Legs: Bicycle Kinematics Enhanced with Legs (abbreviated BL in tables)
BVP: Boundary Value Problem
CoM: Center of Mass
DOF: Degree(s) of Freedom
DRiVE-Legs: Differential dRiVe Enhanced with Legs (abbreviated DL in tables)
DST-DWA: Decision-Tree-Distilled Dynamic Window Approach
DWA: Dynamic Window Approach
FL: Front Left (quadruped leg label)
FR: Front Right (quadruped leg label)
GPC: Gaussian Process Classification
GPR: Gaussian Process Regression
HZD: Hybrid Zero Dynamics
IMU: Inertial Measurement Unit
LCM: Lightweight Communications and Marshalling
LIPM: Linear Inverted Pendulum Model
LSTP: Long Short-Term Planning
MAE: Mean Absolute Error
MDP: Markov Decision Process
MPC: Model Predictive Control
OT: Optimal Traversal time
PCA: Principal Component Analysis
PF: Potential Field(s)
PFL: Partial Feedback Linearization
PPO: Proximal Policy Optimization
PRM: Probabilistic Roadmaps
RL: Reinforcement Learning; also Rear Left (quadruped leg label)
RL-DWA: Reinforcement-Learning-based Dynamic Window Approach
RR: Rear Right (quadruped leg label)
SF: Stretch Factor
SLIP: Spring-Loaded Inverted Pendulum
SLSQP: Sequential Least Squares Programming
UDP: User Datagram Protocol
UGV: Unmanned Ground Vehicle

Notations

Bold lowercase letters are used to denote the vectors and bold uppercase letters for matrices. The following mathematical notations are used throughout this thesis:

$\dot{x}, \ddot{x}$**:** the first and second time derivative of x
x^-, x^+**:** the value of x immediately before and after an instantaneous event (e.g., foot-strike)
x^{ref}, x_{ref}**:** the reference (desired) value of x
$\bar{x}$**:** the nominal value of x
x^i**:** the value of x at discrete step or time index i

Chapter 2: Stepping Control for Bipedal Walking via Partial Feedback Linearization and Gaussian Process Regression.

θ**:** vector of the five joint angles $[\theta_0, \theta_1, \theta_2, \theta_3, \theta_4]$
θ_0**:** torso angle relative to the vertical direction
θ_1, θ_2**:** thigh-link angles of the stance and swing leg, respectively
θ_3, θ_4**:** calf-link angles of the stance and swing leg, respectively
$\mathbf{q}$**:** generalized coordinates, including the floating-base position (x, y) and joint angles θ
$\mathbf{M}, \mathbf{N}, \mathbf{B}$**:** mass matrix, Coriolis/centrifugal/ gravity vector, and torque selection matrix
$\mathbf{u}$**:** vector of joint control torques
$\mathbf{J}_{C_1}, \mathbf{J}_{C_2}$**:** contact Jacobians at the stance and swing foot, respectively
$\mathbf{P}_{C_1}$**:** ground reaction force at the stance foot
$\mathbf{I}_{C_1}, \mathbf{I}_{C_2}$**:** impulsive forces at the stance and swing foot during foot-strike
Θ**:** state at the Poincaré section, $[\theta, \dot{\theta}]$
$\mathbf{U}$**:** discrete step controls (foot placement angle and push-off impulse)
$\mathbf{F}, \overline{F}$**:** the Poincaré (step-to-step) map and its Gaussian process regression approximation
α**:** foot placement (step) angle
I**:** impulsive push-off magnitude
K_p, K_d**:** proportional and derivative gains of the partial feedback linearization controller
m_i, c_i, J_i, ℓ_i**:** mass, center-of-mass distance, inertia, and length of link i
L**:** hyperparameter vector (length scales) of the Gaussian process kernel
ϵ**:** feasibility threshold separating feasible and infeasible steps in the Gaussian process classifier

Chapter 3: Comparative Study of Local Planners for Quadrupeds Using BARN Environments.

(x, y, θ)**:** unicycle position and heading angle
(v, ω)**:** commanded fore-aft linear velocity and yaw (turning) rate
h**:** discretization time step of the unicycle model
$\mathbf{p}, \mathbf{p}_g$**:** planar robot position and goal position
$\mathbf{q}_i, r_i$**:** center and radius of obstacle i
r_s**:** safety (inflation) radius added to the robot footprint
$U, U_{\text{att}}, U_{\text{rep}}$**:** total, attractive, and repulsive potential field
k_a, k_r, k_ω**:** potential field attractive, repulsive, and heading-correction gains
$\mathbf{F}$**:** artificial force derived from the potential field
θ_d**:** desired heading computed by the potential field planner
s**:** potential field speed-scaling factor near obstacles
$\mathcal{DW}, \mathcal{V}_s, \mathcal{V}_d$**:** dynamic window and its admissible-velocity and dynamically-reachable velocity sets
τ**:** a candidate trajectory forward-simulated by the dynamic window approach
$J(v, \omega)$**:** dynamic window approach cost of a candidate velocity command
$w_g, w_s, w_c, w_h, w_p, w_o$**:** goal, speed, clearance, heading, progress, and obstacle weighting parameters used by the local planners
N**:** model predictive control prediction horizon (number of steps)
R**:** control-effort weighting matrix in the model predictive control cost
$\phi(d)$**:** smooth obstacle penalty function used in the model predictive control cost
$L^\star$**:** reference (shortest feasible) path length for a BARN environment
$v_{\max}, \omega_{\max}$**:** maximum linear velocity and maximum angular velocity of the robot

Chapter 4: Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments.

$\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$**:** generalized coordinates, velocities, and accelerations of the 22-DOF humanoid
$\mathbf{M}(\mathbf{q})$**:** inertia matrix
$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$**:** Coriolis and centrifugal matrix
$\mathbf{G}(\mathbf{q})$**:** gravity vector
$\mathbf{S}$**:** actuation selection matrix
$\boldsymbol{\tau}$**:** joint torques
$\mathbf{J}(\mathbf{q})$**:** contact Jacobian
$\mathbf{F}$**:** vector of external contact and loop-joint constraint forces
$\mathbf{X}$**:** floating-base pose in task space
$\mathbf{K}_*, \mathbf{C}_*$**:** task-space proportional and derivative gains
L**:** step length
ϕ**:** bicycle-model steering angle (BiKE-Legs)
ω**:** differential-drive angular velocity (DRiVE-Legs)
ψ**:** leg orientation angle relative to the body heading
T**:** fixed step time
v**:** forward step velocity, $v = L/T$
$C, C_{\text{goal}}, C_{\text{speed}}, C_{\text{body}}, C_{\text{foot}}$**:** total, goal-progress, speed, body-clearance, and foot-clearance costs of the dynamic window approach

α, β, γ: weighting factors balancing the speed and collision-avoidance cost terms
$r_b^{(i)}$: clearance buffer added to the radius of obstacle i when stepping over is not permitted

Chapter 5: Interpretable Adaptive Dynamic Window Approach for Legged Robot Local Navigation.

$\mathbf{x}_t = [x_t, y_t, \theta_t]^T$: planar robot pose: position and heading angle
$\mathbf{u}_t = [v_t, \omega_t]^T$: commanded linear and angular velocity; also the robot's current linear and angular velocity when used as a state feature
Δt: simulation time step
$v_{\min}, v_{\max}, \omega_{\max}$: linear and angular velocity limits of the robot
V_d, V_s, V_a: dynamic window and its static-limit and acceleration-reachable velocity sets, $V_d = V_s \cap V_a$
$J(\mathbf{u}_t; \boldsymbol{\theta})$: dynamic window approach weighted objective function
$G(\mathbf{u}_t), C(\mathbf{u}_t), V(\mathbf{u}_t)$: goal-alignment, obstacle-clearance, and forward-speed terms of the objective function
$\boldsymbol{\theta} = [w_g, w_c, w_v]$: active dynamic window approach weight vector
$\Theta = \{\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_K\}$: set of K predefined dynamic window approach weight profiles
K: number of predefined weight profiles ($K = 5$)
a_t: selected weight-profile index (action) at time t
$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$: Markov decision process: state space, action space, transition dynamics, reward function, and discount factor
$\pi_\phi(a_t \mid s_t)$: stochastic weight-profile selection policy with parameters ϕ
s_t: local feature vector (state) at time t
$f_{\text{front}}, f_{\text{front-left}}, f_{\text{front-right}}, f_{\text{left}}, f_{\text{right}}, f_{\min}$: sector-based obstacle clearances and their minimum
$d_{\text{LR}}, d_{\text{FLR}}$: left–right and front-left–front-right clearance imbalance
$e_\theta, |e_\theta|$: heading error to the local goal and its absolute value
d_g: distance to the local goal
$r_{\max}$: maximum LiDAR-like sensing range
D_t, D_{t-1}, D_0: current, previous, and initial distance to the goal
$r_{\text{progress}}, r_{\text{away}}, r_{\text{time}}$: dense progress reward, away-from-goal penalty, and per-step time penalty
c_t: clearance between the robot body and the nearest obstacle boundary at time t
$r_{\text{clearance}}, r_{\text{near}}$: smooth obstacle-proximity penalty and stronger near-collision penalty
η: remaining episode-budget fraction, $\eta = (N - t)/N$
$r_{\text{goal}}, r_{\text{collision}}, r_{\text{infeasible}}, r_{\text{timeout}}$: terminal reward for reaching the goal and terminal penalties for collision, infeasible planning, and timeout
r_t, r_{terminal}: total reward at step t, and whichever terminal penalty applies at episode end
N: maximum episode-step budget in the reward function; also the total number of collected samples in the distillation dataset
$T, T^\star$: the decision tree mapping features to a weight profile, and the tree that minimizes imitation error
$\mathcal{D}_{\text{distill}} = \{(s_i, a_i^{\text{RL}})\}_{i=1}^N$: distillation dataset of feature vectors and RL-selected profiles
$\hat{a}_i$: decision tree's predicted weight profile for sample i

$\mathrm{Acc}_{\mathrm{imit}}$**:** imitation accuracy of the distilled tree relative to the RL policy
$L^{\star}$**:** reference (shortest feasible) path length for a BARN environment
OT, AT**:** optimal and actual traversal time, $OT = L^{\star}/v_{\max}$
$s_i, \mathbb{I}_i^{\mathrm{success}}$**:** BARN score for environment i and its success indicator

Summary

This dissertation addresses motion planning and control for legged robots navigating cluttered environments, progressing from reduced-order, data-driven step planning on a biped, to local-planner benchmarking on a quadruped, to stepping-enhanced local planning on a humanoid, and to interpretable adaptive tuning of a local planner's objective weights, evaluated on a quadruped. Across the platforms studied, a common hierarchical structure is used: a computationally efficient controller regulates balance and locomotion in real time, while a reduced-order model supports planning decisions over stepping, obstacle negotiation, and long-horizon navigation.

Chapter 2 models the step-to-step dynamics of a 5-link planar biped using partial feedback linearization to reduce the system from 10 to 2 state dimensions, following the established step-to-step control framework. The novel contribution is a Gaussian process approach to modeling the resulting Poincaré map and its region of validity: a Gaussian process classifier identifies which state-action pairs produce a feasible step with 99.5% accuracy, improving on a prior support-vector-machine classifier's 93.6% accuracy, and a Gaussian process regression model approximates the Poincaré map with 98.96% accuracy, improving on a prior quadratic polynomial approximation's 95.61% accuracy. An optimization problem built on these models tracks a reference velocity with 0.21% average error over 35 steps using 12 to 34 function evaluations per step, and crosses five stepping-stone ditches in eight steps using 30 to 155 function evaluations per step.

Chapter 3 evaluates three established local planners, the Dynamic Window Approach (DWA), Potential Fields (PF), and Model Predictive Control (MPC), each originally developed for wheeled or mobile robots. Using a task-space controller adapted from prior work to reduce the Unitree A1 quadruped to a unicycle model for planning, the contribution is a systematic, head-to-head benchmark of all three planners on a legged platform, evaluated on the standardized BARN obstacle courses in both simulation and hardware. Across 300 simulated BARN environments, DWA and PF reach the goal in 70% of trials versus 51% for MPC, while MPC produces shorter, faster trajectories when successful; hardware trials on the physical A1 confirm that these tradeoffs between reactivity and robustness transfer from simulation to real hardware.

Chapter 4 extends the established Dynamic Window Approach (DWA) and standard bicycle and differential-drive kinematic models to stepping-over behavior on the Agility Robotics Digit humanoid, a capability not previously realized in DWA-style local planners. The contribution is threefold: a constrained inverse-dynamics controller with task-space feedback that tracks center-of-mass velocity through a closed-form solution, avoiding the iterative optimization that full-body trajectory-optimization controllers typically require; two new kinematic planning models, BiKE-Legs and DRiVE-Legs, that augment the bicycle and differential-drive models with leg-based stepping; and DWA cost terms for goal progress,

obstacle negotiation, and speed that let the planner choose between stepping over small obstacles and going around large ones. The approach is demonstrated across Python simulation scenarios and validated, in a limited capacity, on a simulated Digit in MuJoCo.

Chapter 5 addresses a limitation of the established DWA planner: its objective weights are fixed scalars, so a single hand-tuned setting cannot fit both open regions and narrow passages. The contribution is an interpretable adaptive planner, DST-DWA: a Proximal Policy Optimization (PPO) reinforcement-learning policy selects among five predefined DWA weight profiles using compact, human-readable local features, and the trained policy is distilled into a shallow decision tree that replaces the neural network at evaluation time. On held-out 2D BARN environments, adaptive weight selection raises the success rate from 40.0% with fixed weights to 68.0%, and the tree reproduces the neural policy with 90.5% imitation fidelity. Transferred unchanged to a Unitree A1 quadruped in MuJoCo over 300 BARN environments, the advantage keeps its direction but is statistically borderline (+4.7 percentage points, McNemar $p = 0.059$); the tree's explicit mode log traces the gain to rescued failures and the dominant failure mode to over-caution in the obstacle-safe profile.

Together, these four chapters show that reduced-order, data-driven models can support real-time step-level and local-planning decisions across increasingly complex legged platforms, from a planar biped through a quadruped to a full 22-degree-of-freedom humanoid, while preserving the dynamic fidelity needed for hardware deployment and, in the final study, keeping the planner's runtime decisions readable as explicit rules.

CHAPTER 1

Introduction

1.1. Motivation

Legged robots can move through spaces that are difficult for wheeled robots. They can place their feet on separated footholds, recover balance by changing step location, and cross small obstacles without needing a continuous traversable surface. These properties make them useful for environments such as buildings, warehouses, construction sites, and outdoor paths where obstacles, gaps, and uneven terrain are common.

The same properties also make navigation more difficult. A wheeled robot can often be modeled as a disk or rectangle that moves in the plane. A legged robot has a body, multiple feet, a gait cycle, swing-leg motion, contact timing, and balance constraints. A path that is collision-free for the body may still fail because the robot cannot place a foot, cannot swing a leg through the available space, or cannot execute the required velocity without losing stability. At the same time, a path that appears blocked to a planar obstacle-avoidance planner may be feasible if the robot can step over the obstacle or place a foot beyond it.

A related tension appears in how local planners are tuned rather than in what actions they can take. Planners such as the Dynamic Window Approach depend on objective weights that trade off goal progress, clearance, and speed. A single fixed weight setting rarely suits every region of a cluttered environment, so some form of adaptation is useful. Learning can supply that adaptation, but a learned policy that changes planner behavior at runtime is only useful if its choices can be inspected. An opaque policy trades one limitation (fixed weights) for another (an unreadable decision process).

This dissertation studies these two tensions. The focus is local navigation for legged robots in cluttered environments, where the robot must make short-horizon decisions fast enough to run in closed loop. The goal is not to replace full-body control with a simpler

model. The goal is to use reduced-order models where they are useful, connect them to controllers that can execute the resulting commands, and make the planner's choices reflect both the actions legged robots can take and the local conditions that should govern how those actions are weighted.

1.2. Legged Robot Navigation as a Coupled Planning and Control Problem

A mobile robot can be viewed broadly as an agent that selects actions from observations to pursue task objectives [**83**]. For navigation, this action-selection process is often separated into layers. A planner selects a path or velocity command, and a controller tracks that command. This separation is useful because full-body dynamics are too expensive to solve over long horizons at every control step. It is also incomplete. For legged robots, the planner's command has meaning only if the controller can realize it while maintaining balance and contact constraints.

This coupling appears at several time scales. At the fastest time scale, the controller regulates joint torques, body motion, and contact forces. At the gait scale, the robot decides where and when to place the feet. At the navigation scale, the robot decides whether to move around an obstacle, pass through a narrow region, or use a stepping action. These decisions interact. A local planner may prefer a short path through clutter, but that path is useful only if the robot can maintain a feasible gait. A controller may track commanded body velocities well in open space, but the same velocity command may be poor near an obstacle if it forces the robot into a region with no feasible footholds.

Hierarchical planning and control are one way to manage this coupling. A reduced-order model is used for planning, while a lower-level controller handles the full robot dynamics. This dissertation uses that structure throughout the research chapters. Chapter 2 studies a biped through a step-to-step model. Chapter 3 evaluates local planners for a quadruped after task-space control reduces the planning problem to a unicycle model. Chapter 4 adds stepping actions to reduced kinematic models and connects those models to a constrained

inverse-dynamics controller for a humanoid. Chapter 5 studies adaptive DWA weight selection, where the planner structure is kept fixed but the objective weights change with local geometry.

1.3. Limitations of Conventional Local Planning for Legged Robots

A mobile robot's navigation stack usually separates localization and mapping from motion planning [**28**], and separates planning itself into two layers. A global planner searches the known or partially known map for a path, and a local planner reacts to what the robot currently senses [**82**]. Global planners range from classical graph search over a discretized map to sampling-based methods built on a configuration-space formulation of the planning problem [**62**], such as rapidly exploring random trees [**58**] and their asymptotically optimal variants [**49**]. Several recent surveys catalog this global-planning landscape, grouping methods as classical, heuristic, bio-inspired, or learning-based [**23, 89, 108, 109, 61, 26**]. Many of these reviews cover local planning alongside global planning but treat it briefly [**19, 111**], and surveys focused specifically on local, reactive collision avoidance are comparatively rare [**45, 29**]. This dissertation is concerned with that local layer, the short-horizon planner that must react to nearby obstacles fast enough to run in the control loop.

Local planners choose short-horizon commands using the robot state, the goal, and nearby obstacles. The Dynamic Window Approach (DWA) samples dynamically admissible velocity commands and scores the resulting trajectories using terms such as goal alignment, clearance, and speed [**5**]. Potential Fields (PF) treat the goal as an attractive field and obstacles as repulsive fields [**52**]. Model Predictive Control (MPC) solves a finite-horizon optimization problem with dynamics and constraints in the loop [**34**], including real-time formulations for legged robots that plan directly over reduced body and leg dynamics [**30**]. These methods are useful because they run online and do not require solving the full global planning problem at each instant.

Other reactive local planners follow similar velocity-space or trajectory-scoring principles. The curvature-velocity method scores circular arcs directly in velocity space [**88**], and velocity

obstacles remove commands that would lead to a collision given the relative motion of other agents [**32**]. The beam curvature method uses visibility beams for longer-horizon prediction in partially known environments [**31**]. Gap-based methods route the robot through free-space gaps between obstacles [**44**], and Timed Elastic Bands optimize a discretized trajectory for smoothness and clearance [**51**].

For legged robots, however, these planners are usually built around obstacle avoidance. The robot is represented as a footprint or a simplified body model, and nearby obstacles are treated as regions to avoid. That representation fits many navigation problems, but it removes part of the legged robot's action set. A robot with legs may be able to step over a short object, step onto a surface, or choose a foothold that a wheeled-robot model would not allow. If the local planner only scores paths around obstacles, it cannot compare avoidance with traversal.

This matters in cluttered environments. A planner that always routes around small obstacles may increase path length and time to reach the goal. A planner that ignores stepping feasibility may choose motions that a legged robot cannot execute. The planning problem is therefore larger than collision avoidance. The robot must decide which nearby obstacles should be avoided and which can be handled through legged contact.

Chapter 3 establishes this baseline by applying DWA, PF, and MPC to a quadruped in BARN-derived environments. The comparison is intentionally limited to conventional local planners. It asks what happens when planners that were largely developed for planar navigation are placed above a quadruped controller. The results provide the reference point for two later extensions. Chapter 4 responds to the limited action set described above by adding stepping actions to the local planner. A different limitation of the same baseline motivates Chapter 5. DWA's objective weights in this comparison are fixed constants, and a single fixed setting cannot fit every region of a cluttered environment. A local planner can be fast and useful, but its performance depends on the reduced model, how the planner represents obstacles, and whether its objective weights are tuned to the obstacles immediately around the robot.

1.4. Reduced-Order Models for Real-Time Legged Navigation

A full-order legged robot model contains many states, constraints, and contact modes. Planning directly with that model is difficult, especially when the planner must run repeatedly during navigation. Reduced-order models are used to make the problem smaller. The tradeoff is that a reduced model must retain the features that matter for the decision being made.

For bipedal locomotion, one useful reduction is the step-to-step map. Instead of controlling the robot at every instant, the model maps the robot state from one point in the gait cycle to the next. This idea is closely connected to limit cycle walking and Poincaré map analysis [**65, 43**]. Reinforcement learning has also been combined with this Poincaré-map framing to learn biped walking policies directly [**70**]. The map is still hard to write analytically for a nonlinear robot model, so Chapter 2 approximates it from simulation data. Partial feedback linearization reduces the state dimension, Gaussian Process Regression models the step-to-step dynamics, and Gaussian Process Classification identifies which state-action pairs lead to feasible steps. The resulting model is then used in optimization for velocity tracking and stepping-stone traversal.

For quadruped and humanoid navigation, the reduced model has a different role. The planner does not need every joint state if the low-level controller can track commanded body velocities. Chapter 3 uses this idea by planning with a unicycle model above a quadruped controller. Chapter 4 extends the idea by adding legs back into the kinematic model only where they matter for local decisions. BiKE-Legs and DRiVE-Legs keep the computational simplicity of bicycle and differential-drive models, but add stepping-over actions so the planner can consider both routes around an obstacle and routes that cross it.

Other approaches to legged motion planning build the tree or optimization directly around the full or reduced dynamics rather than a step-to-step map. Sampling-based methods such as rapidly exploring random trees have been extended with local feedback controllers to grow a tree of stabilizable trajectories, including LQR-Trees [**92, 93**] and RRT-based feedback motion planning for legged systems [**12**]. Full dynamically-stable motion planning has

also been formulated directly for humanoid robots [**55**]. These methods retain more of the robot's dynamics than the step-to-step map used here, at the cost of a larger search or optimization problem.

The common thread is selective model detail. The planner should not carry the full robot model when a lower-dimensional model is enough. It also should not remove the contact actions that determine whether a legged robot can use the environment. The reduced model has to match the decision level.

Chapter 5 does not extend this reduced-model thread further. It reuses the same unicycle-style planning model as Chapter 3. Rather than asking what the model represents, it asks how the planner built on top of it should be tuned to the obstacles around the robot.

1.5. Planner Parameterization and Interpretability

The planners in Chapter 3 and Chapter 4 share a limitation that is independent of the action set: their objective weights are fixed by hand. This is most visible in Chapter 3, where a single weight vector governs DWA across all environments, but Chapter 4 notes the same issue for its cost thresholds. In DWA, the selected command depends on the relative weights assigned to goal progress, obstacle clearance, and speed [**5**]. A speed-heavy setting may work well in open space. A clearance-heavy setting may be needed in narrow passages. A single fixed weight vector cannot represent both situations equally well.

This issue is separate from the robot model. Even with the same reduced-order model, a planner can behave very differently when its weights change. In cluttered environments, fixed weights can make the planner too aggressive, too conservative, or poorly aligned with the local obstacle geometry. The BARN benchmark is useful here because it provides standardized obstacle courses and metrics for comparing navigation behavior across different levels of geometric difficulty [**7**].

Learning can be used to adapt planner parameters, but a neural selector creates an additional inspection problem. If a learned policy changes DWA weights at runtime, it should be possible to trace which observed conditions produced the selected profile [**69, 11**].

Otherwise, the planner may improve performance while becoming harder to debug. This is a poor tradeoff for a method whose original appeal is its simple objective and explicit dynamic window.

Chapter 5 addresses this issue by keeping the DWA objective structure intact and adapting only the weight profile. A reinforcement learning policy selects from a small set of predefined DWA profiles using local geometric features, such as sector clearances, heading error, distance to the goal, and current velocity. The learned policy is then distilled into a shallow decision tree [**84, 79, 17**]. The intent is to keep the performance benefit of adaptive tuning while making each runtime choice readable as feature-threshold rules. The distilled planner is evaluated first in a 2D simulator and then transferred unchanged to a quadruped in MuJoCo, to test whether the adaptive behavior survives legged dynamics.

1.6. Thesis Statement and Research Questions

This dissertation studies how reduced-order models, local planners, traversal-aware action sets, and interpretable adaptive parameter tuning can be combined for legged robot navigation in cluttered environments. The central claim is that navigation performance depends on the coupling between three things: the model used by the planner, the actions the planner is allowed to consider, and the way planner parameters respond to the robot's environment.

Two of the questions below extend the same quadruped baseline in different directions. Chapter 4 enlarges the planner's action set, and Chapter 5 adapts the planner's existing parameters. These directions are not mutually exclusive. Chapter 4's stepping-augmented planner is itself built on fixed DWA-style weights, so it remains a candidate for the same adaptive tuning studied in Chapter 5, a connection returned to in the conclusion.

The dissertation is organized around four research questions:

(1) How can reduced-order, data-driven step-to-step models support computationally tractable stepping decisions for underactuated bipedal systems?

(2) What performance and computation tradeoffs appear when conventional local planners are deployed above a quadruped controller in standardized cluttered environments?

(3) How can local planners be extended so a legged robot can choose between avoiding an obstacle and traversing it through a stepping action?

(4) Can adaptive DWA weight tuning retain the performance benefits of a learned policy while remaining interpretable through explicit feature-threshold rules?

These questions move from dynamics to navigation. The first question asks how a step can be modeled and optimized. The second asks what existing local planners provide when applied to a quadruped. The third asks how stepping actions can be added to the local planner. The fourth asks whether a planner's parameters can adapt to the environment without hiding the decision logic inside a black box.

1.7. Contributions

The main contributions of this dissertation are as follows:

(1) A reduced-order, data-driven step-to-step planning framework for a five-link biped. The framework combines partial feedback linearization, Gaussian Process Regression, and Gaussian Process Classification to support velocity tracking and stepping-stone traversal in simulation.

(2) A quadruped local-planning benchmark that evaluates DWA, Potential Fields, and MPC in BARN-derived environments using a hierarchical control structure. The study compares success rate, traversal time, collision behavior, and computation time in simulation and on hardware.

(3) A stepping-aware local planning framework that augments bicycle and differential-drive kinematic models with leg actions. The resulting BiKE-Legs and DRiVE-Legs models allow a DWA-style planner to compare obstacle avoidance with stepping over selected obstacles.

(4) An interpretable adaptive DWA framework, DST-DWA, that formulates weight-profile selection as a reinforcement learning problem over compact local features and distills the trained policy into a shallow decision tree. The framework is evaluated on BARN environments in a 2D simulator and transferred unchanged to a Unitree A1 quadruped in MuJoCo, with a paired statistical comparison and a mode-level failure analysis.

Together, these contributions examine legged navigation at the level where planning and control meet. They do not solve perception, global mapping, or full autonomy in unknown environments. Instead, they isolate how the local planner represents the robot, how it scores candidate actions, and how those choices affect navigation through clutter.

1.8. Dissertation Organization

Chapter 2 presents a reduced-order step-to-step control framework for a five-link biped. Partial feedback linearization reduces the dynamics used for planning, Gaussian Process Regression approximates the Poincaré map, and a Gaussian Process Classifier identifies feasible step transitions. The chapter evaluates the method on velocity tracking and stepping-stone tasks.

Chapter 3 evaluates local motion planners for a quadruped. A task-space controller allows the high-level planner to use a unicycle model, and DWA, PF, and MPC are compared in standardized BARN-derived environments. This chapter provides the baseline for later work by showing the tradeoffs among classical local planners when applied to legged robot navigation.

Chapter 4 extends local planning beyond obstacle avoidance. It introduces BiKE-Legs and DRiVE-Legs, two reduced kinematic models that add stepping-over actions to conventional planar models. These models are embedded in DWA and paired with a constrained inverse-dynamics controller for a simulated Digit humanoid.

Chapter 5 studies interpretable adaptive tuning of DWA weights. It uses a reinforcement learning policy to select among predefined weight profiles and then distills that policy into

a decision tree so that the selected profile can be traced to local environment features. The distilled planner is evaluated on BARN environments in a 2D simulator and then in a MuJoCo transfer study on a quadruped.

Chapter 6 summarizes the dissertation findings and discusses remaining limitations. The conclusion returns to the main question of how much structure should be kept in the planner, how much should be placed in the controller, and where adaptive methods can be used without making the navigation system difficult to inspect.

CHAPTER 2

Stepping Control for Bipedal Walking via Partial Feedback Linearization and Gaussian Process Regression

Overview: Bipedal robots with small and point feet find it difficult to balance instantaneously due to underactuation, but can be relatively easy to control over the timescale of a step. Thus, models of step-to-step dynamics provide an effective means for the development of step-to-step controllers. Such models are analytically intractable because they are obtained from integrating nonlinear equations of motion. Our hypothesis is that obtaining approximations of such models can be effective for real-time computations of control inputs. We use partial feedback linearization to reduce the system dimensions for the step-to-step map. We formulate and solve an optimal control problem with these approximations to compute control for tracking a reference velocity and walking on stepping stones. These solutions demonstrate the need for bipedal robots to achieve specific control objectives, such as walking speed and foot placement, in order to navigate through restricted environments such as the stepping stones. We provide simulations on a 5-link biped to show the efficacy of the approach.

2.1. Introduction

Bipedal robots are well-suited for applications in homes and warehouses. These applications would require a robust walking gait that can be used in conjunction with task planning,

This chapter is adapted from: D. Torres, E. Hernandez Hinojosa, and P. A. Bhounsule, "Control of a Bipedal Walking Using Partial Feedback Linearization and Gaussian Process Regression-Based of the Step-to-Step Map," *Proceedings of the ASME 2023 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Volume 8: 47th Mechanisms and Robotics Conference (MR), Boston, Massachusetts, USA, August 20–23, 2023, V008T08A063. https://doi.org/10.1115/DETC2023-116945. © 2023 ASME. Reprinted, with permission. Minor formatting and editorial changes were made for consistency with the dissertation. This material is based upon work supported by the National Science Foundation under Grant No. 2128568 to P. A. Bhounsule and by the NSF Graduate Research Fellowship Program under Grant No. 2236870 to D. Torres. Ernesto Hernandez Hinojosa was funded by the American Heart Association Predoctoral Fellowship. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

path planning, motion planning, computer vision, and manipulation for use in practical applications. However, the challenge of reliable walking in simple environments has still not been accomplished.

Bipedal systems generally have point or small feet which leads to underactuation (i.e., the number of degrees of freedom exceeds the number of actuators on the degrees of freedom). Because of underactuation, such systems are nearly impossible to control instantaneously (e.g., balancing over the upright position when perturbed). However, it is possible to control these systems over a finite time horizon, typically of the order of one step [**43**]. For example, one can correct a push given to a standing robot with point feet by taking a step in the direction of the push, thus achieving balance control over one step [**78**]. This type of control, known as step-to-step control, is the most accepted method of controlling underactuated bipedal systems.

The step-to-step level control is typically solved offline with high-fidelity physics models and adapted for real-time control through a look-up table [**104**]. However, this approach fails to generalize to novel situations. Another approach is to use an online method where simple models are used for computing the controller and then mapping to the high-fidelity model during run-time [**56**]. This approach is often conservative because the simple models do not capture the complex dynamics. We present an approach that can be generalized to novel scenarios, but captures the high-fidelity of the physics model while being computationally modest. First, we use Partial Feedback Linearization (PFL) to transform the complex model to low dimensions. Second, we approximate the step-to-step dynamics of the high-fidelity model using a Gaussian Process Regression (GPR). Finally, we solve an optimization problem with the GPR-based step-to-step model during run-time to compute the correct control signal.

2.2. Background and Related Work

Origins of Step-to-Step Control. The step-to-step control was first presented in the context of passive dynamic walking by McGeer [**66**]. McGeer showed that a design that

resembles a human frame when designed properly can walk down a shallow slope with no control. Then McGeer interpreted the resulting periodic motion as a limit cycle (a repeating trajectory) and used the eigenvalues of the linearization of the step-to-step map, also known as the Poincaré map (i.e., the function that maps the initial conditions from one step at a particular instant in the locomotion cycle to the same instant at the next step), to analyze the stability of the system [**65**]. Finally, McGeer showed that by using a linear controller based on the step-to-step map, it was possible to increase the robustness of the passive limit cycle [**67**].

Approximations to the Step-to-Step Map. McGeer's approach to linearize the Poincaré map achieves stability in the small neighborhood of the limit cycle (periodic gait). To achieve a large range of stability, there have been attempts to compute the nonlinear approximation to the step-to-step map.

This is a data-driven approach that relies on a simulator to approximate the step-to-step map. The idea is to sample state and control (or state-action) pairs at the Poincaré section and integrate the equations to the next Poincaré section. This provides data for the step-to-step map which can then be curve fitted to a nonlinear function. The step-to-step map of a springy quadruped has been fitted with a neural network and then a model-predictive controller has been used for control [**98**]. Our past work considered using polynomial, neural network, and GPR to fit the step-to-step map of a spring-loaded inverted pendulum and the use of trajectory optimization for control. The neural network and GPR outperformed the polynomial regression [**13**]. One can also use the step-to-step map approximation of simple models to control more complex robots [**105**]. The step-to-step map may also use data from hardware experiments to obtain a better approximation [**71**].

Approximations to the Region of Validity of the Step-to-Step Map. One issue with these approaches is that not all state-action input pairs to the step-to-step map lead to a successful step; some inputs that may lead to failure. Hence, it is important to model the boundary of the feasibility of the step-to-step map. Our past work used support vector

machines to model the region of validity of the step-to-step map [**41**]. Another approach found that neural networks can be used for modeling the step-to-step map [**35**].

Model Reduction Over Step-to-Step Map. Although bipeds are high-dimensional systems with underactuation, one can use feedback control to reduce the dimensionality of the system.

The hybrid zero dynamics (HZD) approach is a dimensionality reduction technique [**36, 101**]. Here one defines continuous-time outputs (also known as virtual constraints) that map the actuated degrees of freedom to the unactuated degrees of freedom. One then designs a controller to drive these outputs to zero. This method is attractive because it uses control to reduce the dimensionality of the system to the unactuated degrees of freedom and is scalable. As the system complexity increases, it is not very easy to find these virtual constraints that lead to acceptable performance [**40**]. Although one can create exponentially stable continuous-time controllers, the orbital stability is only asymptotically stable [**22**].

In this work, we capitalize on the use of partial feedback linearization (PFL) to reduce the dimensionality of the system to the uncontrolled degree of freedom at the Poincaré map [**41**]. The system considered here has N actuators and $N + 1$ degrees of freedom. We use PFL to do continuous control of N degrees of freedom using the N actuators. However, these N actuator commands have free parameters. These free parameters can then be used to control the step-to-step map achieving control of the 1 unactuated degree of freedom at the timescale of a step.

This chapter is organized as follows. Section 2.3 provides details of the physics-based model for the 5-link biped. Section 2.4 provides the methods for reducing the dimensionality of the system, approximating the Poincaré map, and formulating the non-linear constrained program. Section 2.5 presents the results of the models and optimizations. Section 2.6 is the Discussion and Section 2.7 is the Conclusions and Future Work.

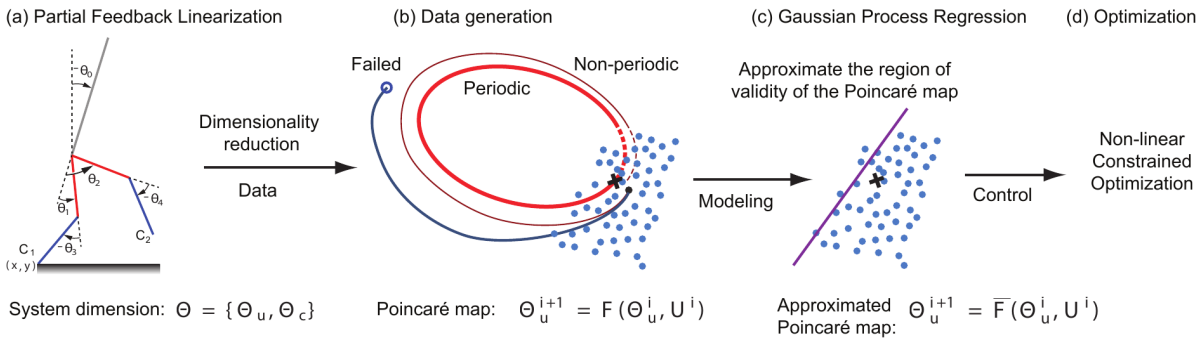


FIGURE 2.1. **Overview of the approach:** (a) PFL reduces the stance phase dynamics from $\Theta = [\Theta_u, \Theta_c]$ (10 dimensions) to $\Theta = \Theta_u$ (2 dimensions). (b) A Poincaré section is chosen at mid-stance. We generate random input state at the Poincaré section and controls at the step and simulate till the next Poincaré section to generate data for the Poincaré map given by $\mathbf{F}$, $\Theta_u^{i+1} = \mathbf{F}(\Theta_u^i, \mathbf{U}^i)$. (c) The Poincaré map is curve fitted $\Theta_u^{i+1} = \overline{\mathbf{F}}(\Theta_u^i, \mathbf{U}^i)$ where $\overline{F}$ is a Gaussian process regression model and Gaussian process classification is used to identify the boundary of the model. (d) Nonlinear programming is used to solve a suitably formulated quadratic program. For a video see [**16**].

2.3. Model

Figure 2.2 shows the 2D, 5-link model used in this study. This model and the ensuing description are from our past work [**41**]. We define the stance leg as the one in contact with the ground and the swing leg as the other. The foot in contact with the ground has coordinates (x, y) where the x-axis is horizontal and y-axis is vertical. The torso angle θ_0 is the angle between the torso and the vertical direction, θ_1 and θ_2 are the relative angles made by the thigh links of the stance and swing leg (respectively with the torso), and θ_3 and θ_4 are the angles made by the calf links of the stance and swing leg with their respective thigh links. The torso mass is $m_0 = 50$ kg, center of mass is at $c_0 = 0.5$ m, and inertia about the center of mass is $J_0 = 10$ kg-m^2. The thigh links have a mass of $m_1 = 7$ kg, center of mass is at $c_1 = 0.25$ m, and inertia about the center of mass is $J_1 = 5$ kg-m^2. The calf links have a mass of $m_2 = 5$ kg, center of mass at $c_2 = 0.25$ m, and inertia about the center of mass is $J_2 = 2$ kg-m^2. Gravity points downwards and is $g = 9.81$ m/s^2. The torso length is $\ell_0 = 1$ m, and the thigh link and calf link are equal, $\ell_1 = \ell_2 = 0.5$ m.

There are two sets of equations: one for the single stance phase where one foot is on the ground and the second for the foot-strike where the legs exchange roles.

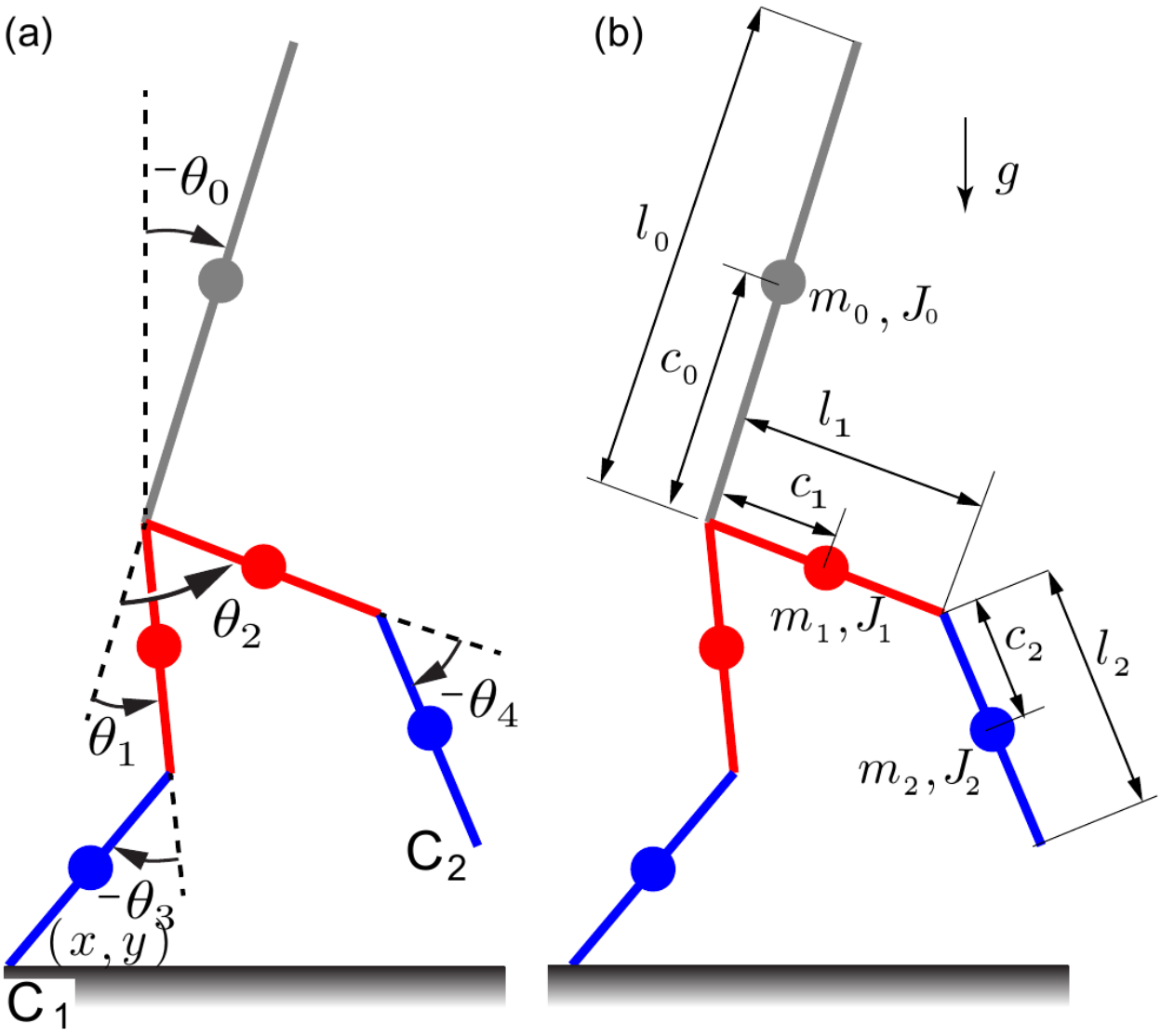


FIGURE 2.2. **Humanoid model:** (a) configuration variables describing the degrees of freedom, (b) mass, center of mass, inertia about center of mass, and length parameters.

2.3.1. Single Stance Equations. The state variables for derivation are defined as $\mathbf{q} = \begin{bmatrix} x & y & \theta_0 & \theta_1 & \theta_2 & \theta_3 & \theta_4 \end{bmatrix}^T$. We include the floating coordinates x and y to derive the equation, but the simplified equation has only five variables: θ_0, θ_1, ... θ_4. The Lagrangian $\mathcal{L} = \mathcal{T} - \mathcal{V} = 0.5 \sum \left(m_i v_i^T v_i + J_i \omega_i^T \omega_i \right) - \sum \left(m_i g y_i \right)$ where v_i, ω_i, y_i are the linear velocity, angular velocity, and y-position center of mass of link i, respectively. We take the summation

over all the five links. Using the Euler-Lagrange equations gives seven equations:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{N}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{B}\mathbf{u} + \mathbf{J}_{C_1}\mathbf{P}_{C_1} \tag{1}$$

where $\mathbf{M}$, $\mathbf{N}$, $\mathbf{B}$ are the mass matrix, accelerations due to Coriolis, centrifugal acceleration and gravity, and torque selection matrices, respectively. The control torques are $\mathbf{u} = \begin{bmatrix}\tau_1 & \tau_2 & \tau_3 & \tau_4\end{bmatrix}^T$ where τ_i is the torque for the joint associated with θ_i. The Jacobian is $\mathbf{J}_{C_1}$ of the contact point C_1 and $\mathbf{P}_{C_1}$ is the ground reaction force on the stance leg.

Without loss of generality, we can assume $x = y = 0$. Also, since C_1 is at rest, $\dot{x} = \dot{y} = \ddot{x} = \ddot{y} = 0$. Using these conditions, we use the first two equations in Eqn. 1 to find the ground reaction forces $\mathbf{P}_{C_1}$ as a function of joint angles, velocities, and acceleration. We may write the remaining five equations as:

$$\mathbf{M}_\theta(\theta)\ddot{\theta} + \mathbf{N}_\theta(\theta, \dot{\theta}) = \mathbf{B}_\theta\mathbf{u} \tag{2}$$

where $\mathbf{M}_\theta$, $\mathbf{N}_\theta$, $\mathbf{B}_\theta$ are versions of the matrices defined earlier. We use this equation for simulating single stance phase and for controller development later.

2.3.2. Foot-Strike Equations. When the swing foot C_2 touches the ground, the single stance phase ends and the robot transitions to an instantaneous foot-strike phase. We assume that the trailing leg applies an impulsive force along the stance leg, $\mathbf{I}_{C_1} = I\begin{bmatrix}-\sin(\theta_0+\theta_1+\theta_3), & \cos(\theta_0+\theta_1$ where I is the scalar impulse. This force comes from the ankle motor at C_1 which is passive during the stance phase, except during the foot-strike phase. Our choice of impulsive push-off is to be able to achieve energy-efficient walking compared to hip actuation (see [**57**]). In this phase, angular momentum is conserved about new contact point C_2. We obtain the equations for this phase by integrating Eqn. 1 and taking the limit as time goes to 0:

$$\begin{bmatrix}\mathbf{M}(\mathbf{q}^-) & -\mathbf{J}_{C_2}^T \\ \mathbf{J}_{C_2} & \mathbf{0}\end{bmatrix}\begin{bmatrix}\dot{\mathbf{q}}^+ \\ \mathbf{I}_{C_2}\end{bmatrix} = \begin{bmatrix}\mathbf{M}(\mathbf{q}^-)\dot{\mathbf{q}}^- + \mathbf{J}_{C_1}^T\mathbf{I}_{C_1} \\ \mathbf{0}\end{bmatrix} \tag{3}$$

where the superscript $-$ and $+$ denote the instance before and after collision, respectively.

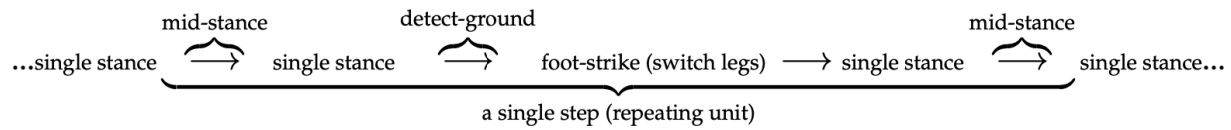


FIGURE 2.3. A representation of the walking cycle.

2.3.3. Simulating a Single Step. Figure 2.3 shows the general equation that describes a single step, the repeating unit, that starts and ends at mid-stance. We now explain the composition of a single step. We start the step at mid-stance when stance leg thigh link is vertical, $\theta_0 + \theta_1 = 0$. Next, we use the single stance Eqn. 2 to integrate the system till foot-strike. The foot strike occurs when the swing foot C_2 touches the ground, $y_{C_2} = \ell_1 \cos(\theta_0 + \theta_1) - \ell_1 \cos(\theta_0 + \theta_2) + \ell_2 \cos(\theta_0 + \theta_1 + \theta_3) - \ell_2 \cos(\theta_0 + \theta_2 + \theta_4) = 0$. Next, we apply the foot-strike condition given by Eqn. 3. Then we swap the legs, $\theta_0^+ = \theta_0^-$, $\theta_1^+ = \theta_2^-$, $\theta_2^+ = \theta_1^-$, $\theta_3^+ = \theta_4^-$, $\theta_4^+ = \theta_3^-$. Similarly, for the angular velocities we have $\dot{\theta}_0^+ = \dot{\theta}_0^-$, $\dot{\theta}_1^+ = \dot{\theta}_2^-$, $\dot{\theta}_2^+ = \dot{\theta}_1^-$, $\dot{\theta}_3^+ = \dot{\theta}_4^-$, $\dot{\theta}_4^+ = \dot{\theta}_3^-$. Finally, we integrate the equations in single stance given by Eqn. 2 till the next mid-stance given by $\theta_0 + \theta_1 = 0$.

2.4. Methods

2.4.1. Partial Feedback Linearization. PFL is utilized to regulate the actuated degrees of freedom during the stance phase. The mass matrix inversion of Eqn. 2 results in:

$$\ddot{\theta} = \mathbf{M}_\theta^{-1}(\theta)(\mathbf{B}_\theta \mathbf{u} - \mathbf{N}_\theta(\theta, \dot{\theta})) \tag{4}$$

The system is underactuated as it only has four actuators to control the five degrees of freedom. PFL is utilized to decouple the following degrees of freedom: the torso θ_0, the swing leg joints θ_2 and θ_4, and the stance leg knee θ_3. The decoupled degrees of freedom can be assigned as $\theta_c = \begin{bmatrix}\theta_0 & \theta_2 & \theta_3 & \theta_4\end{bmatrix}$. A matrix $\mathbf{S}_c$ can be found with the relation $\theta_c = \mathbf{S}_c\theta$, where $\mathbf{S}_c = diag\{1, 0, 1, 1, 1\}$ and $\theta = \begin{bmatrix}\theta_0 & \theta_1 & \theta_2 & \theta_3 & \theta_4\end{bmatrix}$. Taking this relation with Eqn. 4 results in:

$$\ddot{\theta}_c = \mathbf{S}_c\ddot{\theta} = \mathbf{S}_c\mathbf{M}_\theta^{-1}(\theta)(\mathbf{B}_\theta \mathbf{u} - \mathbf{N}_\theta(\theta, \dot{\theta})) = \mathbf{v} \tag{5}$$

where $\mathbf{v}$ is the control input:

$$\mathbf{v} = \ddot{\theta}_c^{ref} + \mathbf{K_d}(\dot{\theta}_c^{ref} - \dot{\theta}_c) + \mathbf{K_p}(\theta_c^{ref} - \theta_c) \tag{6}$$

Reference position θ_c^{ref}, velocity $\dot{\theta}_c^{ref}$, and acceleration $\ddot{\theta}_c^{ref}$ are assigned. A fifth-order polynomial is selected with specified initial and final positions, velocities, and accelerations, most of which are set to 0. $\mathbf{K_p}$ and $\mathbf{K_d}$ are diagonal matrices, with $\mathbf{K_p} = K_p\, diag\{1,1,1,1\}$ and $\mathbf{K_d} = 2\sqrt{K_p}\, diag\{1,1,1,1\}$, chosen to achieve critical damping. The motor torques are computed as follows:

$$\mathbf{u} = (\mathbf{S}_c\mathbf{M}_\theta^{-1}(\theta)\mathbf{B}_\theta)^{-1}(\mathbf{v} + \mathbf{S}_c\mathbf{M}_\theta^{-1}(\theta)(\mathbf{N}_\theta(\theta,\dot{\theta}))) \tag{7}$$

By defining the uncontrolled degree of freedom as $\theta_u = \mathbf{S}_u\theta$, where $\mathbf{S}_u = diag\{0,1,0,0,0\}$ and $\theta_u = \theta_1$, we can formulate an equation for this degree of freedom, incorporating the appropriate control input from the equation above:

$$\ddot{\theta}_u = \mathbf{S}_u\ddot{\theta} = \mathbf{S}_u\mathbf{M}_\theta^{-1}(\theta)(\mathbf{B}_\theta\mathbf{u} - \mathbf{N}_\theta(\theta,\dot{\theta})) \tag{8}$$

This equation is integrated in the single stance phase.

2.4.2. Step-to-Step Dynamics: Poincaré Map. The Poincaré section is a surface of $2N-1$ dimensions (with N being the total degrees of freedom of the system) that represents a particular instance in the locomotion cycle, such as mid-stance or foot-strike. A Poincaré section is depicted as blue dots in Fig. 2.1 (b). The Poincaré map is a function, denoted as $\mathbf{F}$, mapping the initial state at the Poincaré section Θ^i and the controls during the step $\mathbf{U}^i$ to the state at the next Poincaré section Θ^{i+1}. This map $\mathbf{F}$ describes the step-to-step dynamics and is determined by integrating equations from successive Poincaré sections shown in Fig. 2.3. Refer to **[90]** for more details on the Poincaré section and map. The Poincaré section is written as:

$$\Theta^{i+1} = \mathbf{F}(\Theta^i, \mathbf{U}^i) \tag{9}$$

where i represents the step number, the state is represented by $\Theta = \begin{bmatrix}\theta & \dot{\theta}\end{bmatrix}$, and θ is defined as $\begin{bmatrix}\theta_0 & \theta_1 & \theta_2 & \theta_3 & \theta_4\end{bmatrix}$. The discrete controls, foot placement angle and impulsive push-off, are denoted by $\mathbf{U}$ and are set once per step. The Poincaré map, $\mathbf{F}$, is chosen to map the successive mid-stance states. Generally, the Poincaré map is determined numerically as it is not always feasible to obtain an analytical formula. To approximate the Poincaré map, the equations of motion are integrated and/or the algebraic conditions for instantaneous phases (such as footstrike) are applied. Mid-stance is defined as $\theta_0 + \theta_1 = 0$. The Poincaré map is nine dimensional for this system with 10 degrees of freedom.

If the PFL is assumed to function as intended, the step-to-step dynamics rely solely on the uncontrolled degrees of freedom, which can be represented as $\Theta = \begin{bmatrix}\theta_u & \dot{\theta}_u\end{bmatrix}$. For the 5-link biped, θ_u is equivalent to θ_1. However, there is only one degree of freedom, $\dot{\theta}_1$, since the Poincaré map occurs at mid-stance. The controls are chosen as the step angle and push-off at foot-strike, denoted as $\theta_2 = \alpha$ and I, respectively. The system can be expressed as:

$$ {}^{m}\dot{\theta}_1^{i+1} = F({}^{m}\dot{\theta}_1^{i}, \alpha^i, I^i) \tag{10} $$

where ${}^{m}\dot{\theta}_1^{i}$ is the mid-stance speed of θ_1.

2.4.3. Poincaré Map Approximation. An analytical solution for the step-to-step dynamics is not feasible. A simple approximation $\overline{F}$ must be obtained where ${}^{m}_{A}\dot{\theta}_1^{i+1}$ is the estimated mid-stance velocity at step $i+1$:

$$ {}^{m}_{A}\dot{\theta}_1^{i+1} = \overline{F}({}^{m}\dot{\theta}_1^{i}, \alpha^i, I^i) \tag{11} $$

2.4.3.1. *Data Generation.* A simulator is designed as shown in Fig. 2.3 where the mid-stance speed ${}^{m}\dot{\theta}_1^{i}$, the foot placement angle α^i, and the push-off impulse I^i are the inputs and the mid-stance speed at the next step ${}^{m}_{A}\dot{\theta}_1^{i+1}$ is the output. While some input combinations can result in a feasible step, others may cause the model to lose balance and fall before taking the next step, rendering them infeasible. The data set consists of 1000 combinations

of $\dot{\theta}_1^i, \alpha^i, I^i$, and has been divided into training and testing sets with an 80-20 split. The entire training data set has been used to fit a Gaussian Process Classification (GPC) model to predict the feasibility of a step based on its state-action pair, and the data that results in a feasible step is utilized to fit a GPR model for the 2-BVP problem.

2.4.3.2. *Predicting Feasibility of State-Action Pairs.* A GPC model is used to predict whether a state-action pair will result in a feasible step. A GPR model is created with the MATLAB fitrgp function, an automatic relevance determination (ARD) squared exponential kernel, and expected improvement acquisition function. The hyperparameters, inputs, and outputs are:

$$L = \begin{bmatrix} \sigma({}^m\dot{\theta}_1^i) & \sigma(\alpha^i) & \sigma(I^i) \end{bmatrix} \tag{12}$$

$$\sigma_n = \sigma(y) \tag{13}$$

$$X = \begin{bmatrix} {}^m\dot{\theta}_1^i & \alpha^i & I^i \end{bmatrix} \tag{14}$$

$$y_{GPC} = feasibility \in [0, 1] \tag{15}$$

where all inputs X and outputs y_{GPC} pertain to the training data set. The hyper-parameters L and σ_n represent the length scales of the ARD squared exponential (one for each input dimension) and the signal standard deviation. The σ function is the standard deviation of each of the inputs. The intermediary GPR function is converted to a GPC by comparing the output with a threshold value, ϵ, to split the data into two classes, feasible and infeasible. The GPC model is given by:

$$y_{GPC} = \begin{cases} 0 \text{ (infeasible)} & \text{if } GPC(X) < \epsilon \\ 1 \text{ (feasible)} & \text{if } GPC(X) \geq \epsilon \end{cases} \tag{16}$$

where $\epsilon = 0.3$.

2.4.3.3. *Gaussian Process Regression Model of the Poincaré Map.* After developing the classification model, the feasible samples were used to fit a GPR model for the Poincaré map. The GPR hyperparameters and inputs were formulated with Eqns. 12–14. The GPR model is:

$$y_{GPR} = GPR(X) \tag{17}$$

where $y_{GPR} = {}^{m}_{A}\dot{\theta}_1^{i+1}$.

2.4.4. Quadratic Program. An optimization problem was formulated to find the inputs that approximate ${}^{m}_{A}\dot{\theta}_1^{i+1}$. The problem can be written as:

$$\underset{\mathbf{x}}{\text{minimize}} \quad f(\mathbf{x}) \tag{18}$$

$$\text{subject to: } g(\mathbf{x}) = 0 \tag{19}$$

$$h(\mathbf{x}) < 0 \tag{20}$$

$$\mathbf{LB} \leq \mathbf{x} \leq \mathbf{UB} \tag{21}$$

where $\mathbf{x} = \begin{bmatrix} \Theta^{i+1} & \mathbf{U}^i \end{bmatrix}^T$. The cost function is $f(x) = ({}^{m}_{A}\dot{\theta}_1^{i+1} - {}^{m}\bar{\dot{\theta}}_1^{i+1})^2 + (\alpha^i - \bar{\alpha}^i)^2 + (I^i - \bar{I}^i)^2$ where ${}^{m}\bar{\dot{\theta}}_1^{i+1}$, $\bar{\alpha}^i$ and $\bar{I}^i$ have the nominal values -0.97, 0.375, 0.18, respectively. The nominal values pertain to walking at human speed and step length [**14**]. The equality constraint, Eqn. 19, is given by $g(x) = GPR(x) - {}^{m}_{A}\dot{\theta}_1^{i+1}$. The inequality constraint, Eqn. 20, is given by $h(x) = GPC(x) + \epsilon$. **LB** and **UB** are the lower and upper bound limits set to $[-2.5, 0.087, 0.1]$ and $[-0.5, 0.873, 0.4]$, respectively. The bounds were chosen to contain the training data range and extend beyond. This allows for some flexibility in selecting the control parameters and applying them for different activities such as walking in a constrained environment.

2.5. Results

2.5.1. Gaussian Process Classification. Figure 2.4 shows the feature space of the test set, with stance angle α^i (radians) and impulse I^i (N·s) on the two in-plane axes and the approximated velocity ${}^{m}\dot{\theta}_1^i$ (rad/s) on the depth axis; the red and blue dots are the infeasible

and feasible samples, respectively. The two classes separate diagonally through this space: infeasible (red) samples cluster at larger stance angles, roughly α^i above 0.5 rad, paired with less negative approximated velocities, while feasible (blue) samples cluster at smaller stance angles paired with more negative velocities. Only a narrow band of points near this diagonal boundary mixes colors, consistent with the near-perfect classification accuracy reported below. The confusion matrix of the GPC model is shown in Fig. 2.5. Overall, the model has an accuracy of 99.5%. The model predicts infeasible data (0) and feasible data (1) with 100% and 98.2% accuracy, respectively.

2.5.2. Gaussian Process Regression. Figure 2.6 shows that the GPR model was incorrect by at most 7% or 93% accurate. Taking into consideration the nominal value of -0.97 rad/s, the GPR model predicted 67.7% of the data with a 99% accuracy and predicted 96.9% of the data with a 95% accuracy. The average accuracy of the model was 98.96%.

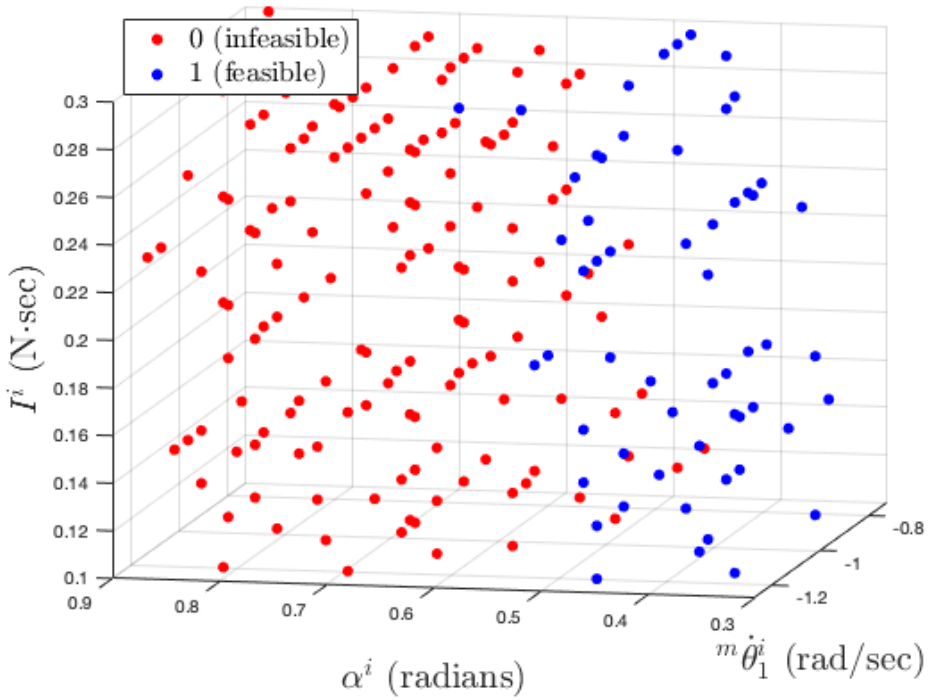


FIGURE 2.4. Feature space of the test set.

2.5.3. Optimization: Velocity Tracking. The model and optimization framework were tested by tracking a sinusoidal velocity profile ($\dot{\theta}_1^{ref}$) for 35 steps. The reference profile had an offset equivalent to the nominal velocity and an amplitude of 0.25 rad/sec. The cost function was modified to account for the reference profile by changing ${}^m\bar{\dot{\theta}}_1^{i+1}$ to $\dot{\theta}_1^{ref}$. The performance of the optimization solution with respect to the reference velocity is shown in Fig. 2.7, where the reference profile is shown in blue and the actual tracked velocity in red. The two curves closely overlap throughout all 35 steps, staying within a small band of each other at every step; the largest visible differences occur at the sharp transitions near the sinusoid's peaks and troughs. The percent error in tracking the reference velocity is shown in Fig. 2.8. The average percent error is 0.21% and shown in red. The number of function evaluations ranged from 12 to 34.

2.5.4. Optimization: Stepping Stones. The model and optimization framework were tested by planning several steps through a terrain with five ditches. At every step,

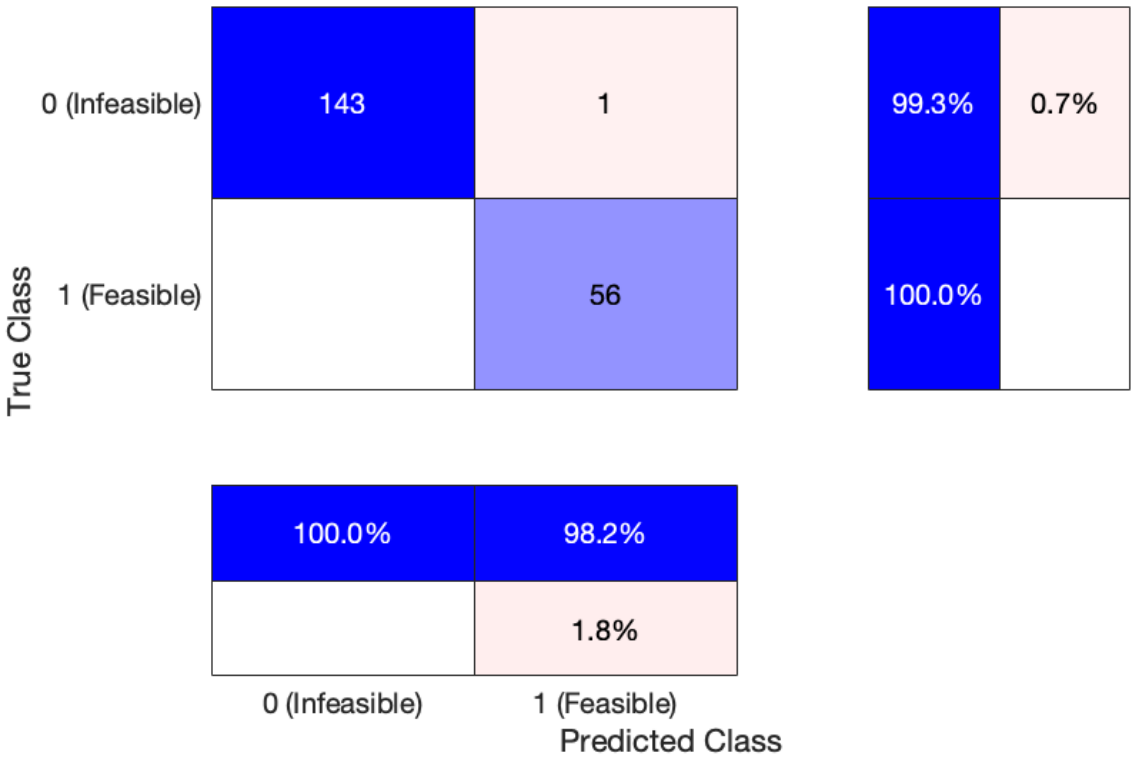


FIGURE 2.5. Confusion matrix for the test set.

the framework was designed to solve two optimization problems, one for stepping before and one for stepping after each ditch. A nonlinear constraint was imposed on the solver to account for this decision. After solving both optimization problems, the solution with the lowest cost is chosen. The optimization ${}^{m}_{A}\dot{\theta}_1^{i+1}$ (Eqn. 11) and simulation ${}^{m}\dot{\theta}_1^{i+1}$ (Eqn. 10) velocities are shown in Fig. 2.9, where ${}^{m}_{A}\dot{\theta}_1^{i+1}$ is shown in blue, ${}^{m}\dot{\theta}_1^{i+1}$ in red, and the nominal value of -0.97 rad/s in dashed black. Both curves track the nominal value closely for steps 1 through 5 and step 8, but rise together to roughly -0.93 rad/s (blue) and -0.94 rad/s (red) during steps 6 and 7, the two steps that clear the ditch obstacles, before returning close to nominal by the final step. The velocities are informed by the control actions in the previous step which in turn are influenced by the terrain. The percent deviation from the nominal control is shown in Fig. 2.10, where α^i is shown in blue and I^i in red. Both deviations stay under about 6% for the first two steps and return to nearly 0% through step 5, then spike sharply between steps 5 and 6 to roughly 56% (α^i) and 109% (I^i), coinciding with the same two ditch-clearing steps, before falling back under 10% for the final step. The planar biped

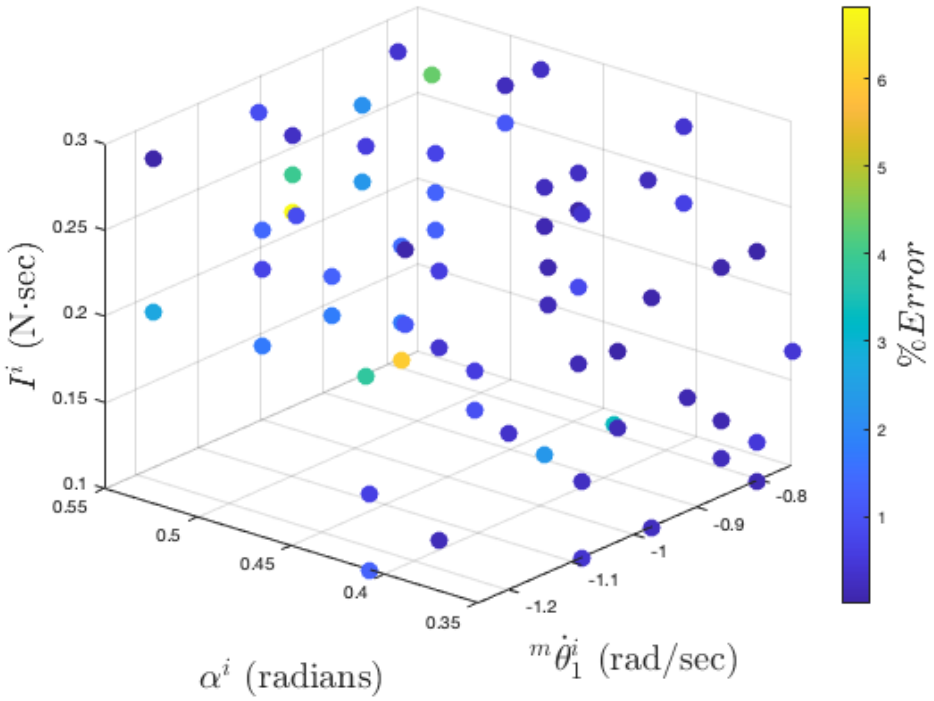


FIGURE 2.6. GPR performance for the test set.

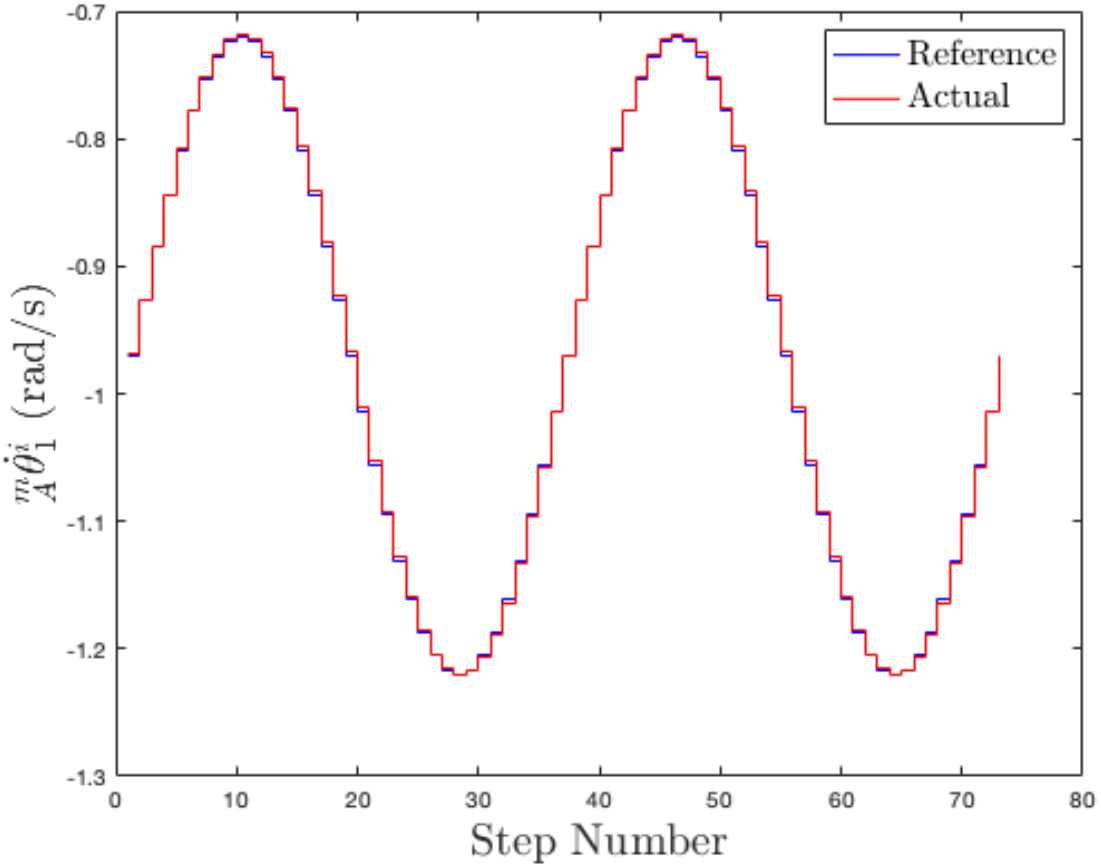


FIGURE 2.7. Sinusoidal velocity tracking performance.

took eight steps to traverse five ditches. The biped varied its step length to achieve clearance and proper foot placement in avoiding all the ditches, shown in [**16**]. The number of function evaluations ranged from 30 to 155.

2.6. Discussion

A 5-link planar biped was reduced via PFL from a 10D to 2D state space. The system was reduced to 1D by taking the Poincaré sections at mid-stance. Previous approaches for orbital stabilization of legged robots have been built around the limit cycle for control, but this control strategy was limited to small deviations from the fixed point. In contrast, data-driven approaches enable robust control capable of stabilizing a wider range of initial conditions. The system was simulated for various initial conditions to fit a GPC model

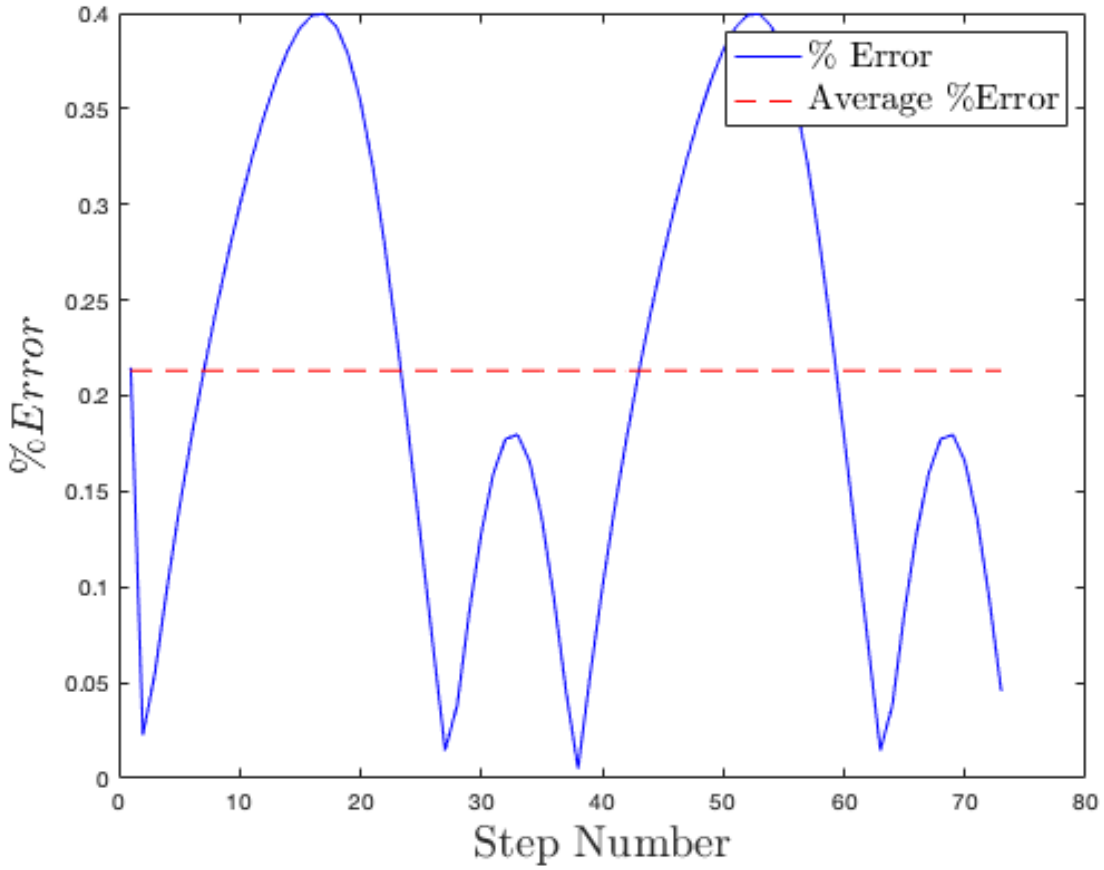


FIGURE 2.8. Velocity tracking percentage error.

capable of predicting whether a state-action pair would result in a feasible or infeasible step. The feasible data set was used to fit a GPR model capable of predicting the velocity at the next mid-stance step as a function of the current state-action pair, an approximation for the Poincaré map. The GPC and GPR models were set as constraints for a quadratic optimization.

We evaluated the optimization framework and model robustness through two trials, where the primary objectives were to track a reference velocity and step over obstacles. The velocity tracking was nearly flawless, with an average percent error of 0.21%. This performance demonstrated a simple scenario where the optimization strategy could be effective and efficient, with less than 34 function evaluations. To showcase the efficacy of our data-driven approach in solving a more complex task, we conducted the stepping stones optimization.

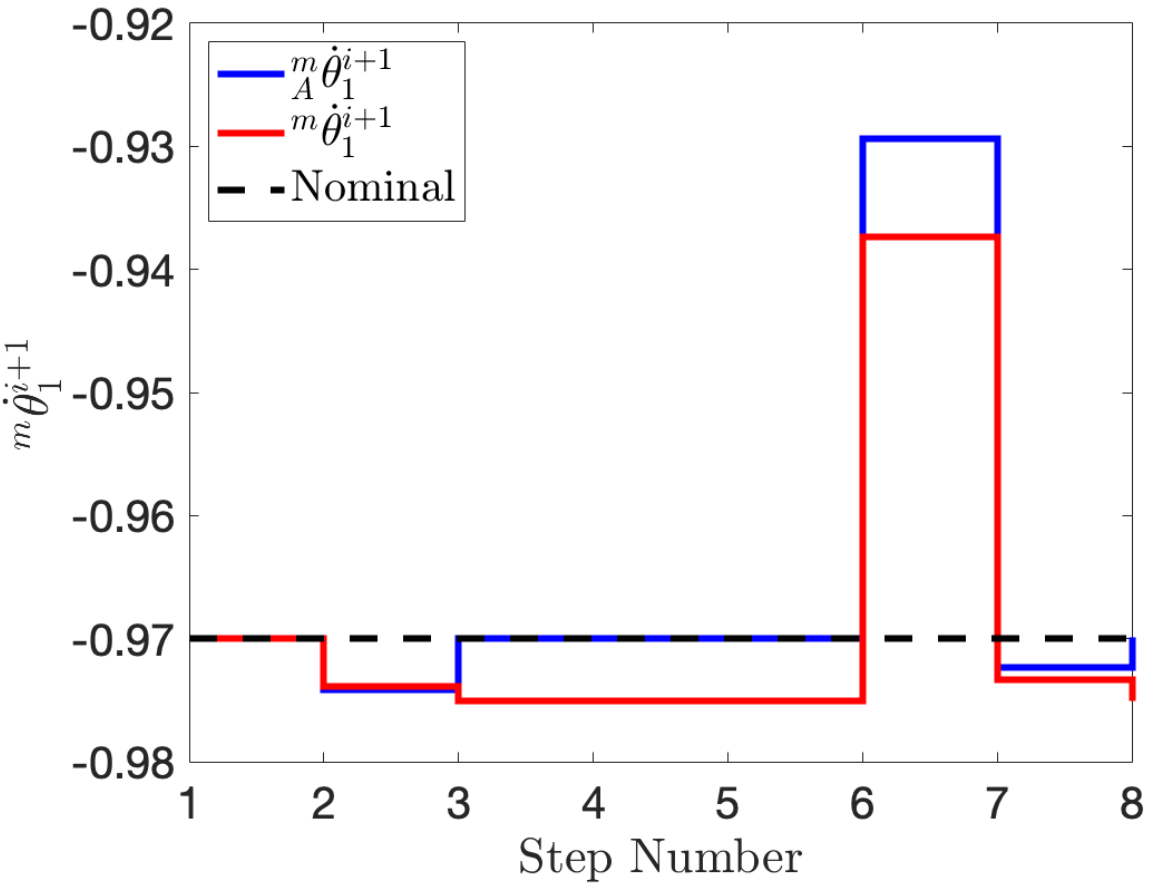


FIGURE 2.9. Optimization and simulation velocity at every step in the stepping stones simulation.

Despite its increased complexity, we successfully identified optimal control inputs using few function evaluations. Our results, presented in Fig. 2.10, illustrate how the system modulates the control states away from nominal to step over the obstacles. In step number five, the biped required drastic deviation from the nominal controls to clear the large ditch. The optimization velocity, shown in blue in Fig. 2.9, decreased in step number six due to the control action taken in the previous step. Overall, the optimization velocity deviated less from the nominal than the simulation velocity possibly due to the optimization constraints, Eqns. 19 and 20. This could be remedied by refining the sampling region for the training data to obtain more feasible data to train the GPR. The lack of deviation from the nominal could cause issues in more complicated scenarios where more flexibility is required. For example, if the simulation contained sensor or process noise the optimization values would be

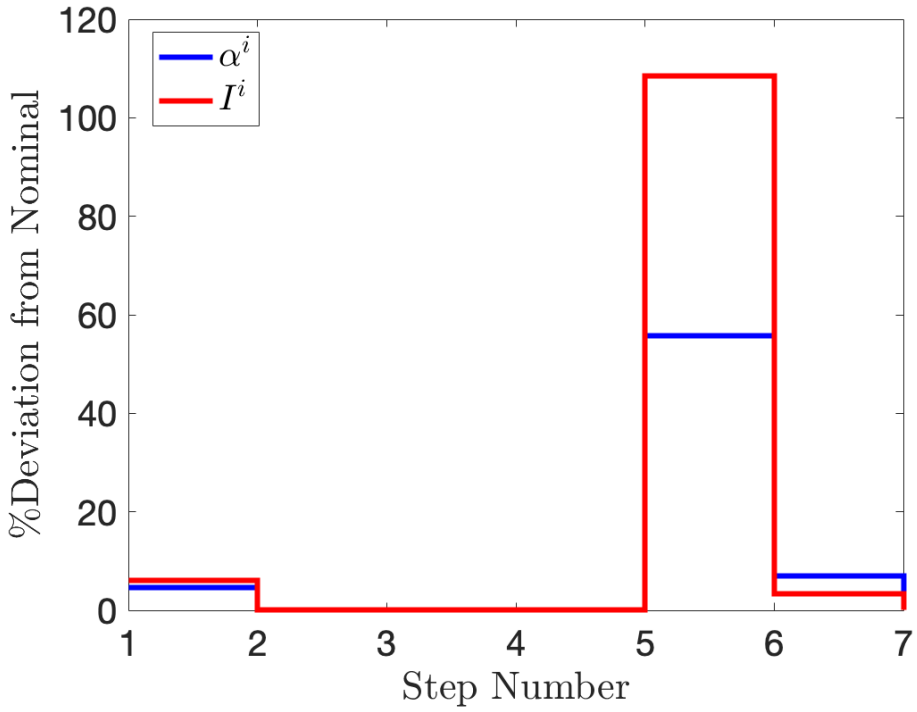


FIGURE 2.10. Nominal control percent deviation at every step in the stepping stones optimization simulation.

impacted and deviate significantly. This could be corrected by training another GPR model to encompass the stochasticity in the system. An alternative is to use the Poincaré map model from the simulation and augment the model with a GPR noise model. The learned control policy with an augmented GPR model improved trajectory prediction. Despite the shortcomings, the optimization strategy proved capable of achieving high accuracy within a low number of function evaluations, 12 to 34 for velocity tracking and 30 to 155 for stepping stones. A similar optimization problem would be formulated on hardware; the expectation would be that a low number of function evaluations would equate to computational efficiency that would allow for real-time control.

This method improves upon [**41**] which utilizes the same biped model, although it should be noted that there are differences in the simulations which do not make the comparisons one-to-one. The previous work approximated the region of validity with a support vector machine classifier and predicted feasibility (86.2%), infeasibility (95.8%), and mean accuracy (93.6%) compared to the GPC with feasibility (98.2%), infeasibility (100%), and mean accuracy

(99.5%). The GPC accurately captures the region of the validity for this system. This work utilized GPR to approximate the Poincaré map and had a mean accuracy of 98.96% compared to a quadratic polynomial regression approximation with a mean accuracy of 95.61%. The GPR is able to capture the Poincaré map's higher order terms, resulting in low error. Although a different velocity profile was tracked, this approach yielded low error, 0.21% compared to 10.6%, and required a similar number of function evaluations, 12 to 34 compared to 15 to 40. The stepping stones simulation is difficult to compare as this approach was undertaken in a different environment with one more ditch. Despite this difference, both optimization methods remained close to the nominal velocity, except when taking longer steps. The max number of function evaluations was higher, 155 compared to 130.

2.7. Conclusions and Future Work

We conclude that GPR and GPC are effective methods to model the Poincaré map and region of validity for bipedal walking. The implemented strategy can achieve computationally efficient and precise control in bipedal walking, thus proving a promising method to implement on hardware.

The optimization strategy can be further improved by augmenting the simulation with a GPR model to account for any model inaccuracies or stochasticity. This would further enhance the robustness and would facilitate the sim-to-real transfer. Eventually, this model will be evaluated on a humanoid with the goal of augmenting the model with hardware generated data, similar to the sim-to-real transfer method explored in [**54**].

CHAPTER 3

Comparative Study of Local Planners for Quadrupeds Using BARN Environments

Overview: Integrating legged robots into motion planning frameworks is challenging because navigation and balance goals must be met simultaneously. We present a hierarchical framework that utilizes task-based motion control to reduce legged dynamics to a unicycle model for motion planning. Then, using the standardized Benchmark for Autonomous Robot Navigation (BARN) environments, we benchmark three established local planners: Dynamic Window Approach (DWA), Potential Fields (PF), and Model Predictive Control (MPC) and validate these approaches via simulation and hardware experiments. Our results demonstrate that a hierarchical framework offers a robust, computationally efficient pathway for deploying quadrupedal robots in obstacle-dense environments.

3.1. Introduction

There has been substantial progress in the motion control of legged robots, enabling stable locomotion, but their broader usage requires them to be integrated into motion planning frameworks. Quadruped robots, unlike wheeled robots, pose a challenge because motion planning must be tightly coupled with balance control. In this chapter, we address this gap by benchmarking three local planning approaches: Dynamic Window Approach (DWA), Potential Fields (PF), and Model Predictive Control (MPC), on BARN environments in simulation and hardware.

This chapter is adapted from the accepted manuscript: D. Torres and P. A. Bhounsule, "Comparative Study of Local Planners for Quadrupeds Using BARN Environments," accepted to the 6th Modeling, Estimation and Control Conference (MECC 2026), Phoenix, AZ, October 25–28, 2026. The final published version may differ from this chapter. This material is based upon work supported by the National Science Foundation under Grant No. 2128568 to P. A. Bhounsule and by the NSF Graduate Research Fellowship Program under Grant No. 2236870 to D. Torres. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

There are broadly two types of planners, global and local. Global planners assume a complete map and compute a collision-free path in advance. Probabilistic Roadmaps (PRM), for example, build a tree over free space and search it with a method such as A-star [**50, 37**]. This gives globally optimal solutions but is computationally intensive and requires re-planning when the environment changes. Local planners instead generate motion commands online from current sensor feedback, trading global optimality for computational efficiency and reactivity, though they can become trapped in local minima.

Prior work on motion planning for legged robots falls into integrated and hierarchical approaches. Integrated approaches solve planning and control simultaneously, enforcing dynamics, contact, and collision constraints directly, at high computational cost. [**85**] combines a reinforcement-learning-based local planner with foothold adaptation, and [**33**] uses nonlinear MPC with collision avoidance and joint optimization of forces and center-of-mass motion. Hierarchical approaches instead decouple planning and control. A low-level controller tracks velocity commands while a higher-level planner uses simplified models, as in [**77**]'s Hybrid Zero dynamics controller with a 3D inverted pendulum planning model. This is computationally efficient but can request motions the controller cannot realize.

In this work, we adopt a hierarchical control framework. At the low level, we employ a computationally efficient task-level controller to ensure robust locomotion [**2**]. This controller enables a reduced-order representation of the robot's center-of-mass dynamics using a unicycle model, which is then used for motion planning. Using the unicycle model, we study three widely used local planning approaches. DWA searches over admissible velocity commands while accounting for robot dynamics, enabling real-time obstacle avoidance [**5**]. PF methods generate motion by combining attractive and repulsive forces, providing simple and reactive behavior but suffering from local minima [**52**]. MPC formulates planning as a constrained optimization problem over a finite horizon, explicitly handling system dynamics and constraints at the cost of higher computational complexity [**34**].

Our contributions are as follows: (1) a hierarchical, computationally efficient control and planning framework for legged robots based on a reduced-order model; (2) an extensive

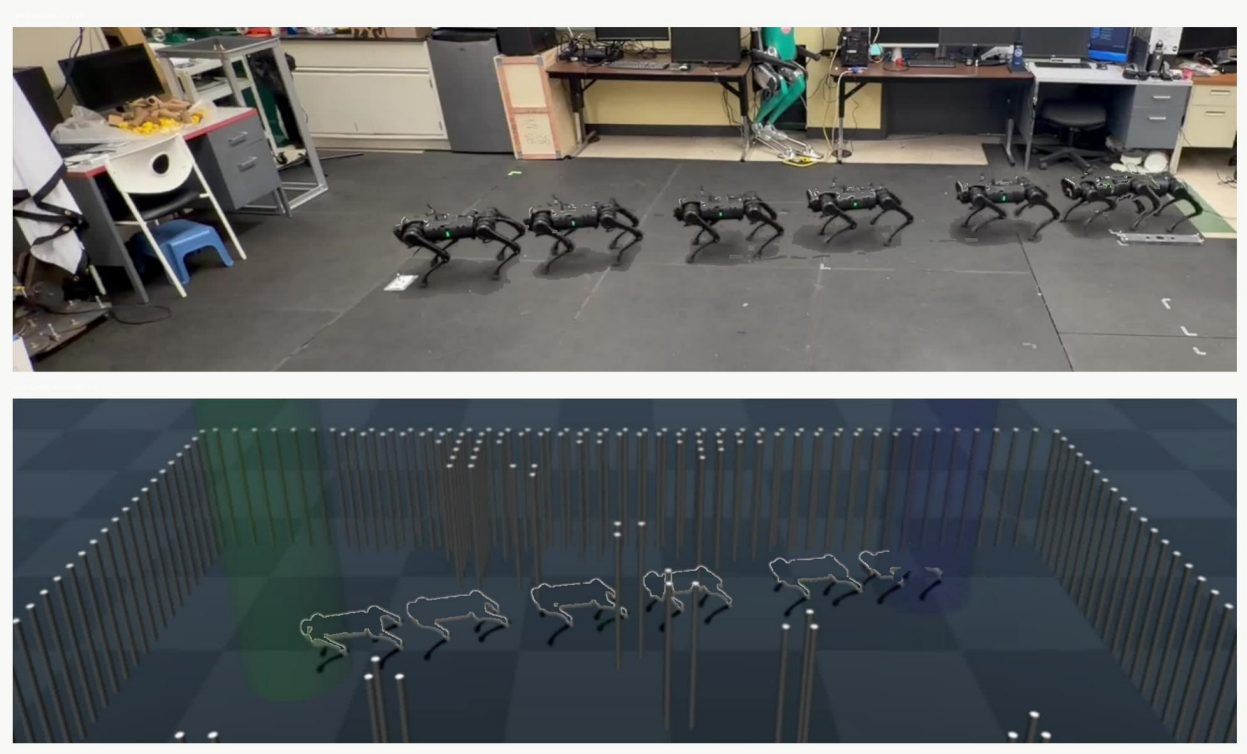


FIGURE 3.1. Still-motion path of the robot in BARN Env. 000. The hardware trajectory is shown on top (obstacles are virtually enforced using a virtual camera), while the corresponding MuJoCo reconstruction from recorded hardware data with obstacles (white) is shown below.

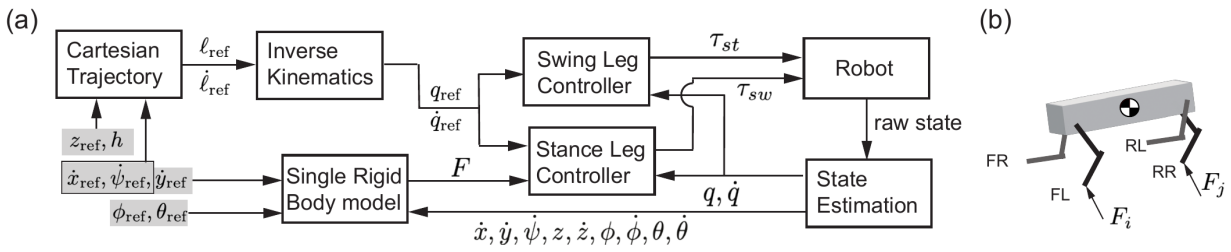


FIGURE 3.2. (a) Block diagram of the robot controller. A state machine is used to program the robot controller logic. The inputs to the state machine are shown in the box with gray highlight, but only two inputs, the linear speed in fore-aft direction $\dot{x}_{\text{ref}} = v_x = v$ and yaw speed $\dot{\phi}_{\text{ref}} = \omega$ are used for control in this paper. (b) The single rigid body model is used for regulating force/torque on the body from the stance leg. Details are provided in [**2**].

comparison of three local planners: DWA, PF, and MPC; an evaluation using the BARN environments; and (3) experimental validation on a hardware testbed to demonstrate real-world performance.

3.2. Methods

3.2.1. Hardware. The quadruped platform used in this chapter is the Unitree A1 [**97**], which has four legs and 12 degrees of freedom (DOF). Each leg has a mass of 2 kg and the torso has a mass of 4.75 kg. The robot is equipped with joint encoders, a torso-mounted inertial measurement unit (IMU), and foot contact sensors. Joint torques are bounded between -33.5 and 33.5 Nm and are estimated via calibrated motor current sensing. Control runs on two systems. An external computer sends torque, position, and velocity commands at 1 kHz, while a proprietary onboard motor controller operates at 10 kHz and is not user-accessible. Communication is handled using Lightweight Communications and Marshalling (LCM) and User Datagram Protocol (UDP). Localization of the robot and obstacles is provided by a 12-camera OptiTrack system streaming over Wi-Fi at approximately 300 Hz.

3.2.2. Motion Control.

3.2.2.1. *Overview of control:* The motion controller shown in Fig. 3.2 (a) coordinates the balance and locomotion of the quadruped and runs at 1 kHz. The legs are labeled as Front Right (FR), Front Left (FL), Rear Right (RR), and Rear Left (RL) as shown in Fig. 3.2 (b). For the trot gait, the combinations of FR-RL and FL-RR move together. The pair of legs cycles between stance and swing based on a fixed step time of 0.2 sec throughout this work. More details on the motion control are provided elsewhere [**2**].

3.2.2.2. *Inputs to the motion planner:* The motion planner (described in the next section) provides inputs to the motion controller at each step, operating at 5 Hz (step time is 0.2 sec). The planner specifies the fore-aft speed, $\dot{x}_{\text{ref}} = v$, and the turning rate, $\dot{\psi}_{\text{ref}} = \omega$. The following inputs are kept fixed in this work: lateral speed, $\dot{y}_{\text{ref}} = 0$; reference leg height, $z_{\text{ref}} = \bar{z}$; ground clearance; and torso roll and pitch, $\phi_{\text{ref}} = 0$ and $\theta_{\text{ref}} = 0$, respectively.

3.2.3. Motion Planning.

3.2.3.1. *Unicycle Model for planning:* We assume that the motion controller presented earlier allows us to model the overall motion of the center of mass of the robot using the unicycle model. This model has the configuration variables (x, y, θ) and control inputs (v, ω).

The continuous dynamics are $\dot{x} = v\cos\theta,\ \dot{y} = v\sin\theta,\ \dot{\theta} = \omega$. These equations are discretized using Euler integration with a time step of $h = 0.2\,\mathrm{s}$.

$$x_{k+1} = x_k + hv_k\cos\theta_k, \quad y_{k+1} = y_k + hv_k\sin\theta_k,$$
$$\theta_{k+1} = \theta_k + h\omega_k. \tag{22}$$

3.2.3.2. *Overview of planning:* We use three local planning strategies: DWA, PF, and MPC. For each trial, the robot position (x_i, y_i) and the goal position (x_g, y_g) are known at all times. We model each obstacle as a circle with center (x_i, y_i) and radius r_i. In order to account for the robot size, we inflate each radius by an additional radius of r_s. This enables us to treat the robot as a point mass, thus simplifying the real-time calculations.

In the planner description, all gains and weighting parameters (e.g., k_a, k_r, k_ω, w_g, w_s, w_c, w_h, w_p, w_o) are positive scalar tuning parameters. All planners require parameter tuning, and gains are selected empirically and are consistent across all experiments.

3.2.3.3. *Potential Field (PF):.* The potential field is defined as

$$U(\mathbf{p}) = U_{\mathrm{att}}(\mathbf{p}) + \sum_{i=1}^{N} U_{\mathrm{rep},i}(\mathbf{p}). \tag{23}$$

The attractive potential uses a conic-paraboloidal form with goal radius r_g:

$$U_{\mathrm{att}} = \begin{cases} \dfrac{1}{2}k_a\|\mathbf{p}-\mathbf{p}_g\|^2, & \|\mathbf{p}-\mathbf{p}_g\| \le r_g, \\ k_a\, r_g\, \|\mathbf{p}-\mathbf{p}_g\|, & \text{otherwise.} \end{cases} \tag{24}$$

The repulsive potential is defined for obstacle distance $d_i = \|\mathbf{p} - \mathbf{q}_i\|$:

$$U_{\mathrm{rep},i} = \begin{cases} \dfrac{1}{2}k_r\left(\dfrac{1}{d_i} - \dfrac{1}{d_0}\right)^2, & d_i \le d_0, \\ 0, & d_i > d_0. \end{cases} \tag{25}$$

The control inputs are derived from the artificial force $\mathbf{F} = -\nabla U$. The desired heading is

$$\theta_d = \text{atan2}(F_y,\, F_x). \tag{26}$$

The linear and angular velocities are computed as

$$v = s\,\|\mathbf{F}_{\text{att}}\|, \tag{27}$$

$$\omega = k_\omega\,\text{wrap}(\theta_d - \theta), \tag{28}$$

subject to saturation limits $v \in [v_{\min}, v_{\max}]$ and $\omega \in [-\omega_{\max}, \omega_{\max}]$. The scaling factor

$$s = \text{clip}\left(\frac{d_{\min} - r_s}{d_0},\, 0,\, 1\right) \tag{29}$$

reduces speed near obstacles, where $d_{\min}$ is the distance to the nearest obstacle and r_s is a safety radius.

3.2.3.4. *Dynamic Window Approach (DWA):.* The dynamic window is defined as

$$\mathcal{DW} = \mathcal{V}_s \cap \mathcal{V}_d, \tag{30}$$

where $\mathcal{V}_s = [v_{\min}, v_{\max}] \times [-\omega_{\max}, \omega_{\max}]$ and

$$\mathcal{V}_d = [v \pm \dot{v}_{\max}\Delta t] \times [\omega \pm \dot{\omega}_{\max}\Delta t]. \tag{31}$$

Each candidate $(v, \omega) \in \mathcal{DW}$ is forward simulated using Eq. (22). A trajectory τ is admissible if the robot can stop before collision:

$$\frac{v^2}{2\dot{v}_{\max}} \leq d_{\min}(\tau) - r_s, \tag{32}$$

where $d_{\min}(\tau)$ is the minimum distance to obstacles.

Admissible trajectories are ranked by

$$J(v,\omega) = w_g \,|\mathrm{wrap}(\theta_h - \theta_{\mathrm{goal}})| \\ + w_s \frac{v_{\max} - v}{v_{\max}} + w_c \, C_{\mathrm{obs}}(\tau), \tag{33}$$

where x_h, y_h, and θ_h denote the robot's position and heading at the end of the forward-simulated trajectory τ, and $\theta_{\mathrm{goal}} = \mathrm{atan2}(y_g - y_h,\, x_g - x_h)$. The optimal control is selected as

$$(v^*, \omega^*) = \underset{(v,\omega)\in\mathcal{DW}}{\arg\min}\, J(v,\omega). \tag{34}$$

3.2.3.5. *Model Predictive Control (MPC):.* The MPC planner optimizes a control sequence $\mathbf{u}_{0:N-1}$ over a horizon of length N, with predicted states generated using Eq. (22). Let $\mathbf{p}_k = [x_k, y_k]^\top$ and $\mathbf{g} = [x_g, y_g]^\top$. The cost function is

$$J = \sum_{k=1}^{N} \Big[\frac{w_g}{d_0^2} \|\mathbf{p}_k - \mathbf{g}\|^2 + w_h\big(1 - \cos\theta_{kg}\big) - w_p v_{k-1} \cos\theta_{kg} \Big] \\ + \sum_{k=0}^{N-1} \mathbf{u}_k^\top R\, \mathbf{u}_k + \sum_{k=1}^{N} w_o \, \phi(d_k), \tag{35}$$

where $\theta_{kg} = \theta_k - \theta_g$, $\theta_g = \mathrm{atan2}(y_g - y_k,\, x_g - x_k)$ and $d_k = \min_i \|\mathbf{p}_k - \mathbf{q}_i\|$.

The matrix $R = \mathrm{diag}(w_v/v_{\max}^2,\ w_\omega/\omega_{\max}^2)$ penalizes control effort. The function $\phi(d)$ is a smooth obstacle penalty that increases sharply within a safety radius.

The optimization is subject to box constraints

$$0 \le v_k \le v_{\max}, \quad -\omega_{\max} \le \omega_k \le \omega_{\max}. \tag{36}$$

A rollout is rejected if any predicted state violates a collision clearance threshold. The MPC optimization problem is solved using the SLSQP solver from the `scipy.optimize` module with a horizon of length 5.

3.2.4. BARN: Standardized Planning Environments. The Benchmark for Autonomous Robot Navigation (BARN) is a standardized dataset of 300 cluttered, narrow-passage environments designed to evaluate navigation performance under challenging local feasibility constraints. It includes precomputed difficulty metrics such as distance to closest obstacle, average visibility, dispersion, characteristic dimension, and tortuosity **[7]**.

The BARN dataset, based on the Clearpath Robotics Jackal unmanned ground vehicle (UGV), utilizes the same environments for the Unitree A1 robot due to their similar circumscribed diameters. To enhance feasibility in these constrained settings, obstacle radii are reduced by 25% (0.075 m) across simulation and hardware experiments.

For each environment, let $L_i^\star$ denote the reference path length. The optimal traversal time (OT_i) is defined as

$$OT_i = \frac{L_i^\star}{v_{\max}}, \tag{37}$$

where $v_{\max} = 0.5$ m/s is the robot's maximum speed.

Let AT_i denote the actual traversal time. The BARN score is defined as **[103]**

$$s_i = 1_i^{\text{success}} \times \frac{OT_i}{\text{clip}(AT_i,\, 2OT_i,\, 8OT_i)}, \tag{38}$$

where the indicator function 1^{success} evaluates to 1 if the robot reaches the navigation goal without any collision and evaluates to 0 otherwise. This bounds the score to [0.125, 0.5] for successful runs and assigns 0 to failures (i.e., collisions or timeouts). Timeouts are defined as trials exceeding 60 seconds of simulation or experimental time.

3.2.4.1. *Perception Model:* We assume a virtual range sensor with a 90° field of view and 1° angular resolution, returning distance measurements at each bearing angle up to a maximum range of 2.0 m. Measurements at or beyond this range are set to the maximum value and discarded. Each valid measurement is converted into a point $\mathbf{q}_i$ from its bearing and range, forming the set of perceived obstacle points.

The sensor model is identical in simulation and hardware. Both generate measurements by ray casting against known obstacle geometry rather than a physical sensing device, allowing controlled and repeatable evaluation.

3.2.5. Execution and Compute Setup. All experiments were conducted on a desktop with an Intel Core i7-9700K CPU (3.6 GHz, 8 cores) running Ubuntu 20.04. Low-level control was implemented in C/C++, while local planners were implemented in Python. Each environment–planner pair was evaluated independently, with runs parallelized across 7 worker processes. All planners were required to operate within a 0.2 s control loop, consistent with the hardware deployment. Compute-time statistics are reported per planner and are independent of parallel execution.

3.3. Results and Discussion

3.3.1. Simulation Experiments. We compare the performance of DWA, MPC, and PF across all 300 BARN environments (900 total runs) to assess robustness under consistent obstacle scaling and varying environment complexity.

DWA and PF achieve identical goal rates, each reaching the goal in 209 environments (70%), while MPC performs worst with 152 successes (51%), as shown in Table 3.1. Timeout rates are similar across planners, though slightly higher for MPC (26, 9%) compared to DWA (21, 7%) and PF (17, 6%). A similar trend is observed in collision rates, with DWA and PF at 70 (23%) and 74 (25%), respectively, and MPC higher at 122 (41%). These results indicate reduced robustness of MPC in cluttered environments under the current formulation and tuning. The BARN score reflects these trends, with DWA and PF achieving identical performance 0.35 ± 0.23, while MPC attains a lower average score (0.25 ± 0.25), consistent with its reduced success rate.

Among successful runs, MPC produces more efficient trajectories, with lower traversal time (12.64±6.67 s) and shorter path length (4.29±0.23 m) than DWA (20.38±11.79 s, 5.82± 2.24 m) and PF (21.10 ± 10.09 s, 5.53 ± 1.52). This highlights a trade-off between efficiency and robustness. Minimum clearance is similar across planners (0.09–0.11 m), indicating comparable safety margins.

Planner performance per environment is shown in Fig. 3.3, where only successful simulations receive a BARN score, with higher values indicated by warmer colors on the color

TABLE 3.1. **Simulation:** Planner performance and compute-time statistics across all environments. Success metrics are computed over all runs, while traversal time, path length, and minimum clearance are reported over successful runs only (mean ± standard deviation).

Metric	**DWA**	**MPC**	**PF**
Goal rate	209 (70%)	152 (51%)	209 (70%)
Collision rate	70 (23%)	122 (41%)	74 (25%)
Timeout rate	21 (7%)	26 (9%)	17 (6%)
BARN score	0.35 ± 0.23	0.25 ± 0.25	0.35 ± 0.23
Traversal time [s]	20.38 ± 11.79	12.64 ± 6.67	21.10 ± 10.09
Path length [m]	5.82 ± 2.24	4.29 ± 0.23	5.53 ± 1.52
Minimum clearance [m]	0.09 ± 0.09	0.09 ± 0.10	0.11 ± 0.10
Mean [ms]	10.27 ± 0.88	25.59 ± 26.06	0.18 ± 0.04
p95 [ms]	13.57	93.16	0.20
Max [ms]	21.46	191.76	1.12

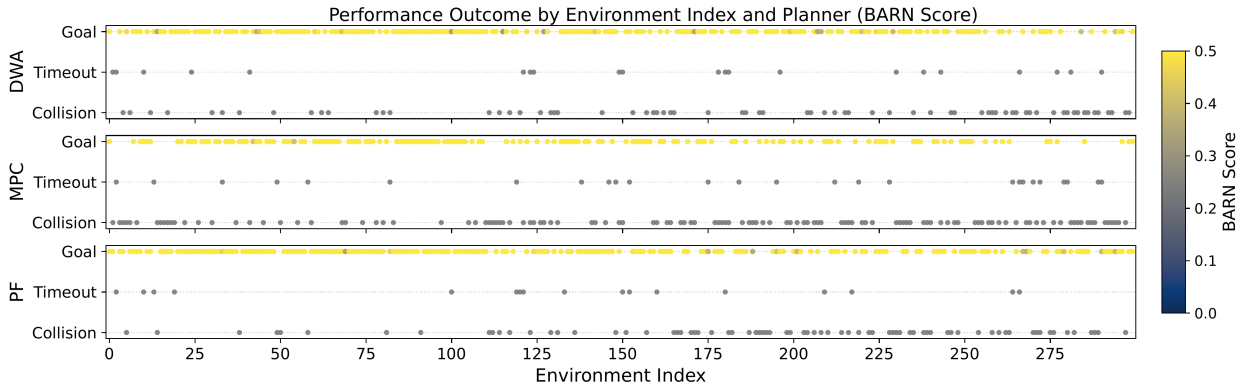


FIGURE 3.3. **Simulation:** Performance outcomes across BARN environments for all planners. Each point is one run; vertical position shows the outcome (collision, timeout, goal), and color indicates BARN score for successful runs.

bar. Collisions cluster more in higher-index, more difficult environments, illustrated by two representative environments and their planner trajectories in Fig. 3.4. Environment 273 is substantially more constrained than Environment 000, with lower average clearance (1.30 vs. 3.68), lower visibility (3.39 vs. 6.85), and a smaller characteristic dimension (1.29 vs. 5.11), along with higher dispersion (4.71 vs. 3.44). In the relatively open Environment 000, all three planners reached the goal. In contrast, Environment 273 produces divergent behaviors. MPC fails to reach the goal, colliding shortly after initiation. DWA reaches the goal but requires an immediate turn to avoid a nearby obstacle. PF maintains greater clearance throughout, resulting in a longer but safer trajectory.

The computational cost of each planner is summarized in Table 3.1. DWA and PF exhibit low and consistent computation times, with mean runtimes of 10.27 ms and 0.18 ms, and minimal variance. MPC shows high variability. Its average runtime is 25.59 ms, but its p95 latency reaches 93.16 ms and its maximum reaches 191.76 ms. It is often fast but can incur substantial latency spikes, which is problematic for real-time deployment since the worst-case delays approach the 0.2 s control loop limit and could degrade or destabilize behavior on hardware. MPC may therefore need additional safeguards or simplification to ensure consistent real-time performance, unlike DWA and PF.

Overall, these results highlight a trade-off between efficiency and robustness. MPC produces more efficient, but computationally expensive, trajectories when successful. DWA and PF favor reliability across a wider range of environments. Performance may also be influenced by the sensitivity of each planner to parameter tuning, particularly in cluttered environments.

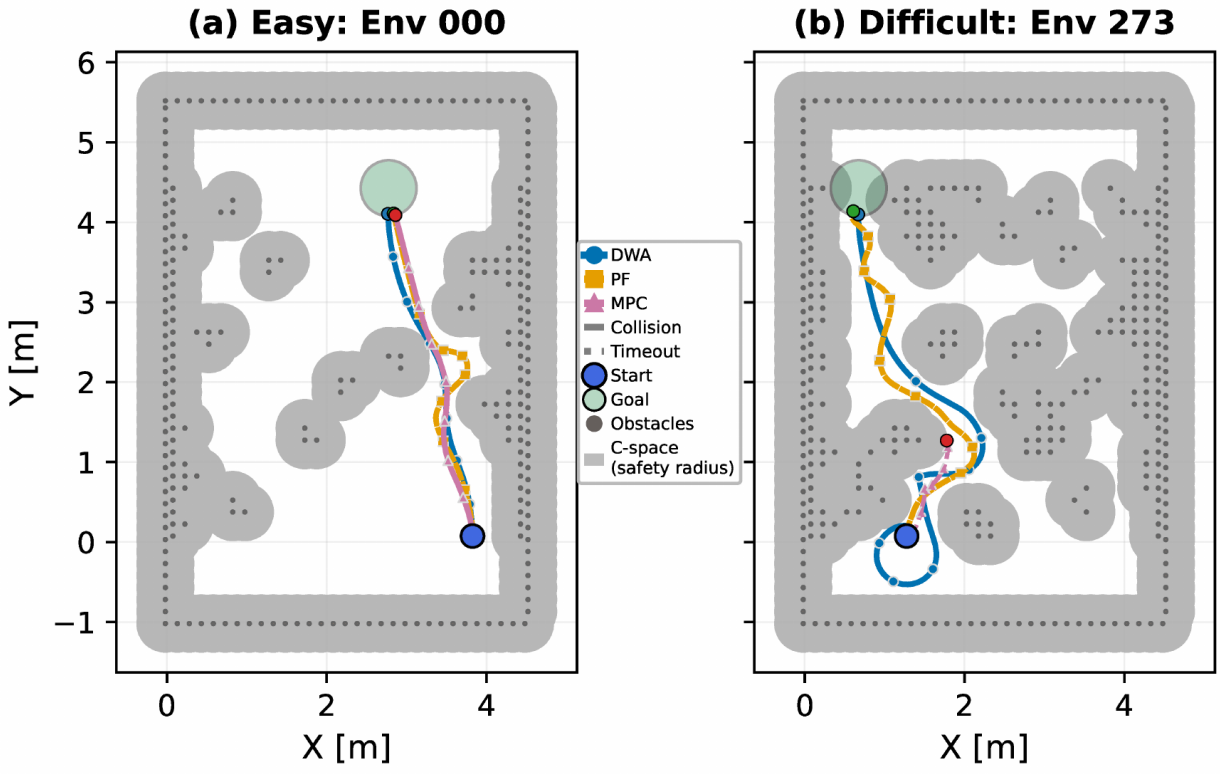


FIGURE 3.4. **Simulation:** Representative BARN environments: (a) sparse with wide passages and (b) dense with narrow corridors. Dashed lines indicate collision.

3.3.2. Hardware Experiments. To evaluate real-world performance, the planners are deployed on hardware in two environments, BARN Env. 042 and 000. Hardware experiments assess execution fidelity and repeatability under real-time constraints and noise, complementing the simulation results. Two experiments were conducted: (1) velocity tracking performance in a simple environment to evaluate the robot's ability to follow commanded velocities, and (2) repeated trials in a single environment to assess planner consistency and robustness on hardware. Physical obstacles were not placed in either experiment. BARN environments contain dozens of obstacles per course, making physical placement impractical at this scale, and the platform carried no onboard perception system, so obstacles would not have been sensed regardless. Obstacle information came instead from the virtual perception model in Section 3.2.4.1, held identical between simulation and hardware.

3.3.2.1. *Velocity Tracking Performance.* Velocity tracking performance is evaluated to isolate the effects of control execution from planning. A simple environment (BARN Env. 042) is chosen due to its direct path to the goal. The robot is commanded to follow planner-generated velocity inputs, and the resulting tracking error is shown in Fig. 3.5. The results indicate low tracking error across all planners, with mean absolute error (MAE) of 0.04 m/s and 0.01 rad/s for DWA, 0.04 m/s and 0.02 rad/s for MPC, and 0.05 m/s and 0.03 rad/s for PF. These errors are small relative to the commanded velocities and are attributed to system dynamics and sensing noise. Overall, the control system demonstrates sufficient accuracy to execute planner commands, suggesting that performance differences observed in more complex environments are primarily due to planning behavior rather than control limitations.

3.3.2.2. *Single Environment Validation.* To evaluate planner consistency on hardware, each planner was tested in a single environment (BARN Env. 000) across five repeated trials. This experiment assessed the repeatability of planner behavior under real-world conditions. Obstacle information is provided through the virtual perception model described in Section 3.2.4.1, while robot pose and goal location are obtained via motion capture.

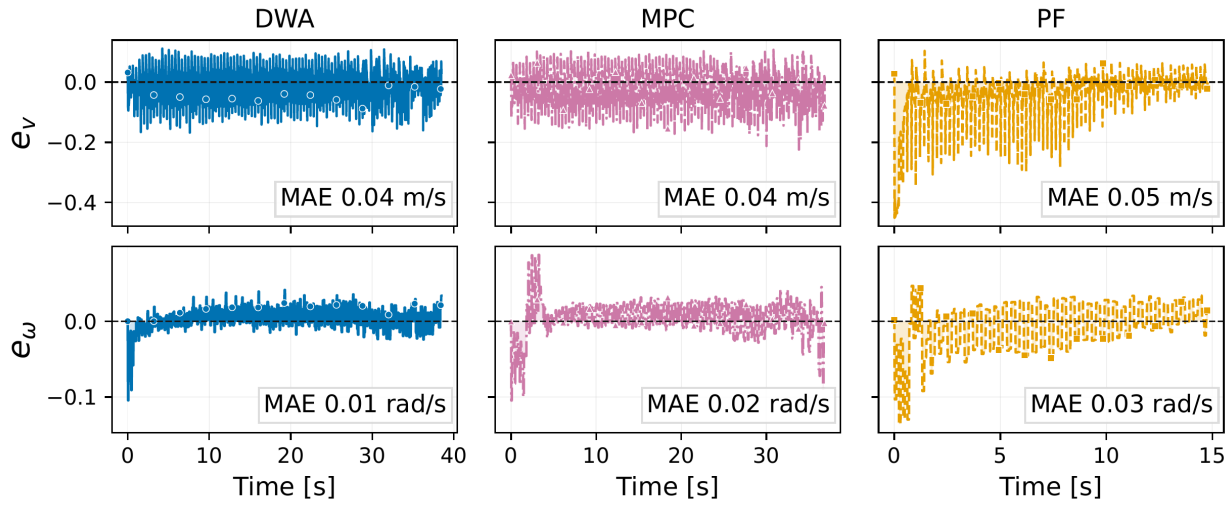


FIGURE 3.5. **Hardware:** Velocity tracking error in BARN Env. 042 for (a) DWA, (b) MPC, and (c) PF. Each panel shows the difference between estimated and commanded linear and angular velocity, with mean absolute error (MAE) annotated for reference.

Figure 3.1 illustrates a representative DWA run, showing both the physical hardware trajectory and the corresponding MuJoCo reconstruction. While the hardware environment contains no physical obstacles, the reconstructed trajectory overlays the virtual obstacle map used for planning, providing a clear visualization of how the robot navigates the environment using the virtual perception model.

All three planners successfully completed BARN Env. 000 across all five trials (100%). Despite identical success rates, clear differences in trajectory efficiency and consistency emerge. PF achieves the highest BARN score of 0.50 ± 0.00 and the lowest traversal time (17.10 ± 0.37 s), while maintaining comparably short paths (4.35 ± 0.03 m). This indicates highly consistent and efficient behavior across repeated runs. DWA exhibits moderate performance, achieving a lower BARN score (0.28 ± 0.02) and significantly higher traversal time (32.54 ± 2.23 s), while producing slightly shorter paths (4.31 ± 0.04 m). In contrast, MPC performs worst across these metrics, with the lowest BARN score (0.26 ± 0.01), highest traversal time (35.10 ± 2.05 s), and longest path length (4.58 ± 0.24 m). In addition to reduced efficiency, MPC exhibits greater variability in trajectory selection across repeated trials, as

shown in Fig. 3.6. While DWA and PF reproduce consistent and repeatable paths, MPC demonstrates noticeable variation between runs, indicating increased sensitivity to noise.

TABLE 3.2. **Hardware:** Performance and compute-time statistics for BARN Env. 000, averaged over five runs.

Metric	**DWA**	**MPC**	**PF**
Goal rate	5 (100%)	5 (100%)	5 (100%)
BARN score	0.28 ± 0.02	0.26 ± 0.01	0.50 ± 0.00
Traversal time [s]	32.54 ± 2.23	35.10 ± 2.05	17.10 ± 0.37
Length [m]	4.31 ± 0.04	4.58 ± 0.24	4.35 ± 0.03
Mean [ms]	10.52 ± 0.12	19.19 ± 9.06	0.19 ± 0.00
p95 [ms]	12.47	44.30	0.26
Max [ms]	21.67	142.77	0.32

The compute-time statistics further highlight these differences. DWA and PF maintain low and consistent runtimes, whereas MPC exhibits significantly higher variability, with a mean runtime of 19.19 ± 9.06 ms, a p95 of 44.30 ms, and a maximum of 142.77 ms. These values remain within the 0.2 s control loop, but the presence of latency spikes is consistent with the variability observed in trajectory execution.

Overall, the hardware results reinforce trends observed in simulation while highlighting the impact of real-world variability. PF achieves the most consistent and efficient performance in this environment. DWA provides stable but less efficient behavior, while MPC exhibits reduced efficiency and increased variability under the current formulation.

A possible explanation for these differences lies in the state information required by each planner. DWA and MPC rely on estimates of the robot's current linear and angular velocities for control computation, making them more sensitive to noise and estimation errors in these quantities. In contrast, the PF method primarily computes control actions based on position information relative to the goal and obstacles, and does not depend on velocity feedback. As a result, PF is less affected by velocity estimation noise, which may contribute to its improved consistency and efficiency in hardware experiments.

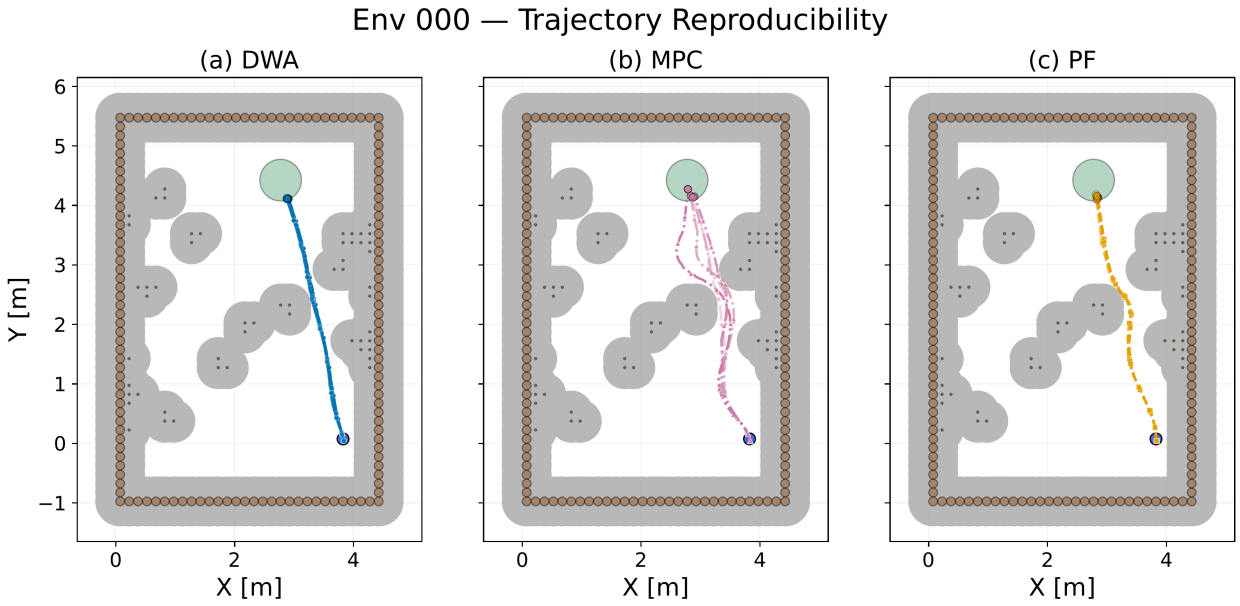


FIGURE 3.6. **Hardware:** Trajectory reproducibility across 5 runs on BARN Env. 000 for (a) DWA, (b) MPC, and (c) PF. All runs successfully reached the goal, with PF producing the most consistent paths and MPC exhibiting the highest path-length variability.

3.4. Conclusion

This chapter presented a reduced-order planning framework and evaluated three local planners: DWA, MPC, and PF. These planners were evaluated using the BARN environments, providing a standardized and validated benchmark for navigation performance. The hierarchical structure itself performed as intended: reducing the robot to a unicycle model at the planning level kept all three planners computationally tractable while running unmodified on the physical quadruped, directly supporting the framework's hardware-validation goal.

Results show that while MPC can produce more efficient trajectories, it exhibits reduced robustness and increased computational variability under the current formulation. In contrast, DWA and PF provide more consistent performance, with PF demonstrating the strongest hardware performance due to its reduced dependence on velocity estimation.

These findings highlight the importance of balancing trajectory optimality with robustness and computational consistency when selecting local planners for real-time robotic systems. Future work will extend evaluations to real-world perception using onboard sensing and physical obstacles. We will also improve the robustness and real-time performance of optimization-based planners, such as MPC, and investigate adaptive, online gain-tuning methods for varying environments.

CHAPTER 4

Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments

Overview: Most planners focus on obstacle avoidance, but legged robots can step over obstacles, reducing travel distance and time. We address this gap by integrating motion control and planning within a sampling-based planner. We develop a constrained inverse-dynamics controller with task-space feedback that tracks center-of-mass velocities via a closed-form solution, thereby avoiding iterative optimization. This enables the use of reduced-order planning models by ensuring dynamic consistency at the full-body level. For planning, we introduce BiKE-Legs and DRiVE-Legs, which augment classical bicycle and differential-drive kinematics with leg-based stepping capabilities. These models are embedded within the Dynamic Window Approach (DWA), with cost functions that balance goal progress, obstacle negotiation, and speed to enable selection between avoidance and stepping. Results across scenarios with small and large obstacles using reduced kinematic models, along with limited validation on a simulated Digit humanoid, demonstrate the effectiveness of the proposed approach.

4.1. Introduction

Legged robots offer the unique ability to place discrete footholds (see Fig. 4.1), enabling them to go around large obstacles (tall blue cylinders) but save time by stepping over small

This chapter is adapted from the accepted manuscript: D. Torres and P. A. Bhounsule, "Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments," accepted to the 6th Modeling, Estimation and Control Conference (MECC 2026), Phoenix, AZ, October 25–28, 2026. The final published version may differ from this chapter. This material is based upon work supported by the National Science Foundation under Grant No. 2128568 to P. A. Bhounsule and by the NSF Graduate Research Fellowship Program under Grant No. 2236870 to D. Torres. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the National Science Foundation.

obstacles (short blue obstacles) to move from the start (green cylinder) to a goal (red cylinder). However, relatively little work has explored stepping over obstacles, as this requires explicit planning of foot placements while accounting for the robot's dynamics. To address this problem, we use a hierarchical approach. At the control level, a computationally efficient constrained inverse dynamics method tracks center-of-mass velocity. At the planning level, 2D kinematic models with leg extensions are integrated into a sampling-based Dynamic Window Approach (DWA). This configuration enables both stepping-over and stepping-around behaviors, allowing the robot to move effectively through cluttered environments.

Motion planning for legged robots requires an underlying model to represent the robot's motion. The simplest models are quasi-static models, which allow for fast computation [**38**]. While computationally efficient, such models often fail to adequately capture the system's dynamics. At the other extreme, full-dynamics models account for nonlinear and hybrid dynamics, but their use in real-time planning is limited by high computational demands [**56**]. Between these two extremes, reduced-order models such as the Linear Inverted Pendulum Model (LIPM) for walking [**76**] and the Spring-Loaded Inverted Pendulum (SLIP) model of running [**110**] provide a useful compromise by retaining essential dynamic behaviors while simplifying computations.

Global motion planning for legged robots has been approached through a variety of search- and sampling-based frameworks that account for terrain feasibility, dynamic feasibility, and locomotion efficiency. Early work demonstrated that footstep primitives derived from full-body simulations could be integrated into an A^* framework, where costs for terrain difficulty, turning, and lateral/longitudinal stepping were optimized to yield energy-efficient step sequences for humanoids [**46**]. More recent efforts have leveraged reduced-order models to generate feasible motion primitives that can be embedded in sampling-based planners, such as RRT-Connect, and that respect both kinematic and dynamic constraints, enabling fast global planning of footstep locations [**72**]. Beyond feasibility and efficiency, safety considerations have also been incorporated, as in the safety-aware informative planners that integrate control barrier functions into the tree expansion process to balance safety with

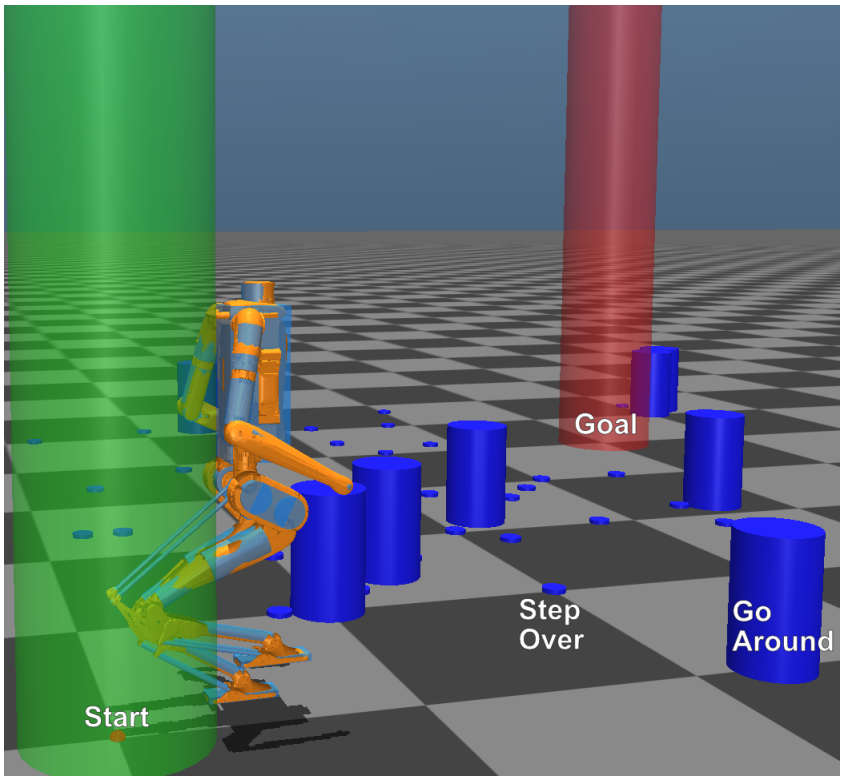


FIGURE 4.1. **Overview:** The BiKE-Legs model (bicycle kinematics enhanced with legs) is used along with the Dynamic Window Approach, a sampling-based approach for motion planning on the humanoid Digit in the MuJoCo simulator.

information gain [**94**]. Hybrid approaches such as the Long Short-Term Planning (LSTP) framework [**48**], which first samples contact sequences and schedules globally, and then refines the solution locally with short-horizon trajectory optimization, offer the best of both worlds.

In the absence of a global map or in changing environments, legged robots should be able to utilize local motion planning for real-time control. Model Predictive Control (MPC) has become a widely adopted approach, where reduced-order models such as the Linear Inverted Pendulum are employed to generate short-horizon trajectories with automatic footstep placement, enabling stable walking on uneven terrain [**39**]. Trajectory optimization methods have also been applied locally, with formulations that continuously refine body and footstep trajectories subject to contact and dynamic constraints [**56**]. Reinforcement learning has provided an alternative, with end-to-end policies trained in simulation to react to local terrain features, producing robust and agile legged behaviors [**75, 47**].

The Dynamic Window Approach (DWA) provides a computationally efficient method for local motion planning by sampling admissible velocity commands and selecting the one that minimizes a cost that weighs goal progress, obstacle avoidance, and speed [**5**]. DWA and its variants have been widely used for car-like and differential-drive navigation [**18, 73**], and more recently applied directly to multi-legged robot navigation [**112**], though without an explicit stepping-over capability. In this work, we extend the DWA to legged locomotion by introducing leg extensions to the bicycle kinematics model (BiKE-Legs) and differential-drive model (DRiVE-Legs) and by designing cost functions that explicitly permit stepping around and over obstacles. Unlike MPC and trajectory-optimization approaches [**39, 56**], which do not explicitly choose between avoiding and stepping over an obstacle, and unlike global footstep-sampling planners [**46, 72**], which require a known map and offline planning, our approach embeds the avoid-or-step decision into a real-time local planner's cost function with no global map and no per-step optimization. Our contributions are as follows: (1) a computationally efficient constrained inverse-dynamics controller for high-fidelity tracking of center-of-mass velocities without online optimization; (2) two leg-enhanced kinematic planning models based on bicycle and differential-drive kinematics; and (3) extensions of DWA that enable obstacle negotiation via stepping, a capability not previously realized in DWA-style local planners. We validate our approach using Agility Robotics' Digit, simulated in MuJoCo [**96**].

4.2. Methods

4.2.1. Motion Control: Constrained Inverse Dynamics.

4.2.1.1. *Digit Model:* The reduced lower-body model shown in Fig. 4.2 has 22 generalized coordinates: six floating-base coordinates $[q_x, q_y, q_z, q_\phi, q_\theta, q_\psi]$ and eight coordinates $[q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8]$ in each leg. Each leg has six actuated DoFs and two passive coordinates, q_5, q_6, associated with the closed-chain leg mechanism. The upper body and torso are lumped as a single rigid body with a combined mass and inertia. The robot's total weight is 48 kg, with about 15 kg in the torso. The robot torso houses the inertial measurement unit

(IMU), and each joint has an encoder that provides the position and velocity. The controller runs at 1 kHz.

4.2.1.2. *State Machine:* As shown in Fig. 4.2, a state machine takes inputs of fore-aft speed $\dot{x}_{\text{ref}}$, lateral speed $\dot{y}_{\text{ref}}$, and yaw rate $\dot{\psi}_{\text{ref}}$ from the motion planner (once per step) and the robot state $\mathbf{q}, \dot{\mathbf{q}}$ (at 1 kHz). The state machine keeps track of phases and transitions (s =single stance/double stance/swing leg) and sets up start and end positions for the swing foot and the torso $\ell_{\text{ref}}, \dot{\ell}_{\text{ref}}$. We assume that the single stance phase is fixed at $T = 0.4$ s and the double stance phase depends on the estimated vertical force in the rear leg computed from the joint control discussed in this section.

4.2.1.3. *Cartesian and Joint Reference Trajectory:* The Cartesian trajectory generator computes the swing foot and torso positions using a fifth-order polynomial with the given initial and final positions from the state machine, and with the initial/final velocities and accelerations set to zero. The joint trajectory generator uses a semi-analytical approach to compute the joint reference angles from the end-effector reference position. The simplification for inverse kinematics is as follows. The loop joint on the leg is approximated as a parallel linkage. Consequently, we solve for the hip pitch q_3 and the knee angle q_4 based on the desired leg length and fore-aft leg position using a root-finding method. Thereafter, the remaining four angles q_1, q_2, q_7, q_8 can be obtained analytically from the desired end-effector angles and the lateral position of the foot. Once the joint reference angles are solved, we compute the joint reference velocities using the Jacobian $\mathbf{J}$ from foot to torso as follows: $\dot{\mathbf{q}}_{\text{ref}} = \mathbf{J}^{-1}\dot{\ell}$.

4.2.1.4. *Joint Control:* The joint control is based on constrained inverse dynamics with task-space feedback, which computes joint torques and constraint forces based on reference joint angles/rates and center-of-mass tracking errors. The dynamics of the Digit are given by

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) = \mathbf{S}^\top \boldsymbol{\tau} + \mathbf{J}^\top \mathbf{F}, \tag{39}$$

where $\mathbf{q} \in \mathbb{R}^{22}$ contains the generalized coordinates, the inertia matrix is $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{22\times 22}$, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} \in \mathbb{R}^{22}$ represents Coriolis and centrifugal terms, and $\mathbf{G}(\mathbf{q}) \in \mathbb{R}^{22}$ is the gravity

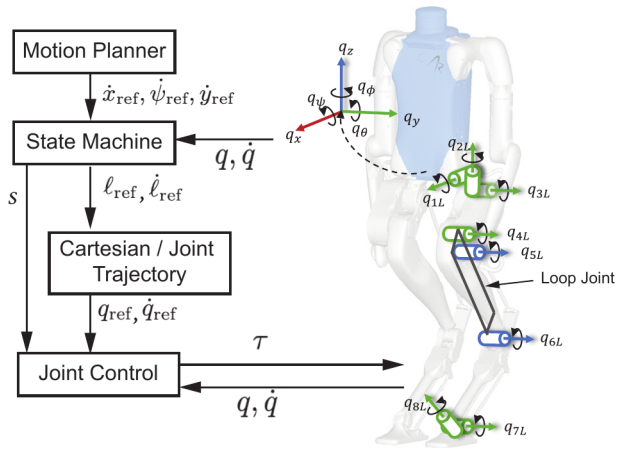


FIGURE 4.2. **Digit model:** The reduced lower-body model has 22 generalized coordinates: six floating-base coordinates and eight coordinates in each leg. Each leg has six actuated DoFs and two passive coordinates associated with the closed-chain leg mechanism. Figure adapted from [**20**].

vector. The actuation selection matrix is $\mathbf{S} \in \mathbb{R}^{12\times 22}$, such that $\boldsymbol{\tau} \in \mathbb{R}^{12}$ are the joint torques applied at the actuated joints. The contact Jacobian is $\mathbf{J}(\mathbf{q})$, written as $\mathbf{J}$ for brevity elsewhere in this chapter, and $\mathbf{F} \in \mathbb{R}^{10}$ denotes the vector of external forces, including 6 contact forces at the feet and 4 constraint forces due to the loop joint on the legs.

The generalized coordinates are partitioned into floating base f and leg ℓ components as $\mathbf{q} = \left[\mathbf{q}_f, \mathbf{q}_\ell\right]^\top, \mathbf{q}_f \in \mathbb{R}^6,\ \mathbf{q}_\ell \in \mathbb{R}^{12}$. Here, $\mathbf{q}_\ell$ contains only the 6 actuated degrees of freedom per leg (12 total); the passive coordinates are excluded from this partition and are instead enforced through the loop-joint constraint forces in $\mathbf{F}$, consistent with the full 22-dimensional $\mathbf{q}$ used in (39). To control the floating-base motion, the dynamics in (39) are partitioned into floating base and leg components:

$$\begin{aligned} \mathbf{M}_f(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}_f(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}_f(\mathbf{q}) &= \mathbf{J}_f^\top \mathbf{F}, \\ \mathbf{M}_\ell(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}_\ell(\mathbf{q},\dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}_\ell(\mathbf{q}) &= \tau_g + \mathbf{J}_\ell^\top \mathbf{F}. \end{aligned} \tag{40}$$

The control objective is to regulate the floating-base motion in task space. Let $\mathbf{X}$ denote the floating-base pose, and define tracking errors $\Delta\mathbf{X} = \mathbf{X}_{\text{ref}} - \mathbf{X}$ and $\Delta\dot{\mathbf{X}} = \dot{\mathbf{X}}_{\text{ref}} - \dot{\mathbf{X}}$. A

desired task-space acceleration is specified using a proportional-derivative control law:

$$\begin{aligned}\ddot{\mathbf{X}}_{\text{des}} &= -\mathbf{K}_*\Delta\mathbf{X} - \mathbf{C}_*\Delta\dot{\mathbf{X}} \\ &= \begin{bmatrix} -C_x(\dot{x} - \dot{x}_{\text{ref}}), -C_y(\dot{y} - \dot{y}_{\text{ref}}), \dots \\ -K_z(\bar{z} - z) - C_z\dot{z}, -K_\phi\phi - C_\phi\dot{\phi}, \dots \\ -K_\theta\theta - C_\theta\dot{\theta}, -C_\psi(\dot{\psi} - \dot{\psi}_{\text{ref}}) \end{bmatrix}^\top\end{aligned}$$

where $\mathbf{K}_*$ and $\mathbf{C}_*$ are user-chosen gains, and $\bar{z}$ is the reference height of the torso from the ground. This is incorporated into the floating-base dynamics, resulting in the following task-space feedback:

$$\mathbf{S}^\top\tau_g + \mathbf{J}^\top\mathbf{F} = \begin{bmatrix} -\mathbf{K}_*\Delta\mathbf{X} - \mathbf{C}_*\Delta\dot{\mathbf{X}} + \mathbf{G}_f(\mathbf{q}_{\text{ref}}) \\ \mathbf{M}_\ell(\mathbf{q}_{\text{ref}})\ddot{\mathbf{q}}_{\text{ref}} + \mathbf{C}_\ell(\mathbf{q}_{\text{ref}}, \dot{\mathbf{q}}_{\text{ref}})\dot{\mathbf{q}}_{\text{ref}} + \mathbf{G}_\ell(\mathbf{q}_{\text{ref}}) \end{bmatrix}$$

where $\mathbf{q}_{\text{ref}}, \dot{\mathbf{q}}_{\text{ref}}, \ddot{\mathbf{q}}_{\text{ref}}$ denote reference joint trajectories. For simplicity, we set $\mathbf{C}_\ell(\mathbf{q}_{\text{ref}}, \dot{\mathbf{q}}_{\text{ref}})\dot{\mathbf{q}}_{\text{ref}} = \mathbf{0}$ and $\mathbf{M}_\ell(\mathbf{q})\ddot{\mathbf{q}}_{\text{ref}} = \mathbf{0}$ in our controller. The equations can then be simplified and solved for the unknowns $\lambda = [\tau_g, \mathbf{F}]$:

$$\mathbf{A}\lambda = \mathbf{b} \quad \Rightarrow \lambda = \mathbf{A}^+\mathbf{b} \tag{41}$$

where $\mathbf{b} = \left[-\mathbf{K}_*\Delta\mathbf{X} - \mathbf{C}_*\Delta\dot{\mathbf{X}} + \mathbf{G}_f(\mathbf{q}_{\text{ref}}), \mathbf{G}_\ell(\mathbf{q}_{\text{ref}})\right]^\top$, $\mathbf{A} = [\mathbf{S}^\top, \mathbf{J}^\top]$, and $\mathbf{A}^+$ is the pseudoinverse of $\mathbf{A}$. The forces $\mathbf{F}$ and consequently the Jacobian $\mathbf{J}$ in these equations are modified based on the phase as follows:

- **Double stance:** $\mathbf{F}$ consists of contact forces at both feet (3 forces per foot) and loop-closure constraint forces (2 per leg), yielding $\mathbf{F} \in \mathbb{R}^{10}$.
- **Single stance:** $\mathbf{F}$ consists of a 6D contact wrench (3 forces and 3 moments) at the stance foot, along with loop-closure constraint forces (2 per leg), again yielding $\mathbf{F} \in \mathbb{R}^{10}$.

Equation 41 is solved to obtain the torques τ_g, which provide gravity compensation, and the forces $\mathbf{F}$, which include constraint and reaction forces. Some of these forces may be used for transitions from double stance to single stance and for safety checks, such as detecting

whether the robot is falling. Finally, to enable stable joint control, additional feedback torques are added at each joint as follows.

$$\tau = \tau_g + \mathbf{K}_q(\mathbf{q}_{\text{ref}} - \mathbf{q}) + \mathbf{C}_q(\dot{\mathbf{q}}_{\text{ref}} - \dot{\mathbf{q}}) \tag{42}$$

where $\mathbf{K}_q$ and $\mathbf{C}_q$ are user-defined position and derivative gains at the joint, respectively.

4.2.2. Motion Planning: Dynamic Window Approach.

4.2.2.1. *Review of Two Kinematic Models:* The kinematic bicycle model and the differential-drive car model **[6]** are given by the following equations.

$$\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta,$$

$$\dot{\theta} = \begin{cases} \dfrac{v}{L}\tan\phi, & \text{(bicycle kinematics, controls } v, \phi) \\ \omega, & \text{(differential-drive car, controls } v, \omega) \end{cases} \tag{43}$$

Here, the center of mass has global position (x, y) and heading or forward direction with respect to the x-axis θ. Both models share the forward body-frame speed v as a control input. The bicycle model uses steering angle ϕ, whereas the differential-drive model uses the angular speed ω as the second control input. In the bicycle model, we define the wheelbase L as the distance between the front and rear axles.

4.2.2.2. *Two Kinematic Models with Leg Enhancement:* Figure 4.3 shows the modifications to these bicycle and differential drive models to obtain the BiKE-Legs model, **Bi**cycle **K**inematics **E**nhanced with **Legs** and DRiVE-Legs, the **D**ifferential d**RiV**e **E**nhanced with **Legs**, respectively. The center of mass is located at (x, y). The orientation of the x-axis of the body frame (dashed line) is at an angle θ, the heading angle, to the global x-axis (points to the right). The bipedal robot has made contact with the front foot at (x_f, y_f) with a step length of L. Since we are showing distance from the center of mass to the foot, this corresponds to half the step length, $0.5L$, as shown in the figure. For the BiKE-Legs model, the leg orientation is at an angle $\psi = \phi$ to the body heading angle, as shown. The DRiVE-Legs

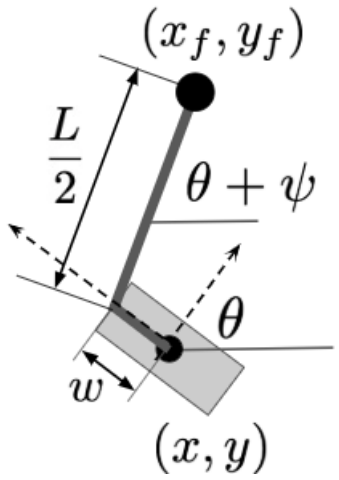


FIGURE 4.3. **Two kinematic models for planning (top-down view):** The center of mass is (x, y) and foot position is (x_f, y_f). The body frame shown using dashed lines is at an angle θ to the global x-axis (not shown). w is the half body width, the lateral offset from the center of mass to the body edge. In the BiKE-Legs, the **Bi**cycle **K**inematics **E**nhanced with **Legs** $\psi = \phi$ and the DRiVE-Legs, the **D**ifferential d**RiV**e **E**nhanced with **Legs** $\psi = \omega T$ with step time T.

model instead uses the leg orientation angle $\psi = \omega T$, where T is the step time. Both models define the step velocity as $v = \frac{L}{T}$. The equations of the BiKE-Legs and DRiVE-Legs models are given by

$$\dot{x} = \frac{L}{T}\cos\theta,\ \dot{y} = \frac{L}{T}\sin\theta, \tag{44}$$

$$x_f = x - s\,w\,\sin\theta + L\cos(\theta + \psi), \tag{45}$$

$$y_f = y + s\,w\,\cos\theta + L\sin(\theta + \psi) \tag{46}$$

$$\dot{\theta} = \begin{cases} \frac{1}{T}\tan\phi,\ \psi = \phi & \text{BiKE-Legs} \\ \omega,\ \psi = \omega T & \text{DRiVE-Legs} \end{cases} \tag{47}$$

where $s = 1$ if the left leg makes contact with the ground and $s = -1$ if the right leg makes contact with the ground. Once the foot makes contact with the ground, it stays at that location for a duration equal to the step time T.

4.2.2.3. *Dynamic Window Approach (DWA):.* The DWA minimizes a cost over a horizon $k = 1, 2, \ldots, N$ steps subject to bounds on the controls.

$$\textbf{BiKE-Legs:} \quad \min_{\{L_k,\phi_k\}_{k=1}^N} J = \sum_{k=1}^{N} C(L_k, \phi_k) \tag{48}$$

$$\text{s.t.} \quad L_{\min} \leq L_k \leq L_{\max},$$

$$\phi_{\min} \leq \phi_k \leq \phi_{\max},$$

$$\textbf{DRiVE-Legs:} \quad \min_{\{L_k,\omega_k\}_{k=1}^N} J = \sum_{k=1}^{N} C(L_k, \omega_k) \tag{49}$$

$$\text{s.t.} \quad L_{\min} \leq L_k \leq L_{\max},$$

$$\omega_{\min} \leq \omega_k \leq \omega_{\max}$$

Let the goal position be (x_g, y_g), and let each circular obstacle projection be parameterized by a center (x_i, y_i) and radius r_i. If the Euclidean distance between two points (x_a, y_a) and (x_b, y_b) is given by $d(a, b) = \sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$ then

$$C = C_{\text{goal}} + \alpha C_{\text{speed}} + \beta C_{\text{body}} + \gamma C_{\text{foot}}, \tag{50}$$

$$C_{\text{goal}} = d(x, y;\, x_g, y_g), \tag{51}$$

$$C_{\text{speed}} = -L, \tag{52}$$

$$C_{\text{body}} = \left| \min_i \left(0,\ d(x, y;\, x_i, y_i) - r_i - r_b^{(i)}\right) \right|, \tag{53}$$

$$C_{\text{foot}} = \left| \min_i \left(0,\ d(x_f, y_f;\, x_i, y_i) - r_i - r_b^{(i)}\right) \right|, \tag{54}$$

$$r_b^{(i)} = \begin{cases} r_{\text{buffer}}, & r_i > r_{\text{thresh}} \quad \text{(large obstacle)}, \\ 0, & r_i \leq r_{\text{thresh}} \quad \text{(small obstacle)}. \end{cases} \tag{55}$$

The total cost C in Eqn. 50 is composed of four terms. C_{goal} (Eqn. 51) penalizes distance to the target position (x_g, y_g), encouraging progress toward the goal. C_{speed} (Eqn. 52) rewards larger step lengths L, promoting faster motion. C_{body} (Eqn. 53) penalizes collisions of the robot's body with large obstacles, where a buffer $r_b^{(i)}$ (Eqn. 55) is added to obstacle radii to maintain safe clearance. C_{foot} (Eqn. 54) penalizes collisions of the robot's feet with small obstacles, allowing the planner to account for stepping over them; no additional buffer is added in this case. The weighting factors α, β, γ balance the contributions of speed and collision avoidance relative to goal-reaching.

4.2.2.4. *Mapping the Two Kinematic Models to Humanoid:* In the humanoid model shown in Fig. 4.2, we have three control inputs from the motion planner. These are the fore-aft speed $\dot{x}_{\text{ref}}$, the lateral speed $\dot{y}_{\text{ref}}$, and the yaw rate $\dot{\psi}_{\text{ref}}$. These are used as follows.

BiKE-Legs constrains yaw motion $\dot{\psi}_{\text{ref}} = 0$ while the other two velocities are mapped to L, ϕ as follows.

$$L = T\sqrt{\dot{x}_{\text{ref}}^2 + \dot{y}_{\text{ref}}^2}, \quad \phi = \text{atan2}(\dot{y}_{\text{ref}}, \dot{x}_{\text{ref}}). \tag{56}$$

DRiVE-Legs constrains the lateral motion $\dot{y}_{\text{ref}} = 0$, while the other two velocities are mapped to L, ω as follows.

$$L = T\dot{x}_{\text{ref}}, \quad \omega = \dot{\psi}_{\text{ref}}. \tag{57}$$

4.3. Results

4.3.1. Motion Planning Using Kinematic Models in Python. In this section, we present motion-planning simulations using the two kinematic models with leg enhancement. All terrains were generated in a 4×4 grid cluttered with N_{obs} user-specified circular obstacles. A fraction f_{obs} were large obstacles with radius $r_0 = 0.15$ m, and the remaining were small obstacles with $r_0 = 0.05$ m. It is assumed that the large obstacles should be avoided, as they are too tall or wide for the robot to step over, and that the small obstacles can be stepped

Type	Variable	BL	DL	BL	DL	BL	DL	BL	DL	BL	DL
Control	Total Obstacles (N_{obs})	60	60	120	120	60	60	60	60	60	60
	Fraction Big Obstacles (f_{obs})	0.20	0.20	0.20	0.20	0.40	0.40	0.20	0.20	0.20	0.20
	Steps for Planning (N_{plan})	2	2	2	2	2	2	4	4	1	1
Metrics	Avg. Steps	11.20	12.00	18.60	13.80	12.00	12.40	14.40	12.60	14.40	12.20
	Avg. Compute Time (s)	0.49	0.46	0.92	0.92	0.48	0.45	0.88	0.91	0.23	0.23
	Max. Compute Time (s)	0.62	0.48	1.19	1.05	0.75	0.54	1.00	1.07	0.37	0.25
	Avg. Step Length (m)	0.50	0.49	0.49	0.48	0.50	0.49	0.50	0.49	0.45	0.47
	Avg. Heading (rad)	0.04	–	0.02	–	0.06	–	-0.02	–	-0.01	–
	Avg. Angular Vel. (rad/s)	–	0.03	–	-0.04	–	-0.03	–	0.14	–	0.12
	Stretch Factor (SF)	1.08x	1.13x	1.76x	1.31x	1.17x	1.21x	1.40x	1.22x	1.27x	1.10x

TABLE 4.1. Performance of BiKE-Legs (BL) and DRiVE-Legs (DL) across various obstacle size/fraction and DWA planning horizons.

over. Obstacle height is not modeled explicitly in this 2D framework; radius is used as a proxy for step-over feasibility, a simplification discussed further in Section 4.4.

For all simulations, we used a start location $x(0) = y(0) = -1.8$ m and orientation $\theta(0) = \frac{\pi}{4}$ rad, and goal location $x_g = y_g = 1.8$ m. The controller was updated at a fixed time step of $T = 0.4$ s, and the dynamics were integrated via Euler's method with $dt = 0.02$ s. Half the body width is $w = 0.125$ m, as shown in Fig. 4.3. The step length bounds are $L_{\min} = 0.1$ m and $L_{\max} = 0.5$ m, the orientation bounds are $\phi_{\min} = -\frac{\pi}{4}$ rad, $\phi_{\max} = \frac{\pi}{4}$ rad, and the angular velocity bounds are $\omega_{\min} = -2$ rad/s, $\omega_{\max} = 2$ rad/s. The algorithm terminates when the step count reaches the maximum $N_{\max} = 50$ or $\sqrt{(x(N) - x_g)^2 + (y(N) - y_g)^2} < 1.5r_g$ where $r_g = 0.15$ m, whichever is earlier. We use the following parameters for DWA: $\alpha = 10$, $\beta = \gamma = 100$, $r_{\text{buffer}} = 0.05$ m, $r_{\text{thresh}} = 0.1$ m.

Table 4.1 sweeps three parameters (N_{obs}, f_{obs}, N_{plan}), each averaged over 5 random scenes, on a 2.8 GHz Intel Core i7 MacBook in Python 3.9.

Table 4.1 reports average step count, compute time, average control parameters, and stretch factor (SF), the ratio of actual path length to the Euclidean distance to goal.

At baseline ($N_{\text{obs}} = 60$, $f_{\text{obs}} = 0.2$, $N_{\text{plan}} = 2$), BL and DL perform comparably across all metrics, with SF near 1.1 for both, indicating near-straight trajectories.

With $N_{\text{obs}} = 120$, both models take more steps and compute time, but BL degrades more sharply: 18.6 steps and $SF = 1.76\times$ vs. DL's 13.8 steps and $SF = 1.31\times$, suggesting BL is more sensitive to clutter under the same planning horizon.

With $f_{\text{obs}} = 0.4$, stretch factor increases moderately for both models but less severely than raising N_{obs}, suggesting obstacle count has more impact on path quality than size distribution.

Varying the planning horizon reveals a cost-quality trade-off. $N_{\text{plan}} = 4$ nearly doubles compute time without improving SF (matched baseline: $1.08\times/1.13\times$; $N_{\text{plan}} = 4$: $1.40\times/1.22\times$). The reactive controller ($N_{\text{plan}} = 1$) halves compute time but yields slightly higher step counts and SF.

Across all conditions, the average step length stays near its upper bound (0.5 m), meaning both planners use maximum stride when possible. The average heading (BL) and angular velocity (DL) stay near zero, consistent with the robot starting roughly oriented toward the goal. Under the current Python implementation, only the reactive controller ($N_{\text{plan}} = 1$) consistently meets the 1 Hz control loop rate.

The DWA uses a greedy per-step rollout rather than a joint horizon search, so cost scales linearly with N_{plan}: 4 samples of L and 8 of ϕ or ω give 32 evaluations per step, so $N_{\text{plan}} = 4$ requires roughly twice the computation time of $N_{\text{plan}} = 2$, matching Table 4.1. A compiled implementation (C/C++) is expected to reduce per-evaluation cost substantially; Python-to-compiled speedups of one to two orders of magnitude are typical for sample-and-evaluate loops of this kind.

4.3.2. Motion Control and Planning on Digit Humanoid. All Digit simulations were performed using the C version of MuJoCo 2.0, while the planner ran in Python 3.9 via a client-server framework.

Figure 4.4 shows tracking of changing fore-aft, lateral, and yaw-rate velocity references (dashed, input mid-step) against step-averaged actual velocities (solid red). Yaw-rate tracking is high-fidelity; fore-aft and lateral show errors of about 0.1 m/s and 0.2 m/s, respectively. Higher gains would reduce these errors but increase instability risk.

Next, we verified the BiKE-Legs kinematic model in simulation. We had to reduce the fore-aft step length to 0.2 m while keeping the step time at 0.4 s, as larger step lengths led to instability. As observed in our earlier work, the lateral motion is less stable than the

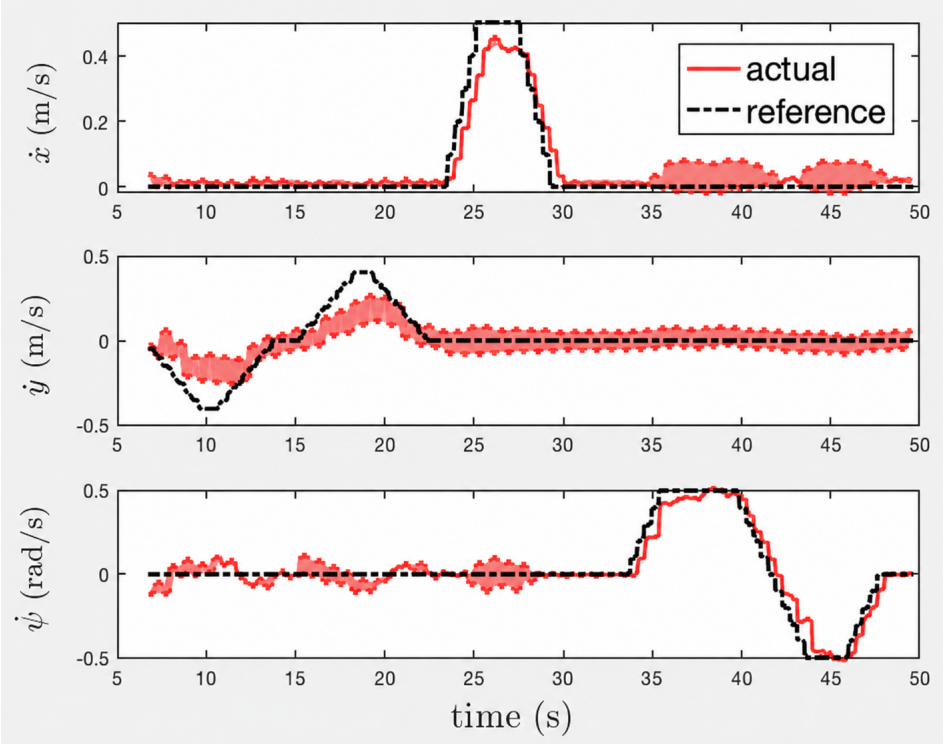


FIGURE 4.4. **Motion Control:** Tracking velocity of the center of mass using constrained inverse dynamics with task-space feedback control.

fore-aft motion in bipeds [**42**]. This implies that if the robot takes a larger side step (large ϕ), it should decrease its speed to ensure stable walking. To circumvent this, we added a maneuver cost $C_{\text{maneuver}} = C_m L|\phi|^p$, with $p = 1, 2, \ldots$, where C_m is a constant and this term is added to Eqn. 50. A higher p indicates a more severe coupling. Figure 4.5 shows results for Digit in simulation for $N_{\text{obs}} = 60$, $f_{\text{obs}} = 0.2$, $N_{\text{plan}} = 3$.

4.4. Conclusion and Future Work

This chapter shows that a constrained inverse dynamics controller with task-space feedback can track center-of-mass reference motions well enough to support 2D kinematic planning models, BiKE-Legs and DRiVE-Legs, for stepping over and around obstacles within the DWA framework.

The current 2D model does not account for obstacle height, vertical foot clearance, or kinematic reachability; using obstacle radius alone as a proxy for step-over feasibility can

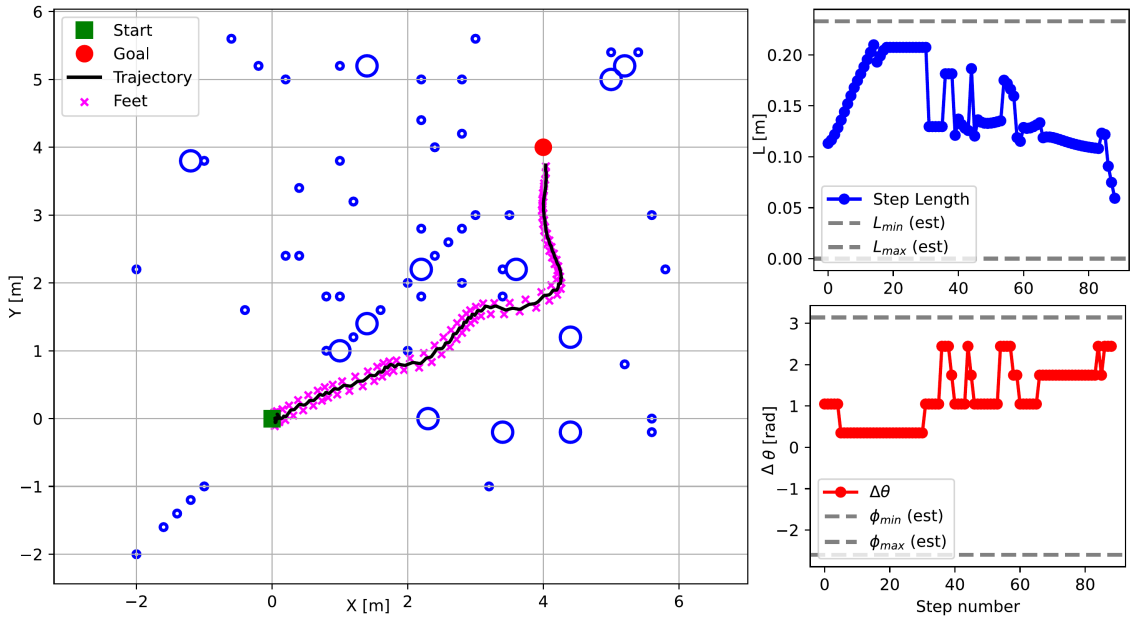


FIGURE 4.5. Results for $N_{\text{obs}} = 60$, $f_{\text{obs}} = 0.2$, $N_{\text{plan}} = 3$ with C_{maneuver} for the BiKE-Legs model in MuJoCo. Small circles mark obstacles the planner may step over; large circles mark obstacles it must avoid. The trajectory and foot markers pass directly over the small obstacles near $(2, 1)$ and $(2, 2)$, demonstrating stepping-over, while visibly routing around every large obstacle along the path.

therefore be optimistic, allowing the planner to attempt steps that the full-order robot cannot physically clear. It also ignores gait stability considerations, limiting the execution of aggressive stepping motions. The DWA cost weights and thresholds were manually selected; a systematic sensitivity analysis is needed to establish how robust the planner's performance and stepping behavior are to these choices. Future work will incorporate reachability constraints, stability constraints, and foot geometry, along with more efficient implementations to enable longer real-time planning horizons, leading to hardware validation.

CHAPTER 5

Interpretable Adaptive Dynamic Window Approach for Legged Robot Local Navigation

Overview: The Dynamic Window Approach (DWA) is a widely used local planner, but its objective weights are fixed scalars that cannot serve every navigation condition. This chapter keeps DWA's structure and makes its weighting adaptive: a PPO policy selects among five predefined weight profiles using compact local features. The trained policy is distilled into a shallow decision tree (DST-DWA) so that every runtime decision can be traced through explicit feature-threshold rules. In practice, the RL policy used only three of the five profiles. In a two-dimensional simulator on Benchmark for Autonomous Robot Navigation (BARN) environments, Baseline DWA achieved 40.0% success on held-out environments, the neural policy (RL-DWA) achieved 60.0%, and DST-DWA achieved 68.0%, with 90.5% imitation fidelity. When the distilled tree is transferred unchanged to a quadruped in MuJoCo on 300 BARN environments, the advantage persists in direction but is statistically borderline (+4.7 percentage points, McNemar $p = 0.059$). A margin decomposition shows the gain comes from rescuing environments where fixed weights fail, and the tree's explicit mode log localizes its dominant failure to over-caution in the obstacle-safe profile.

5.1. Introduction

Autonomous mobile robots are increasingly used in homes, warehouses, and outdoor environments, where they must navigate through cluttered and partially unknown spaces. A reliable local navigation framework should therefore support safe motion, efficient goal reaching, generalization across different obstacle layouts, and interpretability for human debugging and verification. Classical local planners provide structured and computationally efficient motion generation, but their performance often depends strongly on manually selected parameters. This limits their ability to adapt to diverse navigation conditions.

The Dynamic Window Approach (DWA) is a widely used local planner that searches directly in the robot's velocity space [**5**]. At each control step, DWA samples dynamically feasible linear and angular velocity commands, forward-simulates short trajectories, rejects unsafe commands, and selects the best command using an objective function that typically balances goal alignment, obstacle clearance, and forward speed. Although DWA is simple and efficient, its behavior is highly dependent on fixed objective-function weights. A single hand-tuned weight profile may perform well in one environment but fail in another. For example, open spaces may benefit from speed-focused behavior, whereas narrow passages require stronger emphasis on obstacle clearance and cautious motion.

5.1.1. Related Work. Several studies have investigated automatic DWA parameter tuning using optimization, fuzzy logic, and learning-based methods. Offline optimization methods, such as Genetic Algorithms and Multi-Objective Particle Swarm Optimization, have been used to tune DWA weights for faster and safer navigation [**60, 24**]. However, these methods often produce static parameter sets and may not generalize well to new obstacle configurations. Adaptive methods based on fuzzy logic or neuro-fuzzy systems adjust DWA parameters online using local geometric information [**95, 10**]. These approaches improve adaptability and provide rule-based structure, but their behavior still depends on manually designed rules and membership functions.

Other DWA variants redesign the objective or gains directly, for example using Lyapunov stability criteria [**1**], scaling gains with a measure of local environment complexity [**107**], or combining DWA with a global planner in a hybrid pipeline [**9**]. DWA has also been adapted specifically for legged platforms, including multi-legged robots [**112**] and quadrupeds [**21**], though these adaptations still rely on fixed or hand-tuned modifications rather than a learned, auditable selection policy. Decision trees have also been combined with DWA for a different purpose, partitioning the locally sensed region to guide obstacle avoidance rather than making a learned parameter-selection policy interpretable [**59**]. Other work addresses robustness to actuation uncertainty through deterministic sampling of the dynamic window under stochastic dynamics [**8**].

More recent methods use Bayesian optimization or deep reinforcement learning to tune planner parameters [**74**]. For example, constrained Bayesian optimization has been applied to online sampling of kinodynamically feasible controls [**81**], and learning-based methods have been used to map laser observations to continuous DWA parameters [**27, 4**]. Reinforcement learning has also been used to select DWA parameters directly in unknown environments [**3**], and a hierarchical perspective on RL-based planner-parameter tuning has been proposed more generally [**63**]. While these methods can improve navigation performance, neural parameter selectors can make individual parameter choices difficult to inspect directly. This concern is consistent with the broader distinction between predictive performance and model interpretability in the interpretable-machine-learning literature [**69, 11**].

A closely related direction is Adaptive Planner Parameter Learning (APPL), where a learning policy predicts planner parameters instead of directly producing low-level control commands. APPL methods have been studied using demonstrations, human interventions, evaluative feedback, reinforcement learning, and more recently vision-language-action models [**102, 100, 99, 106, 64**]. These methods preserve the classical planner as the final command generator while improving adaptability. However, many APPL variants still rely on neural policies or high-dimensional observations, which makes the learned parameter-selection behavior difficult to interpret directly.

Other recent approaches attempt to overcome DWA's local-horizon limitations by combining DWA with high-level reasoning models, such as large language models [**87**]. These methods can provide long-horizon waypoint guidance, but the high inference latency of large models prevents them from being used inside fast reactive control loops. Therefore, there remains a need for local planner adaptation methods that are both reactive and interpretable.

In this chapter, we propose an interpretable Reinforcement Learning (RL) based DWA framework (RL-DWA) for local navigation in Benchmark for Autonomous Robot Navigation (BARN) style cluttered environments [**91, 7**]. Instead of learning velocity commands directly, a Proximal Policy Optimization (PPO) policy [**86**] selects among a discrete set of DWA weight profiles. The selected profile changes the behavior of the DWA planner, while DWA

still computes the final dynamically feasible velocity command. To improve interpretability, the trained RL policy is distilled into a shallow decision tree using compact local features derived from LiDAR-like observations and robot state. The resulting decision-tree-distilled DWA policy (DST-DWA) provides human-readable rules that explain when different DWA modes, such as obstacle-safe or speed-focused behavior, are selected.

The main contributions of this chapter are:

- We formulate adaptive DWA weight selection as a reinforcement learning problem, where the policy selects discrete DWA behavior modes instead of directly outputting robot velocity commands.
- We design a compact local feature representation based on LiDAR-derived clearance measures, heading error, goal distance, and robot velocity, without using global map information.
- We distill the trained RL policy into a shallow decision tree to obtain interpretable rules for DWA mode selection.
- We evaluate Baseline DWA (weight selection in DWA is fixed), RL-DWA, and DST-DWA on held-out BARN-style environments and more challenging generalized environments using standard navigation performance metrics. We also analyze the decision tree to explain the model's weight-profile selection behavior.
- We perform an initial cross-platform transfer study by testing the policy, originally trained and distilled using a differential-drive robot in a Python-based simulator, in a high-fidelity MuJoCo simulation [**96**] with a quadruped robot.

The remainder of this chapter is organized as follows. Section 5.2 describes the local navigation setup. Section 5.3 presents the RL-DWA framework and decision-tree-based policy distillation method. Section 5.4 reports the navigation performance results and interpretable policy analysis. Section 5.5 presents the quadruped transfer study in MuJoCo. Finally, Section 5.6 concludes the chapter and discusses future directions.

5.2. Local Navigation Setup

This section describes the simulation setup used to develop and evaluate the proposed RL-DWA framework. All experiments are conducted in a custom Python-based 2D simulator with a differential-drive robot navigating in BARN-style cluttered environments. Baseline DWA, RL-DWA, and DST-DWA are evaluated under the same robot model, sensing setup, obstacle representation, and termination conditions. The following subsections describe the robot model, local sensing, the DWA planner, the BARN environments and evaluation metrics, and implementation details.

5.2.1. Differential-Drive Robot Model. The navigation system is evaluated using a differential-drive mobile robot operating in two-dimensional cluttered environments. The robot is modeled with a circular footprint of radius R and its planar configuration is represented as

$$\mathbf{x}_t = [x_t, y_t, \theta_t]^T, \tag{58}$$

where x_t and y_t denote the robot position and θ_t denotes its heading angle. The control input is defined as

$$\mathbf{u}_t = [v_t, \omega_t]^T, \tag{59}$$

where v_t is the linear velocity and ω_t is the angular velocity.

The robot motion follows the unicycle kinematic model [**25**]:

$$\dot{x}_t = v_t \cos\theta_t, \quad \dot{y}_t = v_t \sin\theta_t, \quad \dot{\theta}_t = \omega_t. \tag{60}$$

For simulation, the dynamics are discretized using Euler integration with time step Δt:

$$x_{t+1} = x_t + \Delta t\, v_t \cos\theta_t, \tag{61}$$

$$y_{t+1} = y_t + \Delta t\, v_t \sin\theta_t, \tag{62}$$

$$\theta_{t+1} = \theta_t + \Delta t\, \omega_t. \tag{63}$$

The commanded velocities are constrained by the robot's velocity and acceleration limits, following standard kinodynamic trajectory generation for wheeled robots **[68]**:

$$v_t \in [v_{\min}, v_{\max}], \quad \omega_t \in [-\omega_{\max}, \omega_{\max}]. \tag{64}$$

5.2.2. Local Sensing and Obstacle Representation. The robot receives simulated LiDAR-like range observations from its local surroundings. These observations provide obstacle distance measurements in the robot-centered frame and are used for both local planning and interpretable feature extraction. Invalid or missing range readings are clipped to the maximum sensing range.

Obstacles are represented as circular regions in the two-dimensional environment. Collision checking is formulated in configuration space (C-space), where each obstacle is inflated by the radius of the robot's circular footprint. The robot can therefore be treated as a point moving among the inflated configuration-space obstacles **[62]**.

The proposed learning framework does not rely on a global obstacle map or privileged simulator information for policy input. The RL and decision-tree policies use only local, LiDAR-derived navigation features, goal-relative quantities, and the current robot motion state.

5.2.3. Dynamic Window Approach. DWA is used as the underlying local planner **[5]**. At each control step, DWA samples candidate velocity commands (v, ω) from a dynamically feasible set. The dynamic window is defined as

$$V_d = V_s \cap V_a, \tag{65}$$

where V_s denotes the set of velocities allowed by the robot's static velocity limits and V_a denotes the set of velocities reachable under acceleration constraints within one control interval.

For each candidate command $\mathbf{u}_t = (v_t, \omega_t)$, DWA forward-simulates a short trajectory using the robot kinematic model. The candidate trajectory is then evaluated using a weighted objective function:

$$J(\mathbf{u}_t; \boldsymbol{\theta}) = w_g G(\mathbf{u}_t) + w_c C(\mathbf{u}_t) + w_v V(\mathbf{u}_t), \tag{66}$$

where $G(\mathbf{u}_t)$ measures goal-alignment behavior, $C(\mathbf{u}_t)$ measures obstacle clearance, and $V(\mathbf{u}_t)$ encourages forward motion. The DWA weight vector is

$$\boldsymbol{\theta} = [w_g, w_c, w_v]. \tag{67}$$

The final velocity command is selected as

$$\mathbf{u}_t^* = \arg\max_{\mathbf{u}_t \in V_d} J(\mathbf{u}_t; \boldsymbol{\theta}). \tag{68}$$

In the classical Baseline DWA, $\boldsymbol{\theta}$ remains constant throughout navigation. While this approach is simple and interpretable, a single fixed weight combination may not be suitable for all local navigation conditions. For example, open regions may benefit from speed-focused behavior, whereas narrow passages may require increased emphasis on obstacle clearance. This motivates the adaptive DWA weight-profile selection framework introduced in the following section.

5.2.4. BARN Environments and Evaluation Metrics. Experiments are conducted in BARN-style two-dimensional cluttered navigation environments. The environments use a spatial scale of 0.15 m per grid unit, with obstacles at their full specified sizes. Each environment contains narrow passages and obstacle configurations designed to challenge local navigation planners. The robot is initialized at a start pose and must reach a specified goal without collision.

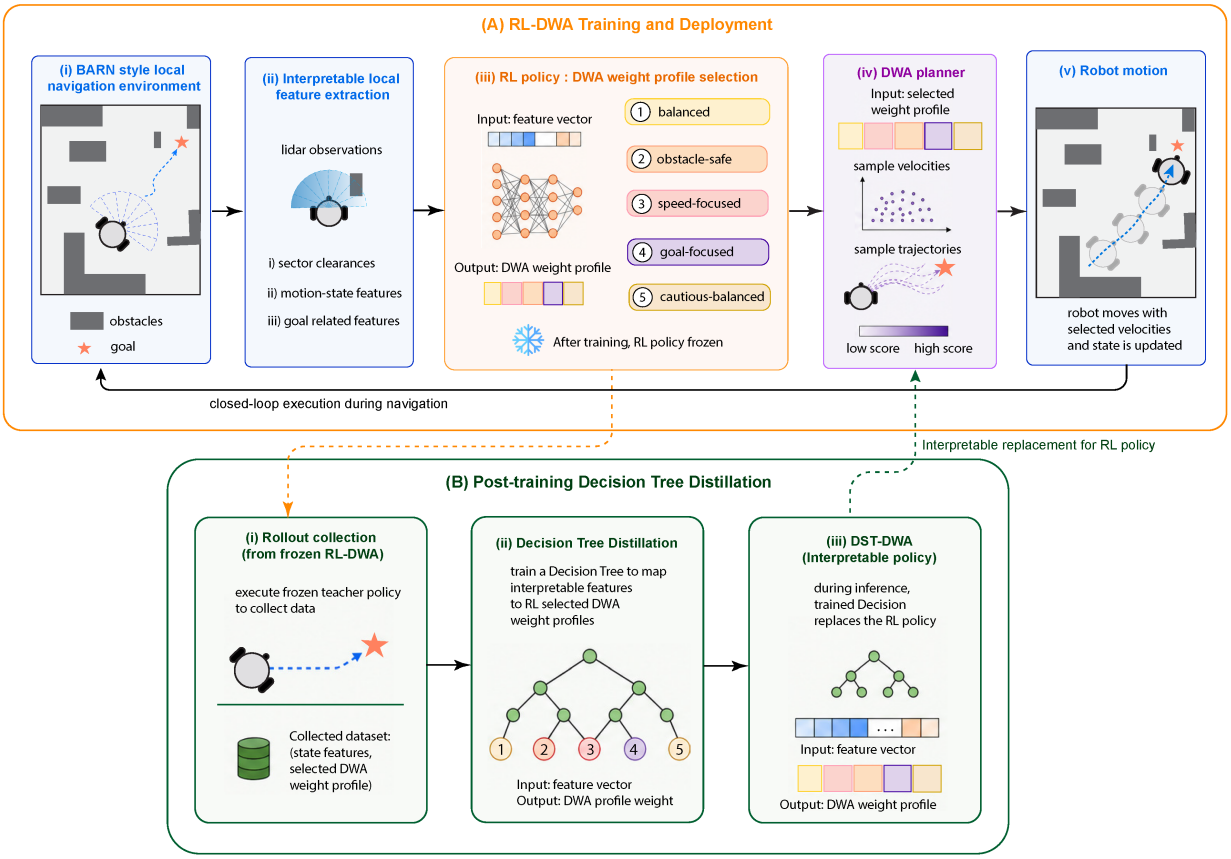


FIGURE 5.1. Overall proposed pipeline for interpretable RL-based DWA navigation. Local LiDAR-like observations, goal information, and robot motion states are converted into compact interpretable features. During RL-DWA execution, a PPO policy selects a discrete DWA weight profile, and the DWA planner computes the final feasible velocity command. After training, rollouts from the frozen RL-DWA policy are used to train a shallow decision tree that maps the same interpretable features to the RL-selected DWA profiles. The resulting DST-DWA policy replaces the neural policy during evaluation, enabling rule-based interpretation while retaining adaptive DWA weight selection.

For each environment, let L_i^* denote the reference path length for environment i. The optimal traversal time is defined as

$$OT_i = \frac{L_i^*}{v_{\max}}, \tag{69}$$

where $v_{\max}$ is the maximum linear speed of the robot. Let AT_i denote the actual traversal time taken by the robot in environment i. The BARN score is computed as

$$s_i = \mathbb{I}_i^{\text{success}} \frac{OT_i}{\text{clip}(AT_i, 2OT_i, 8OT_i)}, \tag{70}$$

where $\mathbb{I}_i^{\text{success}}$ is equal to 1 if the robot reaches the goal without collision and 0 otherwise. Thus, failed trials due to collision, timeout, or other terminal failure conditions receive a score of zero.

The primary evaluation metrics include success rate, BARN score, path length, time to goal, and failure statistics. Path length and time to goal are computed over successful episodes only, since early collisions can otherwise misleadingly reduce these values.

5.2.5. Implementation Details. All planners are implemented in the same custom Python-based 2D simulation environment, and Baseline DWA, RL-DWA, and DST-DWA all replan at the same 0.1 s DWA control step. Baseline DWA holds one fixed weight profile for the whole episode, so it has no profile-decision layer to re-evaluate. RL-DWA and DST-DWA add that layer on top of the same underlying DWA loop. The weight-profile decision is re-evaluated every 5 control steps (0.5 s) rather than every step, since holding a profile for multiple control steps reduces noisy weight-switching and makes PPO training more stable; DST-DWA keeps the same hold so its closed-loop behavior matches the teacher policy it was distilled from. Beyond this, the three methods use identical start-goal pairs, environment splits, sensing range, collision-checking procedure, and episode termination rules. An episode terminates when the robot reaches the goal, collides with an obstacle, or exceeds the maximum time limit. Therefore, the only difference among the compared methods is how the DWA weight profile is selected during navigation.

5.3. Interpretable RL-DWA Framework

This section presents the proposed interpretable RL-DWA framework. The method extends the classical DWA planner by replacing its fixed weight vector with an adaptive weight-profile selection mechanism. Instead of directly outputting linear and angular velocity commands, the learning policy selects one profile from a small discrete set of predefined DWA weight profiles. The selected profile is then used by DWA to evaluate candidate trajectories and compute the final dynamically feasible control command. This design preserves the structured motion-generation process of DWA while allowing the navigation behavior to adapt to the local scene. For example, the robot may select a speed-focused profile in open regions and an obstacle-safe or cautious-balanced profile in cluttered or narrow regions. Since the final command is still generated by DWA, the learned policy operates at the behavioral level rather than directly controlling low-level robot motion. To support interpretability, the policy input is constructed from compact local navigation features instead of raw LiDAR scans. These features summarize obstacle clearance, clearance imbalance, heading error, distance to goal, and current robot velocity. After training, the frozen RL policy is distilled into a shallow decision tree using rollout state-action data. The resulting DST-DWA policy provides an interpretable replacement for the RL policy, where each decision can be traced through explicit feature-threshold rules. The overall framework (see Figure 5.1) therefore consists of two stages: first, an RL policy is used to learn adaptive DWA weight-profile selection for closed-loop navigation; second, the trained policy is distilled into a decision tree that maps the same interpretable features to the RL-selected DWA profiles. The following subsections describe the adaptive DWA profile set, the RL formulation, the training protocol, and the decision-tree distillation procedure.

5.3.1. Adaptive DWA Weight-Profile Selection. In the proposed framework, the DWA objective remains unchanged, but the weight vector $\boldsymbol{\theta}$ is selected adaptively at each control step. Instead of using a single fixed weight vector, the planner chooses from a discrete

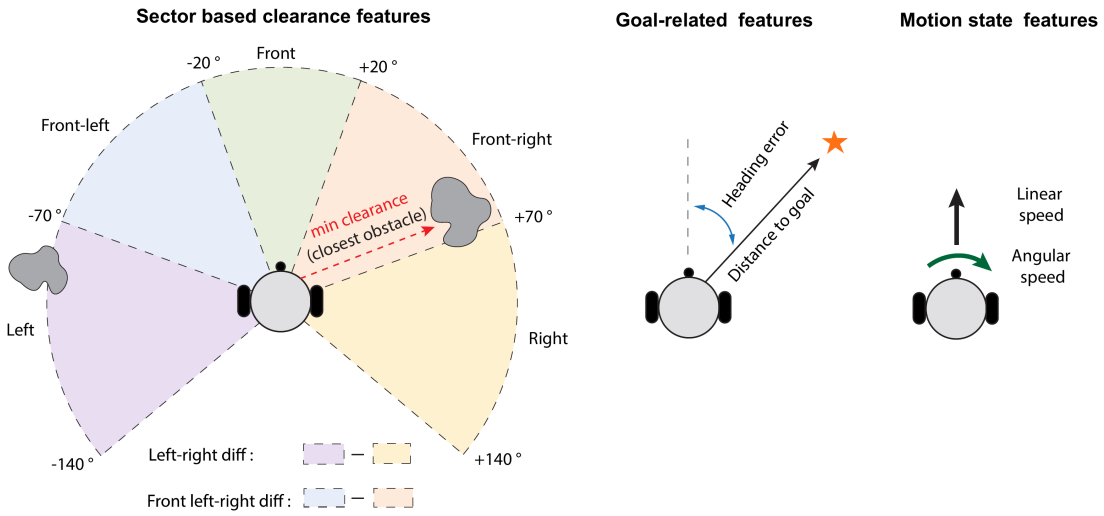


FIGURE 5.2. Interpretable local feature representation used for RL-DWA policy learning and decision-tree distillation. The robot-centric LiDAR observations are summarized into sector-based clearance features in the front, front-left, front-right, left, and right regions, together with the minimum observed clearance. Additional compact features describe left–right and front-left–front-right clearance imbalance, goal-relative quantities such as distance-to-goal and heading error, and motion-state quantities such as linear and angular velocity. These human-readable local features form the state input for both the PPO-based DWA weight-profile selector and the distilled decision-tree policy.

set of K predefined DWA weight profiles:

$$\Theta = \{\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots, \boldsymbol{\theta}_K\}, \tag{71}$$

where each profile is defined as

$$\boldsymbol{\theta}_k = [w_g^{(k)}, w_c^{(k)}, w_v^{(k)}], \quad k = 1, 2, \ldots, K. \tag{72}$$

We manually define five DWA weight profiles to represent different local navigation behaviors. Each profile assigns different relative importance to goal progress, obstacle clearance, and forward speed:

- **Balanced navigation:** balances goal progress, obstacle clearance, and forward speed.

- **Obstacle-safe navigation:** prioritizes obstacle clearance in cluttered or narrow regions.
- **Speed-focused navigation:** encourages faster forward motion when sufficient free space is available.
- **Goal-focused navigation:** emphasizes reducing heading error toward the goal.
- **Cautious-balanced navigation:** uses a conservative balance of goal progress, clearance, and speed.

At each time step, the high-level policy selects an action

$$a_t \in \{1, 2, \ldots, K\}, \tag{73}$$

which determines the active DWA weight profile $\boldsymbol{\theta}_{a_t}$. The DWA planner then computes the final command using

$$\mathbf{u}_t^* = \arg\max_{\mathbf{u}_t \in V_d} J(\mathbf{u}_t; \boldsymbol{\theta}_{a_t}). \tag{74}$$

Thus, the learning problem is reduced to selecting the most appropriate DWA behavior mode for the current local navigation state. This decouples high-level behavior adaptation from low-level velocity generation and makes the learned policy easier to distill into interpretable rules.

5.3.2. RL Formulation with Interpretable Local Features. The adaptive DWA weight selection problem is formulated as a Markov Decision Process (MDP) **[91, 53]**, defined by the tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle, \tag{75}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ is the transition dynamics, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor.

Action Space: The action space is discrete and corresponds to the DWA profile set, $\mathcal{A} = \{1, 2, \ldots, K\}$. The stochastic policy is represented as $\pi_\phi(a_t \mid s_t)$, where ϕ denotes the policy parameters.

State Space (Local Feature Representation): The state $s_t \in \mathcal{S}$ is represented using compact and human-readable local navigation features. Instead of using raw LiDAR scans, the LiDAR-like range observations are summarized into sector-based clearance features, goal-relative features, and robot motion features. This representation is used both for RL policy training and for later decision-tree distillation.

The state vector is defined as

$$\begin{aligned} s_t = \Big[& f_{\text{front}}, f_{\text{front-left}}, f_{\text{front-right}}, \\ & f_{\text{left}}, f_{\text{right}}, f_{\min}, d_{\text{LR}}, d_{\text{FLR}}, \\ & e_\theta, |e_\theta|, d_g, v_t, \omega_t \Big]. \end{aligned} \tag{76}$$

The components of the state vector are defined as follows:

- $f_{(\cdot)}$ denotes the obstacle clearance in a local sensing sector. It is computed as the minimum valid LiDAR-like range measurement in that sector. The sectors include front, front-left, front-right, left, and right. If no obstacle is detected in a sector, the clearance value is set to the maximum sensing range $r_{\max}$. The term $f_{\min}$ denotes the minimum clearance over all sectors.
- d_{LR} and d_{FLR} denote clearance imbalance around the robot. Specifically, d_{LR} is the difference between left and right clearance, while d_{FLR} is the difference between front-left and front-right clearance.
- e_θ denotes the heading error between the robot's current heading direction and the direction to the local goal, while $|e_\theta|$ denotes its absolute value.
- d_g denotes the Euclidean distance from the robot to the local goal.
- v_t and ω_t denote the robot's current linear and angular velocities, respectively.

The local feature representation is illustrated in Figure 5.2. All obstacle-related features are computed exclusively from local LiDAR-like observations. The state representation deliberately excludes global obstacle maps or privileged simulator coordinates, ensuring the policy relies purely on realistic, locally available sensor data.

Reward Function: The reward function is designed to encourage goal-reaching, forward progress, safe obstacle clearance, and efficient navigation. At each internal control step, the reward is computed from dense progress feedback, a small time penalty, obstacle-proximity penalties, and terminal rewards or penalties.

Let D_t denote the current distance to the goal, D_{t-1} denote the previous distance to the goal, and D_0 denote the initial distance to the goal. The progress reward is computed as

$$r_{\text{progress}} = 10.0 \frac{D_{t-1} - D_t}{D_0}. \tag{77}$$

This term gives a positive reward when the robot moves closer to the goal and a negative reward when it moves away. If the robot increases its distance from the goal, an additional small penalty is applied:

$$r_{\text{away}} = \begin{cases} -0.05, & \frac{D_{t-1}-D_t}{D_0} < -10^{-4}, \\ 0, & \text{otherwise.} \end{cases} \tag{78}$$

To encourage efficient motion, a constant step penalty is applied at every non-terminal running step:

$$r_{\text{time}} = -0.05. \tag{79}$$

Obstacle safety is encouraged using clearance-based penalties. Let c_t denote the true physical clearance between the robot body and the nearest obstacle boundary at time step t. This clearance is computed by subtracting both the obstacle radius and the robot safety radius from the center-to-center distance:

$$c_t = \min_i \left(\|p_t - o_i\| - r_i - R \right), \tag{80}$$

where p_t is the robot position, o_i is the center of obstacle i, r_i is the obstacle radius, and R is the robot radius including the safety buffer.

If the clearance falls below 0.30 m, the robot receives a smooth obstacle-proximity penalty:

$$r_{\text{clearance}} = \begin{cases} -\frac{0.02}{\max(c_t, 0.03)}, & c_t < 0.30, \\ 0, & c_t \geq 0.30. \end{cases} \tag{81}$$

In addition, a stronger near-collision penalty is applied when the clearance is below 0.10 m:

$$r_{\text{near}} = \begin{cases} -\min\left(3.0(0.10 - c_t), 2.0\right), & c_t < 0.10, \\ 0, & c_t \geq 0.10. \end{cases} \tag{82}$$

When the robot reaches the goal, the reward function returns a terminal success reward. Let η denote the remaining episode-budget fraction:

$$\eta = \frac{N - t}{N}, \tag{83}$$

where N is the maximum number of episode steps and t is the number of steps taken when the goal is reached. The terminal goal reward is

$$r_{\text{goal}} = \begin{cases} 50.0 + 10.0\eta, & \eta > 0.30, \\ 50.0, & \eta \leq 0.30. \end{cases} \tag{84}$$

Terminal failure conditions are penalized as follows. A collision receives a fixed penalty of

$$r_{\text{collision}} = -50.0. \tag{85}$$

If no feasible DWA trajectory is found, the penalty is

$$r_{\text{infeasible}} = -15.0 \left(0.5 + 0.5 \operatorname{clip}\left(\frac{D_t}{D_0}, 0, 1\right)\right). \tag{86}$$

If the episode ends due to timeout, the penalty is

$$r_{\text{timeout}} = -10.0 \left(0.5 + 0.5 \operatorname{clip}\left(\frac{D_t}{D_0}, 0, 1\right)\right). \tag{87}$$

For a non-terminal running step, the reward combines the dense terms:

$$r_t = r_{\text{progress}} + r_{\text{away}} + r_{\text{time}} + r_{\text{clearance}} + r_{\text{near}}. \tag{88}$$

For terminal failure cases such as collision, infeasible planning, or timeout, the corresponding terminal penalty is added to the dense reward terms:

$$r_t = r_{\text{progress}} + r_{\text{away}} + r_{\text{time}} + r_{\text{clearance}} + r_{\text{near}} + r_{\text{terminal}}, \tag{89}$$

where

$$r_{\text{terminal}} \in \left\{ r_{\text{collision}}, r_{\text{infeasible}}, r_{\text{timeout}} \right\}. \tag{90}$$

For successful goal reaching, the reward function directly returns r_{goal}.

The policy is optimized to maximize the expected discounted return using PPO [**86**]. In this formulation, PPO learns to select DWA weight profiles that make consistent progress toward the goal while maintaining safe obstacle clearance and avoiding terminal failures.

5.3.3. RL Training Protocol. The RL policy is trained using PPO implemented in Stable-Baselines3 [**80**]. All training is conducted in a custom Python-based 2D simulation environment using the first 100 BARN environments. To evaluate generalization, these environments are randomly shuffled and divided into three mutually exclusive sets:

- **Training set (60 environments):** used for PPO policy learning and simulator interaction.
- **Validation set (15 environments):** used to monitor generalization and select the best policy checkpoint.
- **Test set (25 environments):** held out from training and checkpoint selection, and used only for final comparison.

During each episode, the differential-drive robot starts from an initial pose and navigates toward a target goal. The episode terminates when the robot reaches the goal, collides with

an obstacle, or exceeds the time limit. Figure 5.3 shows the training convergence in terms of success rate on the BARN environments.

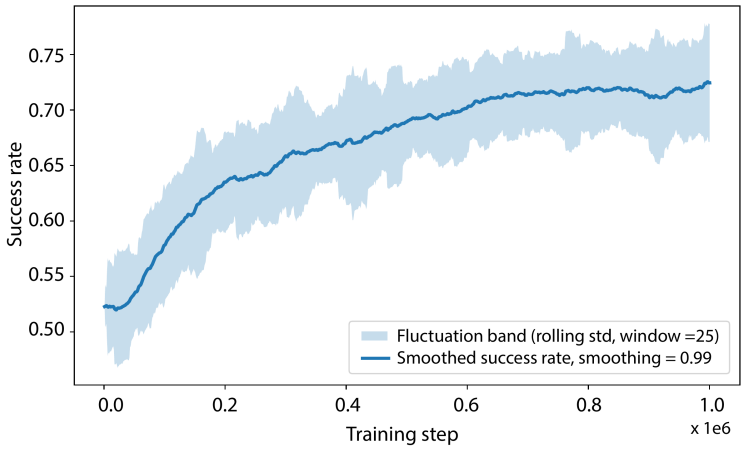


FIGURE 5.3. Training convergence curve showing success-rate progression across PPO training iterations.

5.3.4. Decision-Tree Policy Distillation. A decision tree is a supervised learning model that represents decisions as a sequence of feature-based threshold tests [**79, 17**]. In a classification tree, each internal node applies a condition to one input feature, each branch corresponds to the outcome of that condition, and each leaf node assigns a class label. This structure provides a directly inspectable decision path, since each prediction can be traced through a small set of explicit rules [**69**].

In this chapter, the decision tree is used to distill the trained RL policy into a simpler student policy. Policy distillation generally transfers the behavior of a trained teacher policy to a separate student model [**84**]. After PPO training, the learned RL policy is frozen and treated as a teacher model. The objective is to train a shallow decision tree that imitates the teacher's DWA weight-profile selection behavior while remaining easy to inspect.

Rollout data are collected by executing the frozen RL policy in the training environments. At each time step, the local feature vector and the DWA weight profile selected by the RL policy are recorded. This generates a supervised learning dataset:

$$\mathcal{D}_{\text{distill}} = \left\{ \left(s_i, a_i^{\text{RL}} \right) \right\}_{i=1}^{N}, \tag{91}$$

where s_i is the interpretable local feature vector, a_i^{RL} is the DWA weight profile selected by the trained RL policy, and N is the total number of collected samples.

A decision tree classifier is then trained to map the local feature vector to the RL-selected action:

$$\hat{a}_i = T(s_i), \tag{92}$$

where T denotes the decision tree and $\hat{a}_i$ is the predicted DWA weight profile. The tree is optimized to minimize the classification error between its predictions and the teacher's actions:

$$T^* = \arg\min_{T} \sum_{i=1}^{N} \mathbb{I}\left[T(s_i) \neq a_i^{\mathrm{RL}}\right], \tag{93}$$

where $\mathbb{I}[\cdot]$ is the indicator function.

The goal of policy distillation is to balance imitation accuracy and interpretability. To keep the learned rules readable, the maximum tree depth is restricted to four. With this constraint, each internal node represents a threshold condition on a physically meaningful navigation feature, such as obstacle clearance, heading error, or current velocity, and each leaf node assigns a DWA weight profile.

During evaluation, this distilled decision-tree policy (DST-DWA) replaces the neural PPO policy. At each control step, the tree evaluates the local feature vector s_t, selects a DWA weight profile, and passes it to the DWA planner, which computes the final velocity command.

The decision tree is evaluated in two ways. First, imitation accuracy measures how faithfully the tree reproduces the trained RL policy:

$$\mathrm{Acc}_{\mathrm{imit}} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\left[T(s_i) = a_i^{\mathrm{RL}}\right]. \tag{94}$$

Second, the distilled policy is evaluated in closed-loop navigation, since high imitation accuracy alone does not guarantee safe or efficient navigation performance.

5.4. Evaluation and Explainability

5.4.1. Evaluation Setup. The final evaluation is performed on the held-out test environments, which are not used during RL training, checkpoint selection, decision-tree training, or tree-depth tuning. Three navigation policies are compared:

- **Baseline DWA:** classical DWA with a single hand-tuned weight profile (the balanced profile).
- **RL-DWA:** DWA with adaptive weight-profile selection using the trained PPO policy.
- **DST-DWA:** DWA with adaptive weight-profile selection using the distilled decision tree.

All methods are evaluated using the same test environments, start-goal pairs, and termination conditions. The main metrics are success rate, BARN score, path length, and time to goal. Path length and time to goal are averaged only over successful episodes, since failed trials may terminate early and appear artificially short or fast.

5.4.2. Imitation Fidelity of DST-DWA. The decision-tree policy is first evaluated based on how faithfully it imitates the trained RL-DWA teacher policy. The distilled tree achieved a validation fidelity of 90.52%, indicating that a shallow interpretable model can reproduce most of the PPO policy's DWA profile-selection behavior.

Although five DWA profiles were defined, the trained RL policy actively used only three profiles during rollouts: balanced, obstacle-safe, and speed-focused. The speed-focused class was the dominant mode and was predicted with high accuracy. Most prediction errors occurred between the balanced and obstacle-safe classes, which suggests that the main ambiguity in the distilled policy occurs between similar conservative behaviors.

This result shows that the decision tree captures the dominant action-selection behavior of the RL policy while providing an interpretable representation of the learned profile-selection strategy. Unlike the PPO neural network, the decision tree allows each action to be explained by tracing a path from the root node to the selected leaf. Each internal node corresponds

to a threshold condition on a physically meaningful local navigation feature, and each leaf node corresponds to a selected DWA weight profile.

TABLE 5.1. Quantitative navigation performance across validation and test environments. Path length and time to goal are averaged over successful episodes only.

Split	Method	Total Envs	Success Rate	Mean BARN Score	Path Length (m) *(Success Only)*	Time to Goal (s) *(Success Only)*
Validation	Baseline DWA	15	40.0%	0.200	4.49	11.77
	RL-DWA	15	46.7%	0.233	4.79	13.34
	DST-DWA	15	46.7%	0.233	4.75	12.73
Test	Baseline DWA	25	40.0%	0.200	4.73	12.34
	RL-DWA	25	60.0%	0.300	4.85	12.37
	DST-DWA	25	**68.0%**	**0.336**	5.18	13.34

TABLE 5.2. Generalization performance on harder held-out test environments. Path length and time to goal are averaged over successful episodes only.

Method	Total Envs	Success Rate	Mean BARN Score	Path Length (m)	Time to Goal (s)
Baseline DWA	50	20.0%	0.10	4.62	11.80
RL-DWA	50	32.0%	0.16	4.79	12.34
DST-DWA	50	28.0%	0.14	4.66	11.89

5.4.3. Navigation Performance on Held-Out Environments. Table 5.1 summarizes the quantitative performance across the validation and held-out test environments. On the 25 unseen test environments, Baseline DWA achieved a success rate of 40.0%, while RL-DWA improved the success rate to 60.0%. DST-DWA achieved the highest test success rate of 68.0% and the highest mean BARN score of 0.336.

These results indicate that adaptive DWA weight-profile selection improves closed-loop navigation performance compared with a fixed hand-tuned DWA profile. The improvement is expected because a fixed DWA profile uses the same objective weighting in all situations, whereas the adaptive methods can change the planner behavior depending on the local navigation condition. For example, the robot can emphasize speed in relatively open regions and obstacle clearance in cluttered or narrow regions.

DST-DWA performs comparably to the neural RL-DWA policy and slightly exceeds it on this held-out test split. However, this difference corresponds to only two additional

successful trials out of 25 test environments. Therefore, we do not claim a statistically significant advantage of DST-DWA over RL-DWA. Instead, the result suggests that the distilled tree preserves much of the closed-loop behavior of the RL policy while providing a more interpretable decision structure.

5.4.4. Generalization to Harder Environments. To further evaluate generalization beyond the easier BARN environments used during training, the learned navigation policies were tested on a separate set of 50 comparatively harder BARN environments. This evaluation is intentionally more challenging than the training setting and is used to assess whether the learned DWA weight-selection behavior transfers to more difficult obstacle configurations.

As shown in Table 5.2, Baseline DWA achieved a success rate of 20.0% on the harder test set. In comparison, RL-DWA improved the success rate to 32.0%, corresponding to 16 successful trials out of 50. The DST-DWA policy achieved a success rate of 28.0%, corresponding to 14 successful trials out of 50. Thus, both adaptive DWA variants outperformed the fixed-weight baseline, suggesting that learned weight-profile selection improves robustness in challenging unseen environments.

RL-DWA obtained the highest mean BARN score of 0.16, followed closely by DST-DWA with 0.14, while Baseline DWA achieved 0.10. Although DST-DWA performed slightly below its neural teacher in this harder generalization setting, the difference corresponds to only two additional successful environments for RL-DWA. Therefore, we do not claim a statistically significant performance gap between RL-DWA and DST-DWA.

The success-only path length and traversal time values are similar across the three methods. This indicates that the primary performance difference comes from whether the robot successfully reaches the goal rather than from large differences in trajectory efficiency after success. Overall, these results show that adaptive DWA weight tuning improves generalization over the fixed baseline, and that DST-DWA offers an interpretable approximation of RL-DWA with only a modest reduction in success rate on harder test environments.

5.4.5. Trajectory-Level and Rule-Based Explainability. Beyond aggregate success rates, the proposed framework provides insight into how adaptive DWA weight-profile

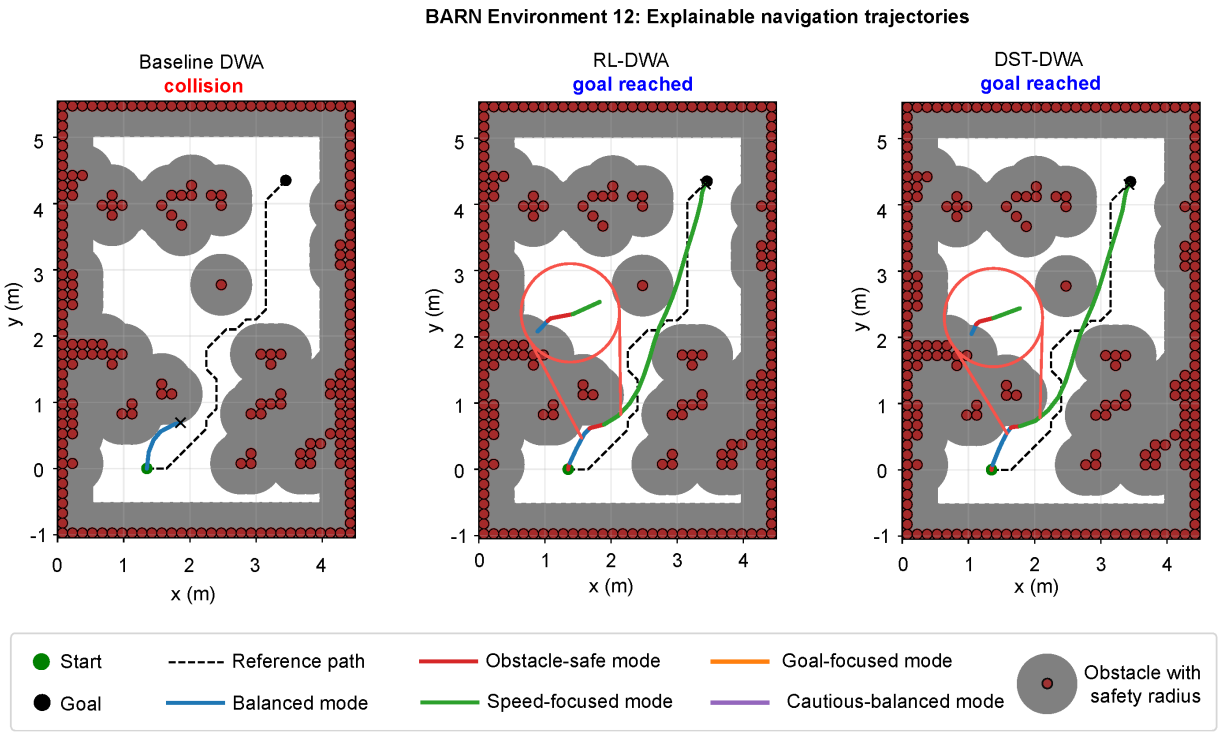


FIGURE 5.4. Action-colored trajectory comparison in BARN Environment 12. The three panels show the closed-loop behavior of Baseline DWA, RL-DWA, and DST-DWA under the same start-goal configuration. Trajectory segments are colored according to the DWA weight profile selected at each control step: balanced, obstacle-safe, speed-focused, goal-focused, and cautious-balanced. Baseline DWA uses a constant balanced profile throughout execution, whereas RL-DWA and DST-DWA adaptively switch profiles based on the local navigation state. In this example, the fixed-weight planner collides, while both adaptive methods reach the goal. Each panel also prints a bold status label above it ("collision" or "goal reached"); that label's color may not meet minimum text-contrast requirements on its own, so the outcome is restated here in this caption.

selection changes the robot's closed-loop behavior. Figure 5.4 shows action-colored trajectories for Baseline DWA, RL-DWA, and DST-DWA in the same BARN environment. Each trajectory segment is colored according to the DWA profile selected at that control step.

Baseline DWA uses the same balanced profile throughout execution. Because its objective weighting does not change with the local scene, the planner is unable to sufficiently adapt when the robot enters a constrained region. As a result, the fixed-weight planner follows a trajectory that eventually collides with obstacles. This illustrates a key limitation of using a single hand-tuned DWA weight vector across all navigation conditions.

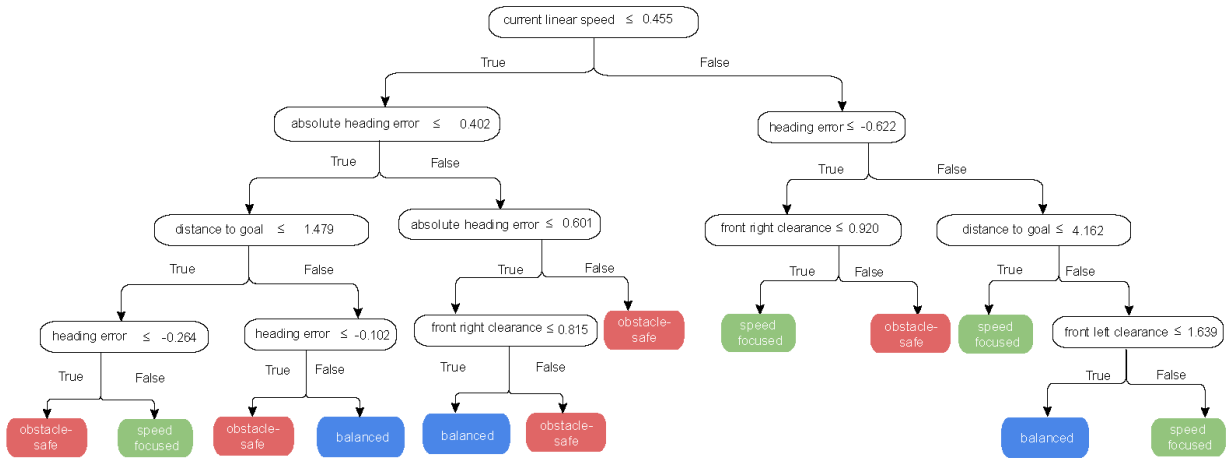


FIGURE 5.5. Example decision tree for distilled DWA profile selection. Internal nodes represent threshold conditions on interpretable local features such as current speed, heading error, distance to goal, and local obstacle clearance. Leaf nodes correspond to selected DWA weight profiles, allowing each navigation mode decision to be traced through explicit feature-threshold rules.

In contrast, RL-DWA adaptively switches the DWA weight profile during navigation. In the shown example, the policy selects an obstacle-safe profile near cluttered regions, which increases the importance of obstacle clearance during DWA trajectory scoring. This encourages DWA to select candidate velocity commands that maintain safer clearance from nearby obstacles, enabling the robot to avoid the failure encountered by Baseline DWA. In more open portions of the trajectory, the policy can select a speed-focused profile to encourage forward progress. The learned policy modulates the DWA objective weights to make the planner more context-sensitive. DWA still generates the final velocity command.

DST-DWA produces a similar successful trajectory while replacing the neural PPO policy with a shallow decision tree. This indicates that the distilled tree preserves the most important mode-selection behavior of the RL policy. The action-colored trajectory therefore provides a trajectory-level explanation of when and where the adaptive planners select different DWA profiles and how these selections affect the resulting path.

Figure 5.5 shows an example of the distilled decision tree used for DWA profile selection. The tree provides rule-based interpretability by expressing the policy as a sequence

of threshold conditions over local features. For any state, the selected DWA profile can be explained by following the active path from the root node to a leaf node.

The root node first separates behavior based on the current linear speed. This split distinguishes low-speed maneuvering or recovery-like states from higher-speed traversal states. Subsequent nodes use heading error, absolute heading error, distance to goal, and local clearance features to refine the decision. For example, splits based on front-right or front-left clearance indicate that the tree uses local obstacle proximity when deciding whether to select the obstacle-safe profile. Similarly, heading-error and distance-to-goal splits indicate whether the robot should prioritize alignment, cautious maneuvering, or forward progress.

Each leaf node corresponds to a selected DWA weight profile, such as obstacle-safe, speed-focused, or balanced navigation. The tree makes the profile-selection process transparent, allowing one to identify which local conditions caused each mode to be selected. This is useful for debugging, safety analysis, and understanding how the adaptive planner responds to different obstacle configurations.

Because the tree in Figure 5.5 distinguishes its three leaf profiles by fill color alone, and the white leaf-node text does not meet minimum contrast against all three fill colors, the complete tree is also given below as one full rule per leaf, numbered left to right as drawn:

(1) Obstacle-safe: current linear speed ≤ 0.455, absolute heading error ≤ 0.402, distance to goal ≤ 1.479, heading error ≤ -0.264.

(2) Speed-focused: current linear speed ≤ 0.455, absolute heading error ≤ 0.402, distance to goal ≤ 1.479, heading error > -0.264.

(3) Obstacle-safe: current linear speed ≤ 0.455, absolute heading error ≤ 0.402, distance to goal > 1.479, heading error ≤ -0.102.

(4) Balanced: current linear speed ≤ 0.455, absolute heading error ≤ 0.402, distance to goal > 1.479, heading error > -0.102.

(5) Balanced: current linear speed ≤ 0.455, absolute heading error > 0.402, absolute heading error ≤ 0.601, front-right clearance ≤ 0.815.

(6) Obstacle-safe: current linear speed ≤ 0.455, absolute heading error > 0.402, absolute heading error ≤ 0.601, front-right clearance > 0.815.

(7) Obstacle-safe: current linear speed ≤ 0.455, absolute heading error > 0.402, absolute heading error > 0.601.

(8) Speed-focused: current linear speed > 0.455, heading error ≤ -0.622, front-right clearance ≤ 0.920.

(9) Obstacle-safe: current linear speed > 0.455, heading error ≤ -0.622, front-right clearance > 0.920.

(10) Speed-focused: current linear speed > 0.455, heading error > -0.622, distance to goal ≤ 4.162.

(11) Balanced: current linear speed > 0.455, heading error > -0.622, distance to goal > 4.162, front-left clearance ≤ 1.639.

(12) Speed-focused: current linear speed > 0.455, heading error > -0.622, distance to goal > 4.162, front-left clearance > 1.639.

The decision tree and action-colored trajectory complement each other. The tree explains the profile-selection rule at the state level, while the trajectory plot shows how these decisions unfold spatially during closed-loop navigation. Together, they show that DST-DWA provides both competitive navigation performance and interpretable decision-making.

5.4.6. Failure Analysis and Efficiency Trade-offs. The path length and time to goal results in Tables 5.1 and 5.2 should be interpreted carefully because they are computed only over successful episodes. Baseline DWA often shows shorter path length and lower traversal time among successful trials. However, this does not necessarily indicate superior efficiency. Instead, it reflects a survivorship bias. Baseline DWA tends to succeed only in easier environments where the path is relatively direct, while failing early in more difficult cluttered scenes.

The adaptive methods, particularly RL-DWA and DST-DWA, are able to solve additional environments that are more constrained and require longer detours or more cautious maneuvering. As a result, their average path length and traversal time over successful trials

can be slightly higher. This increase is expected because the adaptive methods are solving harder cases rather than only the simplest ones.

Failure behavior further supports this interpretation. When Baseline DWA fails, it typically collides early because the fixed objective weighting cannot adapt to local obstacle configurations. In contrast, the adaptive policies often survive longer and progress deeper into cluttered environments before failure. This suggests that adaptive DWA weight-profile selection improves robustness by allowing the planner to alter its behavior in response to local navigation difficulty.

Adaptive DWA weight selection improved success rate over the fixed baseline. The distilled tree preserved most of that performance while providing interpretable rules for the profile choices. The trajectory plot and decision tree together let a reviewer trace specific profile selections to specific stretches of the path, making it possible to analyze navigation behavior rather than aggregate outcomes alone.

5.5. Quadruped Transfer Study in MuJoCo

The preceding sections developed and evaluated DST-DWA in a custom Python-based 2D simulator with an idealized differential-drive robot. This section tests whether the advantage observed in Section 5.4 survives realistic legged dynamics. The same fixed-weight and decision-tree planners are re-evaluated in a physics-based MuJoCo simulation [**96**] of the Unitree A1 quadruped, on a larger set of 300 BARN environments.

5.5.1. Simulation Setup. The MuJoCo evaluation platform (the A1 quadruped model, the trot-gait controller that tracks commanded linear and angular velocity, and the BARN-style environment generation) follows our prior work: the task-level control and sim-to-real trot-gait command-following framework of [**2**], and the quadruped BARN evaluation setup of Chapter 3. The local planners are unchanged from Section 5.2. DWA replans at the 0.2 s control step of that inherited quadruped framework, and the weight profile is re-selected at every control step rather than held for 5 steps as in the 2D study. The decision tree itself, its input features, and its thresholds are unchanged. At each control step the planner outputs a

velocity command (v_t, ω_t), which the gait controller executes on the legged platform. Relative to the 2D study, this changes both the profile-selection cadence and everything downstream of the velocity command: contact dynamics, imperfect velocity tracking, and body oscillation during trot.

Two planners are compared: Baseline DWA with a single hand-tuned weight profile, and DST-DWA with the distilled decision tree of Section 5.3.4 selecting a weight profile at every control step. The neural RL-DWA policy is not part of this study. Its role in the framework is to generate the supervision that the decision tree distills (Section 5.3.4). DST-DWA is the interpretable, lightweight policy intended for deployment. The high-fidelity simulation transfer study compares Baseline DWA with DST-DWA.

Each planner ran once in each of the 300 BARN environments [**7**] under identical start-goal pairs, sensing, and termination rules. An episode ends in success (goal reached), collision, or timeout (60 s budget). With one trial per environment per planner, every comparison below is paired by environment across 300 pairs. The 300 environments span the full BARN benchmark and therefore include the 100 used for training, validation, and testing in the 2D study (Section 5.3.3). This is deliberate. The study asks whether the distilled policy transfers across platforms, not whether it generalizes to unseen layouts (Section 5.4.4 already tested that), and the policy has no channel for memorizing specific environments in any case, since its input is limited to local clearance and goal features and the tree is depth-limited. As in the 2D study, the tree used only three of the five defined profiles (balanced, obstacle-safe, and speed-focused). The goal-focused and cautious-balanced profiles were never selected.

5.5.2. Environment Difficulty Analysis via PCA. Aggregate success rates over 300 heterogeneous environments can hide structure. A planner might help only in cluttered environments and do nothing in open ones. Testing that requires a difficulty measure. The obvious candidate, the fraction of occupied grid cells (density), is coarse and global. It cannot distinguish a single wide corridor from many narrow gaps at the same occupancy. The

TABLE 5.3. PCA of the five BARN path-difficulty metrics: variance explained and component loadings. Loadings are shown for PC1–PC3 only. PC4 and PC5 are omitted because together they account for 2.8% of the variance.

Quantity	PC1	PC2	PC3	PC4	PC5
Eigenvalue	3.801	0.881	0.196	0.111	0.028
Variance explained	75.8%	17.6%	3.9%	2.2%	0.6%
Cumulative	75.8%	93.3%	97.2%	99.4%	100.0%
Loadings					
avg. closest wall	+0.489	+0.027	−0.429		
avg. visibility	+0.496	+0.094	−0.164		
avg. dispersion	−0.462	−0.246	−0.834		
avg. characteristic dim.	+0.504	+0.075	−0.258		
tortuosity	−0.220	+0.961	−0.165		

BARN dataset also provides five path-local difficulty metrics computed along each environment's reference path: mean closest-wall distance, mean visibility (sight line length), obstacle dispersion, characteristic passage width, and tortuosity (arc length over chord length).

These five metrics are largely redundant. Closest-wall distance, visibility, and passage width correlate at $r = 0.87$ to 0.94 with one another, and dispersion at $r \approx -0.8$ against all three, since all four describe how open the space around the path is. Rather than stratifying on five correlated metrics or picking one arbitrarily, we apply principal component analysis (PCA) to the z-scored metrics across all 300 environments.

The first component alone captures 75.8% of the joint variance, and its loadings identify it as an openness composite: closest-wall distance, visibility, and passage width load positively and nearly equally (+0.49 to +0.50), dispersion loads negatively (−0.46), and tortuosity only weakly (−0.22). High PC1 corresponds to an open, wide environment with long sight lines. Low PC1 corresponds to a cluttered, tight one. PC2 (17.6%) is almost pure tortuosity (loading +0.96), an independent axis for how winding the path is. The size of PC1's share reflects the redundancy of the input metrics rather than anything specific to PCA. If the five metrics were independent, each component would capture nearly 20%.

PC1 is also distinct from density. It correlates with occupied fraction at only $r = -0.70$ ($r^2 = 0.48$), so density explains under half of PC1's variance. The remainder comes from

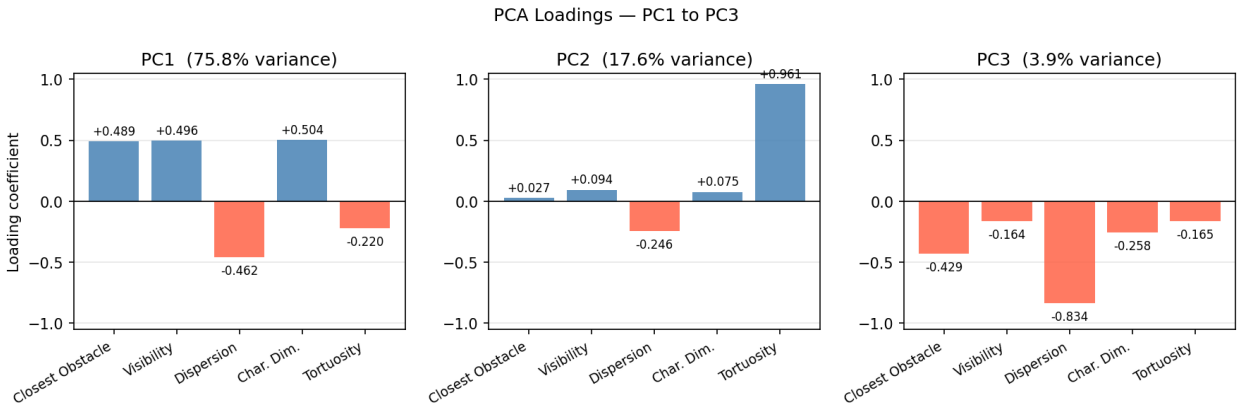


FIGURE 5.6. PC1–PC3 loadings across the five BARN difficulty metrics.

TABLE 5.4. Navigation performance over all 300 BARN environments in MuJoCo. Path length and time to goal are averaged over successful episodes only.

Method	Total Envs	Success Rate	Mean BARN Score	Path Length (m) *(Success Only)*	Time to Goal (s) *(Success Only)*
Baseline DWA	300	29.7%	0.138	5.02	19.16
DST-DWA	300	34.3%	0.153	5.16	21.24

path-local routing structure that a global occupancy statistic cannot see. PC1 therefore provides a second, materially different difficulty axis for the stratified analysis in Section 5.5.4.

We are not aware of a prior study that reduces BARN's five path-difficulty metrics to principal components. Doing so is what lets the stratified comparison in Section 5.5.4 use a single continuous difficulty axis, PC1, in place of density or an arbitrary choice among the five correlated metrics.

5.5.3. Aggregate Navigation Performance. Because both planners face identical start-goal pairs in every environment, the comparison is analyzed as 300 matched pairs rather than two independent samples, a design the smaller splits of Section 5.4 could not support statistically. Success is tested with an exact McNemar test on the discordant pairs (environments where exactly one planner succeeds). The BARN-score difference is tested with a paired sign-flip permutation test, with a bootstrap 95% confidence interval on the mean difference.

TABLE 5.5. Paired statistical comparison over all 300 environments. All differences are computed as DST-DWA − Baseline DWA.

Paired Envs	Δ Success Rate	DST Wins	Baseline Wins	McNemar p	Mean Δ BARN	Δ BARN 95% CI	Perm. p
300	+4.7 pp	31	17	0.059	+0.015	$[-0.006, 0.036]$	0.165

DST-DWA reached the goal in 103 of 300 environments against 89 for Baseline DWA (34.3% vs. 29.7%), as summarized in Tables 5.4 and 5.5. Among the 48 discordant pairs, DST-DWA won 31 and lost 17. The success-rate difference of +4.7 percentage points does not reach significance at $\alpha = 0.05$ (McNemar $p = 0.059$), and the BARN-score difference is likewise directionally positive but non-significant (mean +0.015, permutation $p = 0.165$, confidence interval straddling zero). The primary result of this study is therefore borderline. DST-DWA's advantage points in the same direction as the 2D findings of Section 5.4 but is not statistically confirmed at $n = 300$ paired trials. Throughout this section, the success-rate McNemar test is treated as the primary evidence. The BARN score's clipping (Eq. (70) bounds each score to $[0, 0.5]$ and zeroes failures) saturates many episodes at the floor or ceiling, which compresses its sensitivity, so it serves only as secondary, directional support.

Both planners' absolute success rates are well below their Section 5.4 counterparts (Baseline DWA: 29.7% here vs. 40.0% on the 2D test split). The drop appeared for both planners rather than either one alone, which is consistent with the harder setting, which combines a legged platform with imperfect velocity tracking and evaluation over the full 300-environment set including its hardest instances.

5.5.4. Difficulty-Stratified Comparison. A borderline pooled result is consistent with two hypotheses: a small uniform effect, or a larger effect confined to a subset of environments (for example, cluttered ones, where mode-switching intuitively matters most) that pooling dilutes. We test the localization hypothesis twice. Per-stratum sample sizes give each bin far less power than the pooled test, so bin-level results are exploratory.

Stratifying by obstacle density (seven bins over occupied fraction, Table 5.6) found no significant bin. The largest DST-DWA advantage appeared in bin $[0.12, 0.16)$ (+10.3 pp,

TABLE 5.6. Density-stratified paired comparison (all 300 environments).

Density Bin	Envs	Baseline Success	DST Success	Δ Success	McNemar p	Mean Δ BARN	Perm. p
$[0.08, 0.12)$	43	53.5%	60.5%	+7.0 pp	0.508	+0.033	0.323
$[0.12, 0.16)$	68	41.2%	51.5%	+10.3 pp	0.167	+0.034	0.257
$[0.16, 0.20)$	56	26.8%	28.6%	+1.8 pp	1.000	−0.010	0.687
$[0.20, 0.24)$	46	19.6%	21.7%	+2.2 pp	1.000	+0.013	0.683
$[0.24, 0.28)$	38	10.5%	13.2%	+2.6 pp	1.000	+0.007	1.000
$[0.28, 0.32)$	35	28.6%	28.6%	+0.0 pp	1.000	−0.001	1.000
$[0.32, 0.36]$	14	0.0%	7.1%	+7.1 pp	1.000	+0.036	1.000

TABLE 5.7. PC1-quartile-stratified paired comparison (all 300 environments).

PC1 Quartile	Envs	Baseline Success	DST Success	Δ Success	McNemar p	Mean Δ BARN	Perm. p
Q1 (hardest)	75	5.3%	8.0%	+2.7 pp	0.625	+0.015	0.248
Q2	75	24.0%	25.3%	+1.3 pp	1.000	−0.001	0.969
Q3	75	28.0%	34.7%	+6.7 pp	0.359	+0.020	0.417
Q4 (easiest)	75	61.3%	69.3%	+8.0 pp	0.210	+0.026	0.346

$n = 68$, McNemar $p = 0.167$), and the per-bin deltas do not move monotonically with density.

Stratifying by PC1 quartile (Q1 = most cluttered, Q4 = most open, 75 environments each, Table 5.7) also found no significant quartile. The largest advantage, still non-significant, fell in Q4, the easiest and most open quartile (+8.0 pp, $p = 0.210$). This is the opposite of the intuition that motivated the test.

Because binning a continuous variable can fabricate trends, we also tested PC1 against the paired BARN-score delta without binning. The linear fit gave Pearson $r = +0.067$ ($r^2 = 0.004$, $p = 0.247$; Spearman $\rho = +0.070$, $p = 0.226$, $n = 300$). PC1 explains 0.4% of the variance in the paired outcome, which suggests a flat relationship rather than an underpowered one. These two stratifications plus the continuous check were the only subgroup analyses performed. We find no evidence of a difficulty-dependent effect and do not claim one. The weak, non-significant tilt, if it means anything, points toward open environments, where the speed-focused profile has room to pay off.

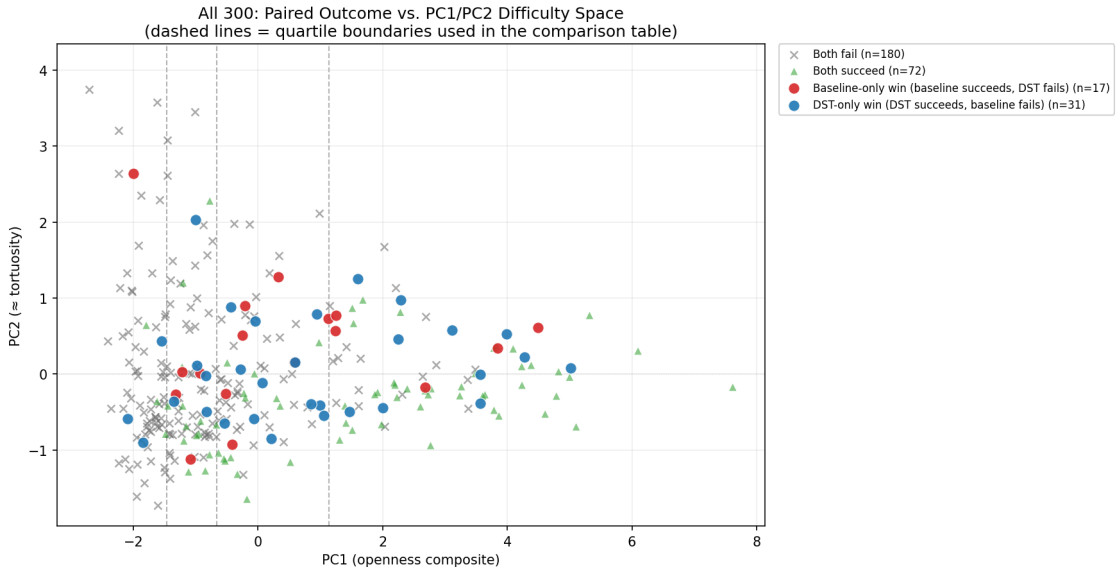


FIGURE 5.7. All 300 environments in PC1/PC2 space colored by paired outcome (both succeed, both fail, or discordant), with quartile boundaries overlaid. Discordant pairs of both colors scatter across the full PC1 range. The "both succeed" marker (light green) is low-contrast against the plot background; that category corresponds to the 72 environments, discussed below, where both planners reached the goal.

TABLE 5.8. Intensive margin: paired differences (DST-DWA − Baseline DWA) restricted to the 72 jointly successful environments.

Metric	Mean Δ	Median Δ	Δ 95% CI	Perm. p
Path Length (m)	+0.21	−0.00	$[-0.13, 0.62]$	0.266
Time to Goal (s)	+1.20	−0.40	$[-0.88, 3.49]$	0.296
BARN Score	−0.014	+0.000	$[-0.034, 0.003]$	0.137

5.5.5. Success Decomposition: Extensive and Intensive Margins. A pooled score difference conflates two mechanisms: an extensive margin (converting the other planner's failures into successes) and an intensive margin (reaching the goal more efficiently when both planners already succeed). Separating them gives the clearest pattern in this dataset.

The extensive margin accounts for all of DST-DWA's pooled gain, a net +14 environments (31 rescues against 17 losses) among the discordant pairs of Table 5.5. The intensive margin, isolated on the 72 environments where both planners reached the goal, favors Baseline DWA on every metric (Table 5.8).

TABLE 5.9. Outcome archetypes over all 300 environments.

Archetype	Baseline DWA	DST-DWA
goal	89 (29.7%)	103 (34.3%)
early collision (≤ 3 s)	12 (4.0%)	12 (4.0%)
navigating collision	1 (0.3%)	2 (0.7%)
stuck timeout	164 (54.7%)	145 (48.3%)
active timeout	34 (11.3%)	38 (12.7%)

On jointly successful runs DST-DWA drove slightly longer paths (+0.21 m), took slightly longer (+1.20 s), and scored slightly lower (−0.014). No individual difference is significant, but all three metrics agree in direction, and the same direction held in every data examined during this study. This indicates that where DST-DWA has an advantage, it comes from environments where the fixed-weight planner fails, rather than from navigating more efficiently once a route is available to both. It also refines the survivorship-bias observation of Section 5.4.6. Even on the same jointly-solved environments, DST-DWA's mode-switching spent some extra path length and time on caution, so the longer average paths in Tables 5.1 and 5.2 are not fully explained by the adaptive planners solving harder instances.

5.5.6. Failure Analysis. Why do runs fail, and what is DST-DWA doing when they do? Every failure is classified from the raw simulation logs into four archetypes: early collision (at or before 3 s, spawn-adjacent), navigating collision (after 3 s), stuck timeout (speed below 0.03 m/s for a sustained period before the 60 s limit, the classic local-minimum signature), and active timeout (still moving meaningfully when time expires). Table 5.9 reports the counts.

Three observations follow. First, timeouts dominated failure for both planners, and roughly 80% of timeouts carried the stuck signature (Baseline: 164/198, 82.8%; DST: 145/183, 79.2%). The characteristic failure on the quadruped is freezing in a local minimum rather than colliding. Second, early collisions occurred in the same 12 environments for both planners. These are spawn-geometry artifacts in which the robot starts wedged between obstacle clusters. The start pose is essentially infeasible, and both planners collide

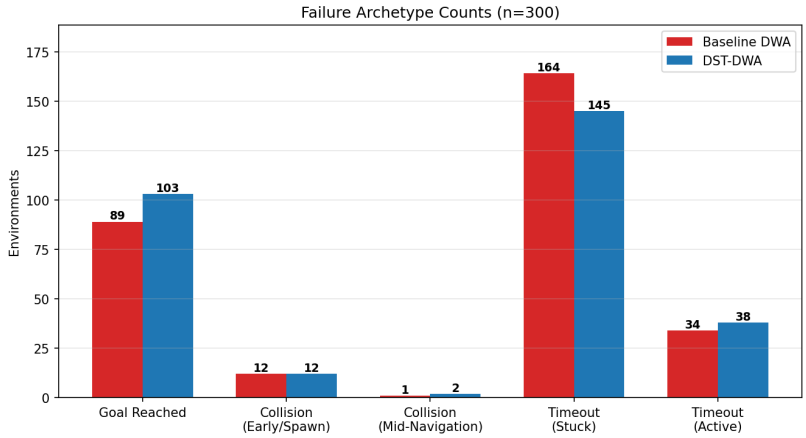


FIGURE 5.8. Failure archetype counts per planner. Exact counts are given in Table 5.9 above.

within the first seconds regardless of weight profile. Genuine mid-navigation collisions were the rarest event in the dataset (1 for Baseline DWA, 2 for DST-DWA, out of 300 each).

Third, DST-DWA's mode log ties its outcomes to specific profiles. A run's dominant mode is the profile holding the largest share of its logged control time. The 12 early-collision episodes end before the first decision-log entry is written, so mode statistics cover the remaining 288 runs. Of DST-DWA's 103 successes, 82 (80%) were dominated by the speed-focused profile. Of its 145 stuck timeouts, 79 (54%) were dominated by the obstacle-safe profile. Aggregated over the logged runs, obstacle-safe consumed 51.9% of total control time, against 30.3% for balanced and 17.8% for speed-focused. This pattern suggests the tree's failure mode is over-caution. It commits to obstacle-safe near clutter and does not build the momentum to push through a gap. The rescue geometry supports the same reading. Across the 31 environments where DST-DWA succeeded and Baseline DWA timed out, DST-DWA's trajectory passed within 0.115 m (mean) of the baseline planner's terminal failure point, indicating it usually got through the same gap with different timing and mode rather than routing around it. The reverse holds for the 17 losses. Baseline DWA's successful trajectory passed 0.350 m (mean) from where DST-DWA froze, so when DST-DWA loses, it has typically stalled off the winning route rather than at the same bottleneck. This diagnosis is possible because the tree's decisions are logged as explicit modes. The dominant failure can be traced to a specific, human-readable region of the rule set, which makes it addressable.

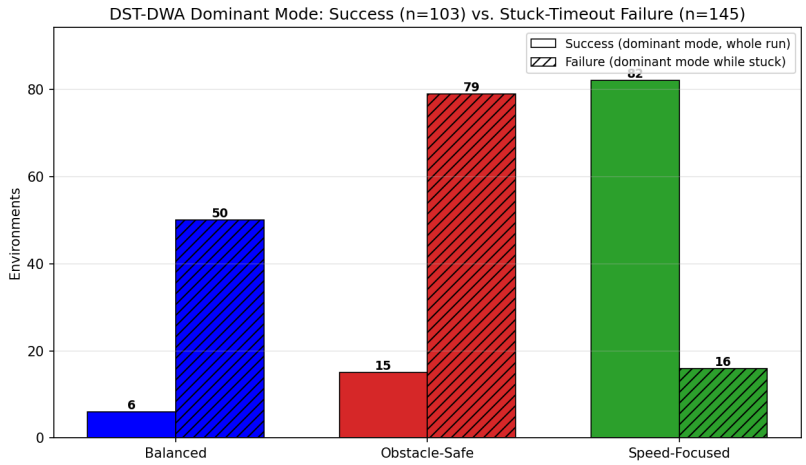


FIGURE 5.9. DST-DWA's dominant mode in successes vs. stuck timeouts.

5.5.7. Trajectory-Level Case Studies. Four environments, one per paired-outcome category, illustrate the mechanisms above (Figures 5.10–5.13). Static comparisons exist for all 300 environments. These four are drawn from a curated set of 20 selected on quantitative criteria. Each figure covers one outcome category. In the rescue case, DST-DWA clears a gap where Baseline DWA timed out (Figure 5.10). The loss case is the over-caution failure, where DST-DWA freezes near the start while the fixed weights push straight through (Figure 5.11). The joint-success case shows the efficiency cost on an environment both planners solve (Figure 5.12), and the joint-failure case a bottleneck that traps both (Figure 5.13). The per-figure captions give the quantitative detail for each case.

5.5.8. Discussion. The advantage observed in 2D shrank under quadruped dynamics. In the 2D simulator, DST-DWA beat Baseline DWA by 28 percentage points on the held-out test split and 8 points on the harder generalization set (Tables 5.1 and 5.2). On the quadruped, over 300 environments, the gap is +4.7 points and statistically borderline ($p = 0.059$). The mode-switching behavior itself does transfer: the tree still selects speed-focused in the open and obstacle-safe in clutter, and those selections still separate successes from failures (Section 5.5.6). Under trot dynamics, imperfect velocity tracking blunts the payoff, and the obstacle-safe profile's caution more often ends in freezing. The profile-reselection cadence also changed between the two studies, though, from every fifth step to every step

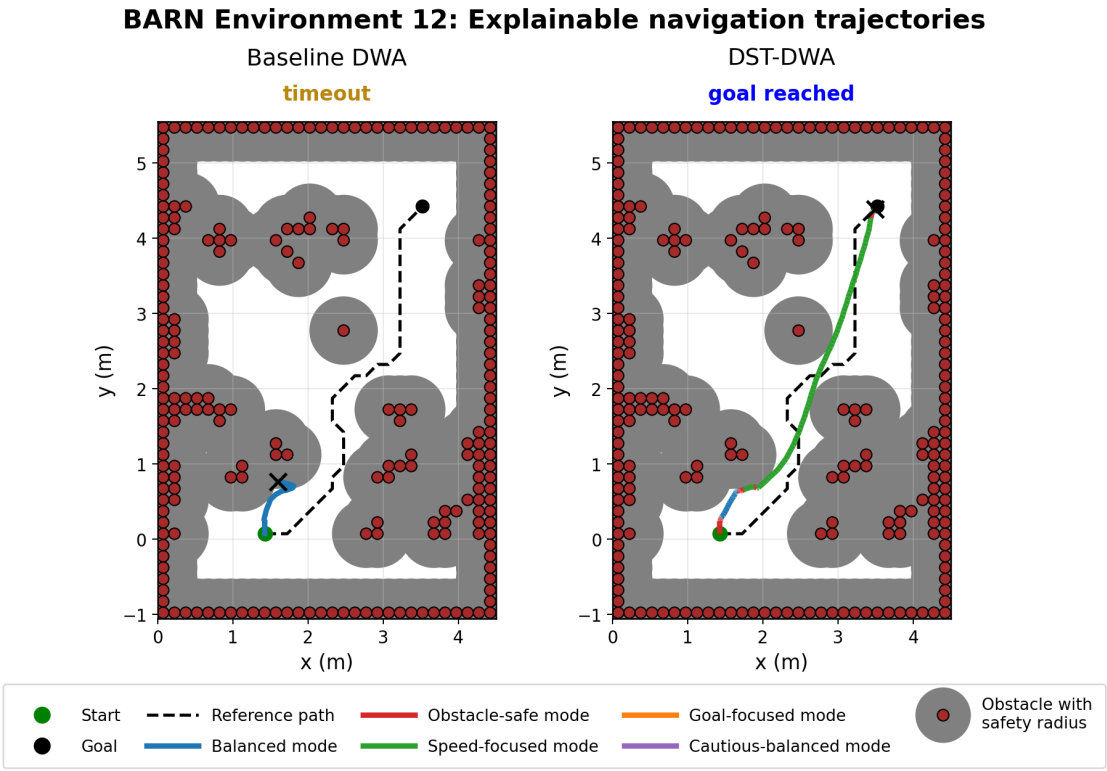


FIGURE 5.10. DST-DWA rescue (BARN Environment 12). Baseline DWA timed out stuck. DST-DWA reached the goal in 18.8 s, passing within 0.13 m of the baseline planner's failure point. Same corridor, different timing and mode. As with Figure 5.4, the bold status label above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here.

(Section 5.5.1). Because dynamics and cadence changed together, the results cannot say how much of the smaller advantage comes from trot dynamics and how much from reselecting the profile more often. Two interim subsets evaluated before the full sweep (the 25 test environments: +8.0 pp; the harder 50: +2.0 pp) disagreed on the effect size. The full-300 result falls between them, which is consistent with sampling noise around a modest common effect rather than a real subset difference.

Where DST-DWA has an advantage, it comes from rescuing failures rather than from navigating more efficiently. Its entire (unconfirmed) gain is the conversion of fixed-weight failures into successes, at a small, consistent cost in efficiency where both planners already succeed (Section 5.5.5). For a deployment-oriented planner on a legged platform this is

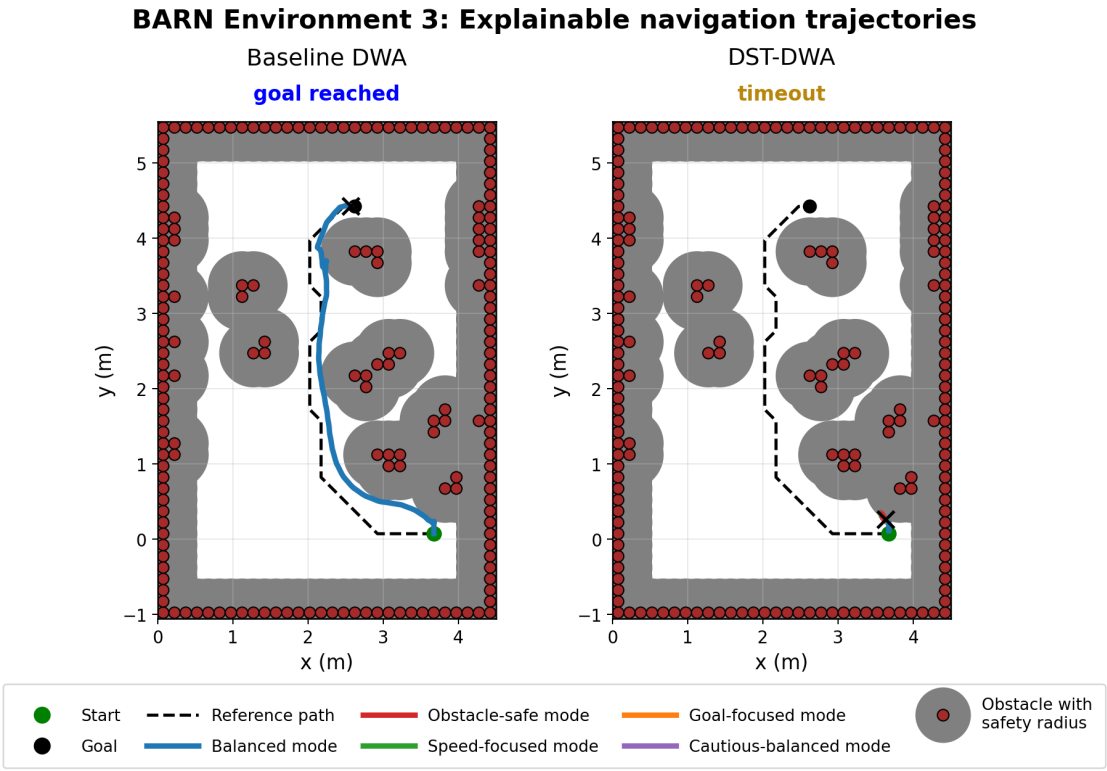


FIGURE 5.11. DST-DWA loss (BARN Environment 3). The canonical over-caution failure: DST-DWA committed to the obstacle-safe profile near the start and froze into a stuck timeout, while the fixed weights pushed straight through. As with Figure 5.4, the bold status label above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here.

arguably the right trade, since failed runs cost more than slightly slower ones, but the claim should be limited to the rescue effect.

Limitations. Each planner ran once per environment, so there is no repeated-trial variance estimate, and an effect of the observed size (about 5 pp) sits near the detection limit of a 300-pair McNemar test. This study does not have enough trials to confirm or rule out an effect of that size. The two difficulty stratifications and the continuous PC1 check were exploratory and found nothing. They are reported in full, and no subgroup effect is claimed. The BARN score's clipping limits it to directional, secondary evidence. Finally, the decision tree evaluated here was distilled from rollouts in the 2D simulator and deployed in MuJoCo unchanged, but the deployment context is not identical. The control step doubles from 0.1 s to 0.2 s and the weight profile is re-selected every control step instead of every fifth step,

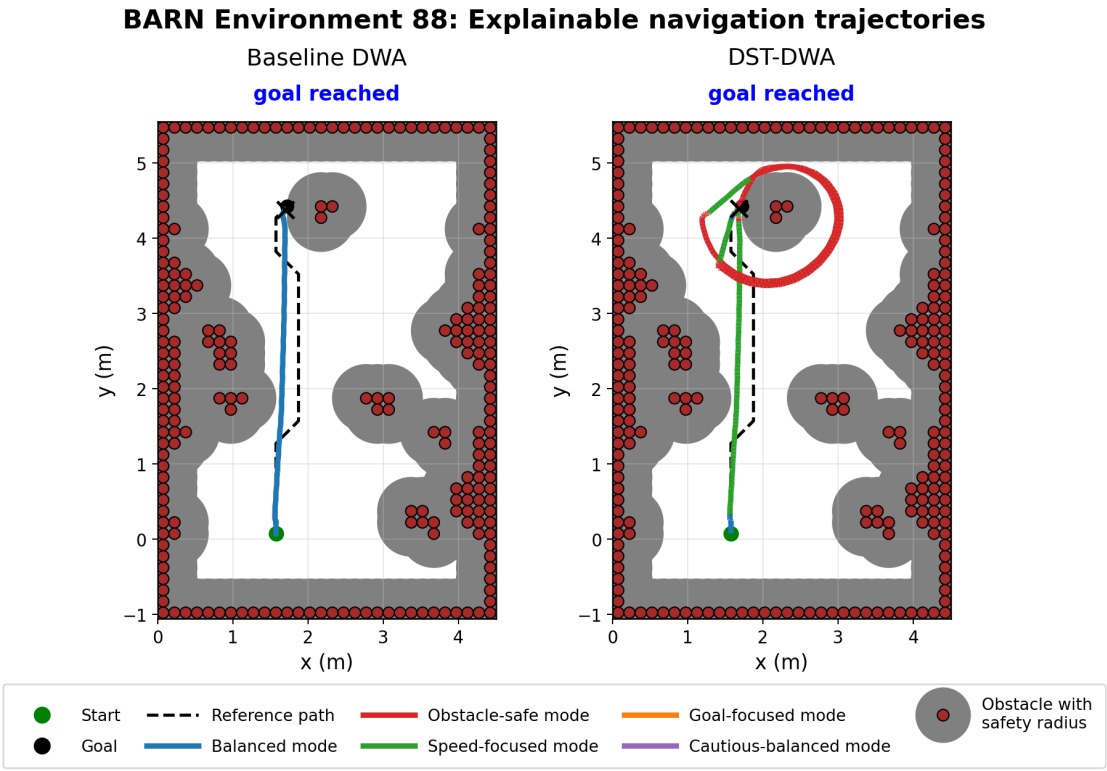


FIGURE 5.12. Joint success, efficiency tail (BARN Environment 88). Both planners reached the goal, but DST-DWA wandered: 38.8 s and 14.6 m against Baseline DWA's 13.2 s and 4.4 m. The typical joint-success case shows near-identical performance (the medians in Table 5.8 are near zero). This example illustrates the cautious tail that drives the intensive-margin means. As with Figure 5.4, the bold status label above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here.

so the tree reconsiders its mode roughly 2.5× more often here than during the rollouts it was trained to imitate. This cadence change is confounded with the legged-dynamics question the transfer study asks, and no experiment in this chapter isolates the two effects. No re-distillation on quadruped-fidelity data was performed.

5.6. Conclusion

This chapter presented an interpretable framework for adaptive local navigation. DWA weight-profile selection is formulated as an RL problem over compact, human-readable local features; a PPO policy is trained to select among discrete behavior profiles; and the frozen policy is distilled into a shallow decision tree (DST-DWA) whose decisions can be traced

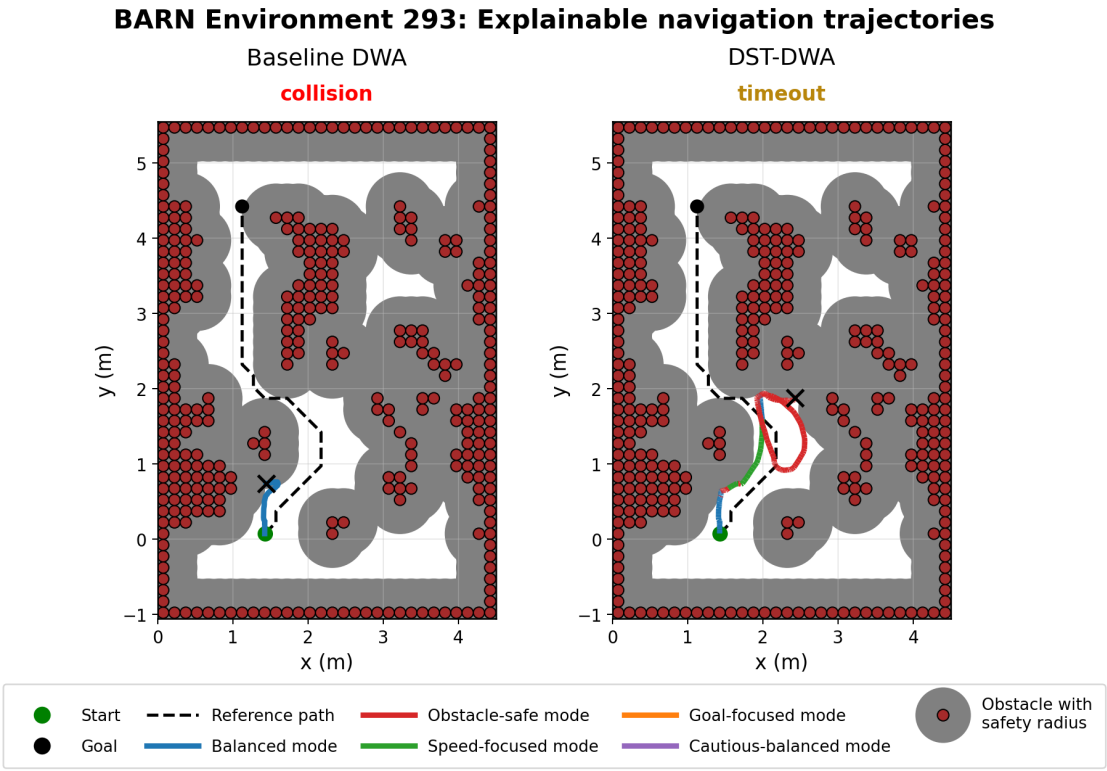


FIGURE 5.13. Joint failure, different mechanisms (BARN Environment 293). Both planners were trapped by the same bottleneck but failed differently: Baseline DWA stalled and clipped an obstacle at near-zero speed (collision), while DST-DWA circled in obstacle-safe mode until timeout. As with Figure 5.4, the bold status label above each panel may not meet minimum text-contrast requirements on its own; the outcome is restated here.

through explicit feature thresholds. The RL policy's role ends at training time, and the decision tree is the artifact intended for deployment.

In the 2D simulation study, adaptive weight selection outperformed a fixed hand-tuned profile (68.0% vs. 40.0% success on held-out test environments), and the distilled tree reproduced the neural policy's behavior (90.5% imitation fidelity) at no meaningful performance cost, while adding rule-level and trajectory-level explainability. In the MuJoCo quadruped study over 300 BARN environments, the advantage persisted in direction but did not reach statistical significance: +4.7 percentage points in success rate (McNemar $p = 0.059$). A margin decomposition indicates that the gain, where it exists, comes from rescuing fixed-weight failures rather than from efficiency on jointly solved environments. We report this as a borderline result rather than a confirmed win.

The interpretability mattered most at this weaker result. Because DST-DWA's choices are logged as explicit modes, its failure signature can be read off directly: 80% of its successes rode the speed-focused profile, 54% of its stuck timeouts were dominated by the obstacle-safe profile, and its rescues typically passed through the same gap where the fixed planner stalled. A black-box policy with the same aggregate numbers would leave the cause unclear; the tree points to a specific over-caution region in its rule set that can be retrained.

Several directions follow. Re-distilling or fine-tuning the tree on quadruped-fidelity rollouts, including activating the unused goal-focused and cautious-balanced profiles or adding switching hysteresis, targets the diagnosed obstacle-safe freezing. Repeated trials per environment and larger environment sets would give the statistical power this study lacked. The framework should also be compared against the broader set of quadruped local planners evaluated in Chapter 3. The last step of the transfer sequence, from 2D simulation through MuJoCo to hardware, is deployment on the physical Unitree A1 quadruped, where the failure diagnosis shown here matters most.

CHAPTER 6

Conclusion

This dissertation studied motion planning and control for legged robots in cluttered environments. The common problem across the chapters was the mismatch between what a simplified navigation planner can represent and what a legged robot can physically do. A planar local planner can choose body motions quickly, but it usually treats obstacles only as regions to avoid. A related mismatch is internal to the planner itself. Even a well-chosen action set is governed by objective weights that are usually fixed constants, and when learning is used to adapt those weights instead, the result is often a policy whose choices cannot be inspected. A full-body controller can represent contacts, torques, and balance constraints, but it is too expensive to use as the only decision layer for navigation. The work in this dissertation used reduced-order models between these two levels. The reduced model was not used as a substitute for the robot dynamics. It was used as a tractable planning model whose decisions had to remain connected to feasible stepping or velocity tracking. When its parameters adapted at runtime, that adaptation had to remain traceable to the local geometry that produced it.

The chapters approached this problem from four directions. Chapter 2 used a reduced-order step-to-step model for a planar biped and learned the Poincaré map with Gaussian process regression. Chapter 3 evaluated conventional local planners on a quadruped using a hierarchical controller that reduced the planning interface to a unicycle model. Chapter 4 extended local planning with leg-enhanced kinematic models so that a humanoid planner could select between going around an obstacle and stepping over it. Chapter 5 made DWA weight selection adaptive through a learned policy distilled into an interpretable decision tree, and tested how that planner transfers to a quadruped. Together, these chapters show

where reduced-order planning is useful, where it is limited, and what must be added when the robot's legged capabilities should affect the local planning decision.

6.1. Main Findings

The first finding is that reduced-order models can support step-level planning when their validity is modeled explicitly. In Chapter 2, partial feedback linearization reduced the five-link biped model before fitting a step-to-step map. The Gaussian process model provided a way to predict the next step outcome, and the Gaussian process classifier separated feasible from infeasible steps. This made the optimization problem small enough to solve over multiple steps while still using data generated from the higher-fidelity model. The resulting controller tracked walking velocity and planned foot placement for a stepping-stone task. The chapter did not solve full humanoid navigation, but it established a pattern used later in the dissertation: plan with a model that is simple enough to evaluate repeatedly, and include a feasibility boundary so the planner does not freely choose motions outside the model's useful region.

The second finding is that conventional local planners can be used on quadrupeds through a hierarchical interface, but their behavior depends strongly on how the planner handles clutter. Chapter 3 compared DWA, PF, and MPC on a Unitree A1 quadruped in BARN environments, in both simulation and hardware. The reduced planning interface made it possible to test these planners under a common setup. The results showed that MPC could produce efficient trajectories under the tested formulation, but it was less consistent in clutter and had more variable computation time. DWA and PF were more consistent, and PF performed well on hardware because it was less dependent on accurate velocity estimation. This chapter provided a baseline rather than a new stepping planner. Its value in the dissertation is that it made the limits of the standard local-planning setup concrete: the robot can be controlled through a simple model, but a planner that only avoids obstacles cannot use the full set of actions available to a legged system.

The third finding is that stepping can be added to a sampling-based local planner without making every planning step a full-body optimization problem. Chapter 4 introduced BiKE-Legs and DRiVE-Legs, which augment bicycle and differential-drive kinematic models with leg-based stepping actions. These models were embedded in DWA, so the planner could sample candidate motions that included both avoidance and stepping behavior. A constrained inverse-dynamics controller then tracked the resulting center-of-mass velocities on a full humanoid model. This approach kept the fast sampling structure of DWA while adding a discrete contact action that conventional local planners do not represent. The chapter also showed the main limitation of a planar stepping-augmented model. A step that is reasonable in 2D may still fail on the full robot if the swing leg does not have clearance, if the foothold is outside kinematic reach, or if the motion creates a stability problem. The result is not that reduced-order stepping models are sufficient by themselves. The result is that they are useful if they are paired with additional feasibility checks and a controller that can reject or correct infeasible commands.

Rather than extending Chapter 4's stepping capability, Chapter 5 returned to the fixed-weight limitation of the Chapter 3 baseline. The fourth finding is that adaptive DWA weight selection can be made interpretable without giving up its performance benefit, and that the interpretability itself has diagnostic value. Chapter 5 addressed this limitation of DWA, its fixed objective weights. A single weight setting can work in one type of local geometry and fail in another, since open regions may benefit from speed-focused weights while narrow passages need stronger obstacle clearance. The chapter used an RL policy to select among a small set of predefined weight profiles, then distilled that policy into a shallow decision tree (DST-DWA) that replaces the neural network with explicit feature-threshold rules at evaluation time. In the 2D simulation study, adaptive selection improved closed-loop success over fixed weights, and the distilled tree reproduced the neural policy's behavior while remaining fully rule-based. When the planner was transferred unchanged to a quadruped in MuJoCo, the advantage kept its direction but was not statistically confirmed, and a margin decomposition traced what gain there was to rescuing environments where fixed weights fail rather than to

greater efficiency where both planners already succeed. The interpretability mattered most at this weaker result. Because the tree's decisions are logged as explicit modes, the dominant failure could be read off directly as over-caution in the obstacle-safe profile.

6.2. Answers to the Research Questions

The first research question asked how reduced-order, data-driven step-to-step models can support computationally tractable stepping decisions for underactuated bipedal systems. Chapter 2 answered this by combining partial feedback linearization to reduce the state dimension, Gaussian process regression to learn the step-to-step map from simulation data, and Gaussian process classification to mark the feasibility boundary so the optimizer does not freely explore infeasible regions. Together these three components made multi-step optimization tractable without requiring an analytic model of the full nonlinear biped dynamics. The same principle recurs in the later chapters: plan with a reduced model that is cheap to evaluate repeatedly, and include an explicit feasibility boundary. The form of the reduced model changes with the robot and the decision level.

The second research question asked what tradeoffs distinguish DWA, PF, and MPC on a quadruped platform. Chapter 3 showed that the best planner depends on the failure mode that matters most. MPC can produce efficient paths, but in the tested setup it was less robust in clutter and had higher computation variability. DWA and PF were more consistent under the same hierarchical interface. PF performed especially well in the hardware tests because it did not depend as strongly on velocity estimation. These results do not mean that one planner is always better. They show that a planner selected for legged navigation must be judged by more than path length or nominal optimality. Real-time consistency, sensitivity to state estimation, and behavior near obstacles can dominate the practical result.

The third research question asked how much traversal capability stepping-enhanced local planners add in obstacle fields. Chapter 4 showed that adding stepping actions gave the planner a larger local action set. The planner no longer had to choose only among paths that route around the obstacle. It could also evaluate candidate motions that step over an object

when that was consistent with the reduced model. This could reduce unnecessary detours and made the planner better matched to a legged robot than a footprint-only avoidance model. The answer remained incomplete because the stepping formulation did not fully model 3D clearance, foot geometry, reachability, or stability. The added capability was therefore real, but the planner needed stronger feasibility filtering before it could be treated as a complete traversal planner.

The fourth research question asked whether adaptive DWA weight tuning can keep the performance of a learned policy while remaining interpretable. In the 2D BARN setup of Chapter 5, the answer is yes: the distilled decision tree matched, and on the held-out test split slightly exceeded, the neural policy's closed-loop success rate while replacing it with explicit feature-threshold rules. On the MuJoCo quadruped the answer is more guarded. The tree kept a directional advantage over fixed weights, but the effect was statistically borderline at 300 paired trials, so the transfer supports the approach without confirming it. The claim is not that the tree is universally better than the neural policy. The claim is narrower: adaptive DWA can be made interpretable by constraining the policy output to a small set of weight profiles and distilling the learned selector into explicit rules, and that interpretability is what let the chapter trace its weakest result to over-caution in a specific region of the rule set.

6.3. Limitations

The work in this dissertation uses several simplifications. These simplifications were useful for isolating planning and control questions, but they also limit what can be claimed.

First, the work does not solve perception. The quadruped and humanoid studies primarily evaluate planning and control with known obstacle information or structured benchmark environments. This makes the experiments repeatable and keeps the focus on local-planner behavior, but it avoids the errors that would come from onboard mapping, object segmentation, terrain estimation, or partial observability. A planner that works with a clean map may behave differently when obstacle locations are delayed, noisy, or incomplete.

Second, most reduced-order models used here are planar. This is appropriate for studying local navigation decisions, but it does not capture all constraints that determine whether a legged robot can execute a motion. Vertical clearance, foot orientation, terrain height, joint limits, friction, contact timing, and stability margins can all change the feasibility of a stepping action. The humanoid chapter makes this limitation visible: adding legs to the kinematic model creates useful candidate actions, but those actions need more filtering before they can be executed reliably on the full robot.

Third, the experiments are limited in scale and platform coverage. The biped study uses a planar model. The quadruped study includes simulation and hardware, but it evaluates conventional local planners rather than traversal-aware planning on hardware. The humanoid stepping study is simulation-based. The DST-DWA study is also simulation-only: its strongest results come from a custom 2D simulator, and the MuJoCo quadruped transfer used one trial per environment, so the observed effect remained statistically borderline rather than confirmed. These choices are reasonable for a dissertation that builds from modeling to planning to adaptation, but they mean the results should not be read as a complete field deployment pipeline.

Fourth, the benchmark environments simplify the world. BARN-style environments are useful because they provide repeatable obstacle fields and allow planners to be compared under the same conditions. They do not include all of the contact, terrain, compliance, and perception problems that appear in buildings or outdoor settings. The benchmark results therefore measure controlled navigation performance, not complete autonomy in unstructured environments.

6.4. Future Work

The most direct next step is to complete the bridge from planar stepping planning to full-body feasibility. The stepping-enhanced planners should include explicit reachability checks, swing-leg clearance, foot geometry, contact timing, and stability constraints. These checks do not all need to be solved through a large optimization at every step. Some can be

implemented as fast filters on the candidate trajectories generated by the local planner. The important point is that a motion the reduced model treats as feasible may still be infeasible for the full robot, so the planner needs enough feasibility filtering to catch and reject those cases.

A second direction is hardware validation of traversal-aware local planning. The quadruped chapter already showed that conventional local planners can be placed above a real legged controller. The next step is to test whether a robot can choose between avoidance and traversal in physical clutter. This would require obstacles whose size and placement make both choices possible in different situations, along with measurements of time, path length, collisions, failed steps, and computation time. The hardware experiments should separate planner failure from perception and controller failure as much as possible, at least in the first version.

A third direction is to connect adaptive planner tuning with the stepping action set introduced in Chapter 4. BiKE-Legs and DRiVE-Legs embed stepping-over actions in a DWA-style cost function, but that cost function still uses fixed weights for goal progress, obstacle negotiation, and speed, the same limitation the Chapter 3 baseline had before Chapter 5 addressed it. The RL selection and decision-tree distillation approach used in Chapter 5 could be applied directly to this stepping-augmented planner, with stepping preference added as a term the policy learns to weight alongside clearance, heading, and speed. Combining the two extensions this way would produce a single planner that is both stepping-capable and adaptively and interpretably tuned, a planner that neither chapter alone produces. The same interpretability requirement should remain: if the planner changes behavior near an obstacle, the reason should be visible in the features and thresholds used to make the decision.

A fourth direction is to include perception without losing repeatability. The planner studies in this dissertation used external tracking, such as motion capture, or known obstacle information, which isolates planning and control from perception error. Future experiments should move to onboard sensing in stages, first depth or LiDAR for the planner input, then partially observed or changing obstacle maps. This staged approach would make it easier

to identify whether failures come from sensing, reduced-order planning, full-body control, or the interaction between them.

6.5. Closing Remarks

The main argument of this dissertation is that conventional local planners for legged robots have two related shortcomings. They typically restrict the action set to obstacle avoidance even when the robot has useful contact actions available, and their parameters are usually fixed constants that, when learning is used to adapt them instead, become difficult to inspect. At the same time, full-body reasoning cannot be pushed into every local planning decision without creating a computation problem. The work here uses reduced-order models as the middle layer for addressing both shortcomings. They make planning fast enough to run online, but they must be tied to feasibility models and controllers that reflect the robot's body. Where their parameters adapt at runtime, that adaptation must remain traceable to the local geometry that produced it.

Across the chapters, the reduced model changes: a learned step-to-step map for a biped, a unicycle abstraction for a quadruped, and leg-enhanced kinematic models for a humanoid. The final chapter keeps the same unicycle-style model as the quadruped and adds an interpretable adaptive weight selector on top of it, rather than extending the stepping capability introduced in Chapter 4. That change in direction is not a dropped thread. The two extensions studied separately in this dissertation, stepping actions and interpretable adaptive tuning, address different limitations of the same fixed-weight, avoidance-only baseline, and Chapter 4's stepping-augmented planner is itself still fixed-weight. Combining them, so that a stepping-capable planner also tunes its weights adaptively and interpretably, is the natural target the two branches point toward, left to future work.

The same principle remains. A planner for a legged robot should expose the choices that matter locally, including speed, clearance, goal progress, and stepping when the robot can use it. It should also keep those choices inspectable enough that a failure can be diagnosed. The

final chapter's mode-level failure diagnosis is an example of what that inspectability provides, and the future work that follows from these chapters continues in the same direction.

APPENDIX A

Copyright Permissions

In this appendix, we document the copyright status and required attribution for the material adapted from each published or submitted manuscript used in this thesis.

Chapter 2: Stepping Control for Bipedal Walking via Partial Feedback Linearization and Gaussian Process Regression.

This chapter is adapted, with permission, from D. Torres, E. Hernandez Hinojosa, and P. A. Bhounsule, "Control of a Bipedal Walking Using Partial Feedback Linearization and Gaussian Process Regression-Based of the Step-to-Step Map," *Proceedings of the ASME 2023 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Volume 8: 47th Mechanisms and Robotics Conference (MR), V008T08A063, Boston, Massachusetts, USA, August 20–23, 2023. `https://doi.org/10.1115/DETC2023-116945`. © 2023 ASME. Under ASME's author-retained-rights policy, authors may reproduce and include the paper, in full or in part, in a thesis or dissertation for non-commercial purposes, provided ASME is acknowledged as the original publisher and the paper title and author names are included alongside the copyright notice above; no separate formal permission license is required for this reuse. This policy is stated on ASME's Copyright Guidelines page for conference proceedings.

Chapter 3: Comparative Study of Local Planners for Quadrupeds Using BARN Environments.

This chapter is adapted from the accepted manuscript D. Torres and P. A. Bhounsule, "Comparative Study of Local Planners for Quadrupeds Using BARN Environments," accepted to the 6th Modeling, Estimation and Control Conference (MECC 2026), Phoenix,

AZ, October 25–28, 2026. Copyright is retained by the authors, with IFAC holding exclusive publishing rights. Under the IFAC author agreement, authors may include the article in a thesis or dissertation without a separate formal permission request. Once published, the article will appear in IFAC-PapersOnLine under the Creative Commons CC BY-NC-ND 4.0 license. As of the submission of this thesis, the manuscript had been accepted but not yet published; no DOI or final copyright/license statement had been assigned at that time, and the version reproduced here is the authors' accepted manuscript. This policy is stated on IFAC's Copyright Conditions page: `https://www.ifac-control.org/publications/copyright-conditions`.

Chapter 4: Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments.

This chapter is adapted from the accepted manuscript D. Torres and P. A. Bhounsule, "Constrained Full-Body Inverse Dynamics Enables Leg-Enhanced Kinematic Models for Obstacle Avoidance and Stepping Over in Cluttered Environments," accepted to the 6th Modeling, Estimation and Control Conference (MECC 2026), Phoenix, AZ, October 25–28, 2026. Copyright is retained by the authors, with IFAC holding exclusive publishing rights. Under the IFAC author agreement, authors may include the article in a thesis or dissertation without a separate formal permission request. Once published, the article will appear in IFAC-PapersOnLine under the Creative Commons CC BY-NC-ND 4.0 license. As of the submission of this thesis, the manuscript had been accepted but not yet published; no DOI or final copyright/license statement had been assigned at that time, and the version reproduced here is the authors' accepted manuscript. This policy is stated on IFAC's Copyright Conditions page: `https://www.ifac-control.org/publications/copyright-conditions`.

Bibliography

[1] H. Berti, A. D. Sappa, and O. E. Agamennoni, "Improved dynamic window approach by using Lyapunov stability criteria,", Latin American Applied Research, vol. 38, no. 4, pp. 289–298, 2008.

[2] Pranav A. Bhounsule, Daniel Torres, Ernesto Hernandez Hinojosa, and Adel Alaeddini, "Task-Level Control and Poincaré Map-Based Sim-to-Real Transfer for Effective Command Following of Quadrupedal Trot Gait,", 2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids), pp. 1-8, 2023.

[3] Lu Chang, Liang Shan, Chao Jiang, and Yuewei Dai, "Reinforcement based mobile robot path planning with improved dynamic window approach in unknown environment,", Autonomous Robots, vol. 45, pp. 51–76, 2021.

[4] Matej Dobrevski and Danijel Skočaj, "Adaptive Dynamic Window Approach for Local Navigation,", 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 6930–6936, 2020.

[5] Dieter Fox, Wolfram Burgard, and Sebastian Thrun, "The dynamic window approach to collision avoidance,", IEEE Robotics and Automation Magazine, vol. 4, no. 1, pp. 23-33, 1997.

[6] Steven M. LaValle, "Planning Algorithms,", Cambridge University Press, 2006.

[7] Daniel Perille, Abigail Truong, Xuesu Xiao, and Peter Stone, "Benchmarking Metric Ground Navigation,", 2020 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR), pp. 116–121, 2020.

[8] Shinya Yasuda, Taichi Kumagai, and Hiroshi Yoshida, "Safe and Efficient Dynamic Window Approach for Differential Mobile Robots With Stochastic Dynamics Using Deterministic Sampling,", IEEE Robotics and Automation Letters, vol. 8, no. 5, pp. 2614–2621, 2023.

[9] Lunxin Zhong, Xiaogang Tang, Litian Liu, Guangyu Yang, KeFeng Guo, and HaoPeng Sun, "Hybrid Path Planning Algorithm Based on Improved Dynamic Window Approach,", 2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE), pp. 57–62, 2021.

[10] Omer Ali Abubakr, Mohammad A. Jaradat, and Mamoun F. Abdel-Hafez, "Intelligent optimization of adaptive dynamic window approach for mobile robot motion control using fuzzy logic,", IEEE Access, vol. 10, pp. 119368–119378, 2022.

[11] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador García, Sergio Gil-López, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera, "Explainable Artificial Intelligence (XAI): Concepts, Taxonomies, Opportunities and Challenges Toward Responsible AI,", Information Fusion, vol. 58, pp. 82–115, 2020.

[12] Pranav A. Bhounsule, Ali Zamani, Jeremy Krause, Steven Farra, and Jason Pusey, "A sampling-based approach to find controls policies for large region of attraction for legged systems,", Robotica, 2018.

[13] Pranav A Bhounsule, Myunghee Kim, and Adel Alaeddini, "Approximation of the step-to-step dynamics enables computationally efficient and fast optimal control of legged robots,", International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, pp. V010T10A050, 2020.

[14] Pranav A Bhounsule, Ernesto Hernandez Hinojosa, and Adel Alaeddini, "One-Step Deadbeat Control of a 5-Link Biped Using Data-Driven Nonlinear Approximation of the Step-to-Step Dynamics,", Robotics, vol. 9, pp. 90, 2020.

[15] Pranav A Bhounsule and Chun-Ming Yang, "A simple controller for omnidirectional trotting of quadrupedal robots: Command following and waypoint tracking,", Robotics, vol. 12, no. 2, pp. 35, 2023.

[16] Daniel Torres, "Stepping Stones,", `https://www.youtube.com/watch?v=p2TLU3Jzul8`, 2023.

[17] Leo Breiman, Jerome Friedman, Richard A. Olshen, and Charles J. Stone, "Classification and Regression Trees,", Chapman and Hall/CRC, 2017.

[18] Oliver Brock and Oussama Khatib, "High-speed navigation using the global dynamic window approach,", Proceedings 1999 ieee international conference on robotics and automation (Cat. No. 99CH36288C), pp. 341–346, 1999.

[19] Sean Campbell, Niall O'Mahony, Anderson Carvalho, Lenka Krpalkova, Daniel Riordan, and Joseph Walsh, "Path Planning Techniques for Mobile Robots: A Review,", 2020 6th International Conference on Mechatronics and Robotics Engineering (ICMRE), pp. 12–16, 2020.

[20] Guillermo A Castillo, Bowen Weng, Wei Zhang, and Ayonga Hereid, "Robust feedback motion policy design using reinforcement learning on a 3d digit bipedal robot,", 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 5136–5143, 2021.

[21] Zhiming Chen, Ze Wang, Miao Wu, Hua Chen, and Wei Zhang, "Improved dynamic window approach for dynamic obstacle avoidance of quadruped robots,", IECON 2020 The 46th Annual Conference of the IEEE Industrial Electronics Society, pp. 2780–2785, 2020.

[22] C. Chevallereau, J.W. Grizzle, and C.L. Shih, "Asymptotically stable walking of a five-link underactuated 3-D bipedal robot,", IEEE Transactions on Robotics, vol. 25, no. 1, pp. 37–50, 2009.

[23] Márcia M. Costa and Manuel F. Silva, "A Survey on Path Planning Algorithms for Mobile Robots,", 2019 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC), pp. 1–7, 2019.

[24] Thi-Kien Dao, Truong-Giang Ngo, Jeng-Shyang Pan, Thi-Thanh-Tan Nguyen, and Trong-The Nguyen, "Enhancing path planning capabilities of automated guided vehicles in dynamic environments: multi-objective PSO and dynamic-window approach,", Biomimetics, vol. 9, no. 1, pp. 35, 2024.

[25] Alessandro De Luca, Giuseppe Oriolo, and Marilena Vendittelli, "Control of Wheeled Mobile Robots: An Experimental Overview,", RAMSETE: Articulated and Mobile Robotics for Services and Technologies, pp. 181–226, 2001.

[26] Pedro A. Dias, João Pedro Carvalho de Souza, E. J. Solteiro Pires, Vítor Filipe, Daniel Figueiredo, Luís F. Rocha, and Manuel F. Silva, "Robot Path Planning: From Analytical to Computer Intelligence Approaches,", Journal of Intelligent & Robotic Systems, vol. 111, no. 4, pp. 112, 2025.

[27] Matej Dobrevski and Danijel Skocaj, "Adaptive dynamic window approach for local navigation using deep reinforcement learning,", IEEE Access, vol. 8, pp. 167885–167897, 2020.

[28] Hugh Durrant-Whyte and Tim Bailey, "Simultaneous Localization and Mapping: Part I,", IEEE Robotics & Automation Magazine, vol. 13, no. 2, pp. 99–110, 2006.

[29] Mohamed Elbanhawi and Milan Simic, "Sampling-Based Robot Motion Planning: A Review,", IEEE Access, vol. 2, pp. 56–77, 2014.

[30] Farbod Farshidian, Edo Jelavic, Asutosh Satapathy, Markus Giftthaler, and Jonas Buchli, "Real-time motion planning of legged robots: A model predictive control approach,", 2017 IEEE-RAS 17th International Conference on Humanoid Robotics (Humanoids), pp. 577–584, 2017.

[31] J. L. Fernández, R. Sanz, J. A. Benayas, and A. R. Diéguez, "Improving Collision Avoidance for Mobile Robots in Partially Known Environments: The Beam Curvature Method,", Robotics and Autonomous Systems, vol. 46, no. 4, pp. 205–219, 2004.

[32] Paolo Fiorini and Zvi Shiller, "Motion Planning in Dynamic Environments Using Velocity Obstacles,", The International Journal of Robotics Research, vol. 17, no. 7, pp. 760–772, 1998.

[33] Magnus Gaertner, Marko Bjelonic, Farbod Farshidian, and Marco Hutter, "Collision-free MPC for legged robots in static and dynamic scenes,", 2021 IEEE International Conference on Robotics and Automation (ICRA), pp. 8266–8272, 2021.

[34] Carlos E Garcia, David M Prett, and Manfred Morari, "Model predictive control: Theory and practice–A survey,", Automatica, vol. 25, no. 3, pp. 335–348, 1989.

[35] Kevin Green, John Warila, Ross L Hatton, and Jonathan Hurst, "Motion Planning for Agile Legged Locomotion using Failure Margin Constraints,", 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 10350–10355, 2022.

[36] J.W. Grizzle, G. Abba, and F. Plestan, "Asymptotically stable walking for biped robots: Analysis via systems with impulse effects,", IEEE Transactions on Automatic Control, vol. 46, no. 1, pp. 51–64, 2001.

[37] Peter E Hart, Nils J Nilsson, and Bertram Raphael, "A formal basis for the heuristic determination of minimum cost paths,", IEEE transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[38] Kris Hauser, Timothy Bretl, Jean-Claude Latombe, Kensuke Harada, and Brian Wilcox, "Motion planning for legged robots on varied terrain,", The International Journal of Robotics Research, vol. 27, no. 11-12, pp. 1325–1349, 2008.

[39] Andrei Herdt, Holger Diedam, Pierre-Brice Wieber, Dimitar Dimitrov, Katja Mombaur, and Moritz Diehl, "Online walking motion generation with automatic footstep placement,", Advanced Robotics, vol. 24, no. 5-6, pp. 719–737, 2010.

[40] Ayonga Hereid, Eric A Cousineau, Christian M Hubicki, and Aaron D Ames, "3D dynamic walking with underactuated humanoid robots: A direct collocation framework for optimizing hybrid zero dynamics,", 2016 IEEE International Conference on Robotics and Automation (ICRA), pp. 1447–1454, 2016.

[41] Ernesto Hernandez-Hinojosa, Aykut Satici, and Pranav A Bhounsule, "Optimal Control of a 5-Link Biped Using Quadratic Polynomial Model of Two-Point Boundary Value Problem,", International Design Engineering Technical Conferences and Computers and Information in Engineering Conference, pp. V08BT08A005, 2021.

[42] Ernesto Hernandez Hinojosa and Pranav A Bhounsule, "Data-driven Identification of a Non-homogeneous Inverted Pendulum Model for Enhanced Humanoid Control,", 2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids), pp. 1–8, 2023.

[43] D.G.E. Hobbelen and M. Wisse, "Limit cycle walking,", Humanoid Robots Human-like Machines, pp. 277–294, 2007.

[44] Hosein Houshyari and Volkan Sezer, "A New Gap-Based Obstacle Avoidance Approach: Follow the Obstacle Circle Method,", Robotica, vol. 40, no. 7, pp. 2231–2254, 2021.

[45] Michael Hoy, Alexey S. Matveev, and Andrey V. Savkin, "Algorithms for Collision-Free Navigation of Mobile Robots in Complex Cluttered Environments: A Survey,", Robotica, vol. 33, no. 3, pp. 463–497, 2014.

[46] Weiwei Huang, Junggon Kim, and Christopher G Atkeson, "Energy-based optimal step planning for humanoids,", Robotics and Automation (ICRA), 2013 IEEE International Conference on, pp. 3124–3129, 2013.

[47] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter, "Learning agile and dynamic motor skills for legged robots,", Science Robotics, vol. 4, no. 26, pp. eaau5872, 2019.

[48] Edo Jelavic, Kaixian Qu, Farbod Farshidian, and Marco Hutter, "LSTP: Long short-term motion planning for legged and legged-wheeled systems,", IEEE Transactions on Robotics, vol. 39, no. 6, pp. 4190–4210, 2023.

[49] Sertac Karaman and Emilio Frazzoli, "Sampling-Based Algorithms for Optimal Motion Planning,", The International Journal of Robotics Research, vol. 30, no. 7, pp. 846–894, 2011.

[50] Lydia E Kavraki, Petr Svestka, J-C Latombe, and Mark H Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces,", IEEE transactions on Robotics and Automation, vol. 12, no. 4, pp. 566–580, 2002.

[51] Martin Keller, Frank Hoffmann, Carsten Hass, Torsten Bertram, and Alois Seewald, "Planning of Optimal Collision Avoidance Trajectories with Timed Elastic Bands,", IFAC Proceedings Volumes, vol. 47, no. 3, pp. 9822–9827, 2014.

[52] Oussama Khatib, "Real-time obstacle avoidance for manipulators and mobile robots,", The international journal of robotics research, vol. 5, no. 1, pp. 90–98, 1986.

[53] Mykel J. Kochenderfer, "Decision Making Under Uncertainty: Theory and Application,", MIT Press, 2015.

[54] Jeremy Krause, Adel Alaeddini, and Pranav A. Bhounsule, "Gaussian Process Regression for Sim-to-Real Transfer of Hopping Gaits,", ASME-International Design Engineering & Technical Conference, 2023.

[55] James J. Kuffner, Satoshi Kagami, Koichi Nishiwaki, Masayuki Inaba, and Hirochika Inoue, "Dynamically-stable motion planning for humanoid robots,", Autonomous Robots, vol. 12, no. 1, pp. 105–118, 2002.

[56] Scott Kuindersma, Robin Deits, Maurice Fallon, Andrés Valenzuela, Hongkai Dai, Frank Permenter, Twan Koolen, Pat Marion, and Russ Tedrake, "Optimization-based locomotion planning, estimation, and control design for the atlas humanoid robot,", Autonomous robots, vol. 40, no. 3, pp. 429–455, 2016.
[57] A.D. Kuo, "Energetics of actively powered locomotion using the simplest walking model,", Journal of Biomechanical Engineering, vol. 124, pp. 113-120, 2002.
[58] Steven M. LaValle, "Rapidly-exploring random trees: A new tool for path planning,", Research Report 9811, Department of Computer Science, Iowa State University, 1998.
[59] Xiuyun Li, Fei Liu, Juan Liu, and Shan Liang, "Obstacle Avoidance for Mobile Robot Based on Improved Dynamic Window Approach,", Turkish Journal of Electrical Engineering & Computer Sciences, vol. 25, pp. 666–676, 2017.
[60] Yue Li, Jianyou Zhao, Zenghua Chen, Gang Xiong, and Sheng Liu, "A robot path planning method based on improved genetic algorithm and improved dynamic window approach,", Sustainability, vol. 15, no. 5, pp. 4656, 2023.
[61] Lixing Liu, Xu Wang, Xin Yang, Hongjie Liu, Jianping Li, and Pengfei Wang, "Path Planning Techniques for Mobile Robots: Review and Prospect,", Expert Systems with Applications, vol. 227, pp. 120254, 2023.
[62] Tomás Lozano-Pérez, "Spatial Planning: A Configuration Space Approach,", IEEE Transactions on Computers, vol. C-32, no. 2, pp. 108–120, 1983.
[63] Wangtao Lu, Yufei Wei, Jiadong Xu, Wenhao Jia, Liang Li, Rong Xiong, and Yue Wang, "Reinforcement Learning for Adaptive Planner Parameter Tuning: A Perspective on Hierarchical Architecture,", arXiv preprint arXiv:2503.18366, 2025.
[64] Yuanjie Lu, Beichen Wang, Zhengqi Wu, Yang Li, Xiaomin Lin, Chengzhi Mao, and Xuesu Xiao, "APPLV: Adaptive Planner Parameter Learning from Vision-Language-Action Model,", arXiv preprint arXiv:2603.08862, 2026.
[65] T. McGeer, "Passive dynamic walking,", The International Journal of Robotics Research, vol. 9, no. 2, pp. 62–82, 1990.
[66] T. McGeer, "Passive dynamic biped catalogue,", In Proc. of 2nd International Symposium on Experimental Robotics, pp. 465–490, 1991.
[67] Tad McGeer, "Dynamics and control of bipedal locomotion,", Journal of Theoretical Biology, vol. 163, no. 3, pp. 277–314, 1993.
[68] Marcell Missura and Sven Behnke, "Efficient Kinodynamic Trajectory Generation for Wheeled Robots,", 2011 IEEE International Conference on Robotics and Automation, pp. 4883–4888, 2011.
[69] Christoph Molnar, "Interpretable Machine Learning: A Guide for Making Black Box Models Explainable,", 2nd ed., `https://christophm.github.io/interpretable-ml-book/`, 2022.
[70] J. Morimoto, J. Nakanishi, G. Endo, G. Cheng, C. G. Atkeson, and G. Zeglin, "Poincaré-map-based reinforcement learning for biped walking,", Proceedings of International Conference on Robotics and Automation, Barcelona, Spain, pp. 2381–2386, 2005.
[71] Jun Morimoto and Christopher G Atkeson, "Nonparametric representation of an approximated poincaré map for learning biped locomotion,", Autonomous robots, vol. 27, pp. 131–144, 2009.
[72] Joseph Norby and Aaron M Johnson, "Fast global motion planning for dynamic legged robots,", 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 3829–3836, 2020.
[73] Petter Ogren and Naomi Ehrich Leonard, "A convergent dynamic window approach to obstacle avoidance,", IEEE Transactions on Robotics, vol. 21, no. 2, pp. 188–195, 2005.
[74] Chinonso Ovuegbe, "Parameter Selection in the Dynamic Window Approach Robot Collision Avoidance Algorithm using Bayesian Optimization,", The University of Texas at San Antonio, 2020.
[75] Xue Bin Peng, Glen Berseth, KangKang Yin, and Michiel Van De Panne, "Deeploco: Dynamic locomotion skills using hierarchical deep reinforcement learning,", Acm transactions on graphics (tog), vol. 36, no. 4, pp. 1–13, 2017.
[76] Chengyang Peng, Victor Paredes, and Ayonga Hereid, "Unified path and gait planning for safe bipedal robot navigation,", arXiv preprint arXiv:2403.17347, 2024.
[77] Chengyang Peng, Victor Paredes, Guillermo A Castillo, and Ayonga Hereid, "Real-time safe bipedal robot navigation using linear discrete control barrier functions,", 2025 IEEE International Conference on Robotics and Automation (ICRA), pp. 14903–14909, 2025.

[78] Jerry Pratt, John Carff, Sergey Drakunov, and Ambarish Goswami, "Capture point: A step toward humanoid push recovery,", 2006 6th IEEE-RAS international conference on humanoid robots, pp. 200–207, 2006.

[79] J. Ross Quinlan, "Induction of Decision Trees,", Machine Learning, vol. 1, no. 1, pp. 81–106, 1986.

[80] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernest, and Noah Dormann, "Stable-Baselines3: Reliable Reinforcement Learning Implementations,", Journal of Machine Learning Research, vol. 22, no. 268, pp. 1–8, 2021.

[81] Sourav Raxit, Abdullah Al Redwan Newaz, Paulo Padrao, Jose Fuentes, and Leonardo Bobadilla, "BOW: Bayesian Optimization Over Windows for Motion Planning in Complex Environments,", IEEE Robotics and Automation Letters, 2025.

[82] Francisco Rubio, Francisco Valero, and Carlos Llopis-Albert, "A Review of Mobile Robots: Concepts, Methods, Theoretical Framework, and Applications,", International Journal of Advanced Robotic Systems, vol. 16, no. 2, 2019.

[83] Stuart J. Russell and Peter Norvig, "Artificial Intelligence: A Modern Approach,", 4th ed., Pearson, 2021.

[84] Andrei A. Rusu, Sergio Gomez Colmenarejo, Caglar Gulcehre, Guillaume Desjardins, James Kirkpatrick, Razvan Pascanu, Volodymyr Mnih, Koray Kavukcuoglu, and Raia Hadsell, "Policy Distillation,", arXiv preprint arXiv:1511.06295, 2015.

[85] Junli Ren, Yingru Dai, Bowen Liu, Pengwei Xie, and Guijin Wang, "Hierarchical Vision Navigation System for Quadruped Robots with Foothold Adaptation Learning,", Sensors, vol. 23, no. 11, 2023.

[86] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, "Proximal Policy Optimization Algorithms,", arXiv preprint arXiv:1707.06347, 2017.

[87] Jeonghee Seo, Eunsung Kim, and Andrew Jaeyong Choi, "LLM-DWA: a hybrid path planning framework combining large language models with the dynamic window approach,", Scientific Reports, vol. 16, no. 1, pp. 9898, 2026.

[88] Reid Simmons, "The Curvature-Velocity Method for Local Obstacle Avoidance,", Proceedings of IEEE International Conference on Robotics and Automation, vol. 4, pp. 3375–3382, 1996.

[89] Omar Souissi, Rabie Benatitallah, David Duvivier, Abdelhakim Artiba, Nicolas Belanger, and Pierre Feyzeau, "Path Planning: A 2013 Survey,", Proceedings of the 5th International Conference on Industrial Engineering and Systems Management (IESM), Rabat, Morocco, 2013.

[90] S.H. Strogatz, "Nonlinear dynamics and chaos,", Addison-Wesley Reading, 1994.

[91] Richard S. Sutton and Andrew G. Barto, "Reinforcement Learning: An Introduction,", 2nd ed., MIT Press, 2018.

[92] R. Tedrake, "LQR-Trees: Feedback motion planning on sparse randomized trees,", Proceedings of Robotics: Science and Systems (RSS), Zaragoza, Spain, 2009.

[93] Russ Tedrake, Ian R. Manchester, Mark Tobenkin, and John W. Roberts, "LQR-trees: Feedback motion planning via sums-of-squares verification,", The International Journal of Robotics Research, vol. 29, no. 8, pp. 1038–1052, 2010.

[94] Sangli Teng, Yukai Gong, Jessy W Grizzle, and Maani Ghaffari, "Toward safety-aware informative motion planning for legged robots,", arXiv preprint arXiv:2103.14252, 2021.

[95] Daniel Teso-Fz-Betono, Ekaitz Zulueta, Unai Fernandez-Gamiz, Aitor Saenz-Aguirre, and Reynaldo Martinez, "Predictive Dynamic Window Approach Development with Artificial Neural Fuzzy Inference Improvement,", Electronics, vol. 8, no. 9, pp. 935, 2019.

[96] Emanuel Todorov, Tom Erez, and Yuval Tassa, "MuJoCo: A physics engine for model-based control,", 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 5026–5033, 2012.

[97] Unitree-Robotics, "A1 More Dexterity, More Possibilities,", `https://www.unitree.com/products/a1/`, 2022.

[98] Austin Shih-Ping Wang, William Wei-Lun Chen, and Pei-Chun Lin, "Control of a 2-D bounding passive quadruped model with Poincaré map approximation and model predictive control,", 2016 International Conference on Advanced Robotics and Intelligent Systems (ARIS), pp. 1–6, 2016.

[99] Zizhao Wang, Xuesu Xiao, Garrett Warnell, and Peter Stone, "APPLE: Adaptive Planner Parameter Learning from Evaluative Feedback,", IEEE Robotics and Automation Letters, vol. 6, no. 4, pp. 7744–7749, 2021.

[100] Zizhao Wang, Xuesu Xiao, Bo Liu, Garrett Warnell, and Peter Stone, "APPLI: Adaptive Planner Parameter Learning from Interventions,", 2021 IEEE International Conference on Robotics and Automation (ICRA), pp. 6079–6085, 2021.

[101] E. R. Westervelt and J. W. Grizzle, "Design of asymptotically stable walking for a 5-link planar biped walker via optimization,", Proceedings 2002 IEEE International Conference on Robotics and Automation (Cat. No.02CH37292), pp. 3117–3122, 2002.

[102] Xuesu Xiao, Bo Liu, Garrett Warnell, Jonathan Fink, and Peter Stone, "APPLD: Adaptive Planner Parameter Learning from Demonstration,", IEEE Robotics and Automation Letters, vol. 5, no. 3, pp. 4541–4547, 2020.

[103] Xuesu Xiao, Zifan Xu, Saad Abdul Ghani, Aniket Datar, Daeun Song, Peter Stone, Amna Mazen, Kamyab Yazdipaz, Innocent Mateyaunga, Mariam Faied, Mohan Krishnan, Yuanjie Lu, Tong Xu, Nick Mohammad, Woosung Kim, Jon Reasoner, Rohan Chandra, and Nicola Bezzo, "Autonomous Ground Navigation in Highly Constrained Spaces: Lessons Learned From the Fourth BARN Challenge at ICRA 2025,", IEEE Robotics & Automation Magazine, vol. 33, no. 1, pp. 186–196, 2026.

[104] Zhaoming Xie, Glen Berseth, Patrick Clary, Jonathan Hurst, and Michiel van de Panne, "Feedback control for cassie with deep reinforcement learning,", 2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 1241–1246, 2018.

[105] Xiaobin Xiong, Jenna Reher, and Aaron D Ames, "Global position control on underactuated bipedal robots: Step-to-step dynamics approximation for step planning,", 2021 IEEE International Conference on Robotics and Automation (ICRA), pp. 2825–2831, 2021.

[106] Zifan Xu, Gauraang Dhamankar, Anirudh Nair, Xuesu Xiao, Garrett Warnell, Bo Liu, Zizhao Wang, and Peter Stone, "APPLR: Adaptive Planner Parameter Learning from Reinforcement,", 2021 IEEE International Conference on Robotics and Automation (ICRA), pp. 6086–6092, 2021.

[107] Wan Xu, Yuhao Zhang, Leitao Yu, Tingting Zhang, and Zhao Cheng, "A Local Path Planning Algorithm Based on Improved Dynamic Window Approach,", Journal of Intelligent & Fuzzy Systems, vol. 45, no. 3, pp. 4917–4933, 2023.

[108] Liwei Yang, Ping Li, Song Qian, He Quan, Jinchao Miao, Mengqi Liu, Yanpei Hu, and Erexidin Memetimin, "Path Planning Technique for Mobile Robots: A Review,", Machines, vol. 11, no. 10, pp. 980, 2023.

[109] Mohd. Nayab Zafar and J. C. Mohanta, "Methodology for Path Planning and Optimization of Mobile Robots: A Review,", Procedia Computer Science, vol. 133, pp. 141–152, 2018.

[110] Ali Zamani and Pranav A Bhounsule, "Nonlinear model predictive control of hopping model using approximate step-to-step models for navigation on complex terrain,", 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 3627–3632, 2020.

[111] Han-ye Zhang, Wei-ming Lin, and Ai-xia Chen, "Path Planning for the Mobile Robot: A Review,", Symmetry, vol. 10, no. 10, pp. 450, 2018.

[112] Dongyang Zhang, Zhenai Song, Feng Zhang, and Long Cui, "Improving the dynamic window approach for path planning in multi-legged robots,", 2024 2nd International Conference on Artificial Intelligence and Automation Control (AIAC), pp. 9–12, 2024.

Vita

DANIEL TORRES

Education

Ph.D., Mechanical Engineering, University of Illinois Chicago (UIC) Aug. 2026
Dissertation: *Reduced-Order Planning and Control for Legged Robot Navigation in Cluttered Environments*
Advisor: Dr. Pranav A. Bhounsule

B.S., Mechanical Engineering, University of Illinois Chicago (UIC) May 2020
Summa Cum Laude

Research Interests

Motion planning, legged locomotion, robotics, optimal control, interpretable learning, and sim-to-real transfer

Academic Experience

Graduate Research Assistant, Robotics and Motion Laboratory, UIC May 2021 – Aug. 2026

Graduate Research Assistant, Nanoscale Materials and Engineering, UIC Aug. 2020 – May 2021

Graduate Research Assistant, Nanoscale Materials Science and Engineering, UIC May 2020 – Aug. 2020

Research Intern, Computational Multiphase Transport Laboratory, UIC Jun. 2017 – Jul. 2017

Teaching Experience

- **ME 397: Senior Design II**, Graduate Teaching Assistant, Spring 2026
- **ME 411: Mechatronics I**, Graduate Teaching Assistant, Fall 2022 and Fall 2025
- **ME 347: Engineering Design and Graphics with Computer-Aided Design and Simulation**, Graduate Teaching Assistant, Fall 2025

Publications

- **Torres, D.**, & Bhounsule, P. A. (2026). Comparative study of local planners for quadrupeds using BARN environments. In *Proceedings of the 6th Modeling, Estimation and Control Conference (MECC 2026)*, Phoenix, AZ, October 25–28, 2026. Accepted.
- **Torres, D.**, & Bhounsule, P. A. (2026). Constrained full-body inverse dynamics enables leg-enhanced kinematic models for obstacle avoidance and stepping over in cluttered environments. In *Proceedings of the 6th Modeling, Estimation and Control Conference (MECC 2026)*, Phoenix, AZ, October 25–28, 2026. Accepted.
- Bhounsule, P. A., **Torres, D.**, & Echeveste, S. (2026). A low-cost, scalable mechatronics course centered on tinker-and-build laboratories and project-based learning. Paper presented at the *2026 ASEE Illinois–Indiana Sectional Conference*, Valparaiso University, Indiana. `https://doi.org/10.18260/1-2-1125-58094`
- **Torres, D.**, Hernandez Hinojosa, E., & Bhounsule, P. A. (2023). Control of a bipedal walking using partial feedback linearization and Gaussian process regression-based of the step-to-step map. In *Proceedings of the ASME 2023 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*, Volume 8: 47th Mechanisms and Robotics Conference (MR) (p. V008T08A063). Boston, MA. `https://doi.org/10.1115/DETC2023-116945`
- Bhounsule, P. A., **Torres, D.**, Hinojosa, E. H., & Alaeddini, A. (2023). Task-level control and Poincaré map-based sim-to-real transfer for effective command following of quadrupedal trot gait. In *2023 IEEE-RAS 22nd International Conference on Humanoid Robots (Humanoids)*, pp. 1–8. Austin, TX. `https://doi.org/10.1109/Humanoids57100.2023.10375183`
- Hinojosa, E. H., **Torres, D.**, & Bhounsule, P. A. (2022). Quadratically constrained quadratic programs using approximations of the step-to-step dynamics: Application on a 2D model of Digit. In *2022 IEEE-RAS 21st International Conference on Humanoid Robots (Humanoids)*, pp. 96–103. Ginowan, Japan. `https://doi.org/10.1109/Humanoids53995.2022.10000251`

Selected Presentations

- **Torres, D.**, & Bhounsule, P. A. (2026). Community Hours: Reframing TA office hours to promote academic support and peer engagement. Poster presented at the ASEE Illinois–Indiana Section Conference.
- **Torres, D.**, & Bhounsule, P. A. (2024). Towards a unified motion planning framework for diverse robotic platforms: From simulations to real-world applications. Poster presented at the Midwest Robotics Workshop.
- **Torres, D.**, & Bhounsule, P. A. (2022). Standing balance controller for humanoid with Gaussian process-based learning for model errors, noise, and disturbances. Presented at Dynamic Walking 2022.

Honors and Awards

- UIC College of Engineering Graduate Student Award for Teaching Assistance Excellence, 2026
- National Science Foundation Graduate Research Fellowship, 2021

- Ford Foundation Predoctoral Fellowship, Honorable Mention, 2021
- NSF LSAMP Bridge to the Doctorate Fellowship, 2020

Selected Service

- Mentored undergraduate and precollege students in robotics, computer vision, and engineering research projects, 2022–2026.
- Conference Planning Committee Member, ASEE Illinois–Indiana Regional Conference, 2024.
- Vice President, Mechanical and Industrial Engineering Graduate Association, UIC, 2024–2025.